

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 18:52:26
Файлов исходного кода: 82
Суммарный объём кода: 1.12 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
```


- ```
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density}
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit
continue-on-error: true
  - name: Summary  
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full\_code\_all\_pages"
 echo "| \public/export/full\_code\_all\_pages"
 echo "| PNG-страниц | \$(ls tmp/export-pages
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7

```
 steps:
 - name: Checkout
 uses: actions/checkout@v4
+ uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
+ uses: actions/setup-node@v7
 with:
 node-version: "22"
+ node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
@@ -107,7 +107,7 @@ jobs:
 retention-days: 30

 - name: Upload artifacts (PNG-страницы)
 uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png

`index.html`
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

diff --git a/public/favicon.svg b/public/favicon.svg

new file mode 100644

index 00000000..3bbbba0

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
  <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

```

```

  {activeTab === "guide" ? (

```

```

    <>

```

```

      <MobileToc

```

```

        selectedChapterId={activeChapterId}

```

```

        setSelectedChapterId={goTo}

```

```

        currentTitle={activeChapter.title}

```

```

        index={index}

```

```

        total={chapters.length}

```

```

      />

```

```

    <div className="flex flex-col lg:flex-row max-w-7xl" >

```

```

      {/* Сайдбар только на десктопе – на мобильном скрывается */}

```

```

      <div className="hidden lg:block lg:w-80 shrink-0" >

```

```

        <Sidebar selectedChapterId={activeChapterId} />

```

```

      </div>

```

```

      <main id="main-content" className="flex-1 min-w-0" >

```

```

        {/* Шапка главы с быстрыми переходами */}

```

```

        <div className="flex items-center justify-between" >

```

```

          <div className="min-w-0" >

```

```

            <div className="text-[10px] uppercase" >

```

```

              {activeChapter.category || "Универсальное пособие"}

```

```

            </div>

```

```

            <h2 className="text-base sm:text-xl font-medium" >

```

```

              {activeChapter.title}

```

```

            </h2>

```

```

          </div>

```

```

          <ChapterNav prev={prev} next={next} index={index} />

```

```

        </div>

```

```

      <div className="h-1 w-full bg-slate-800 rounded" >

```

```

        <div

```

```

          className="h-full bg-gradient-to-r from-slate-800 to-slate-900"

```

```

          style={{ width: `${progress}%` }}

```

```

        />

```

Patch относительно `main`

```

`diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-   const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+   const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
   const [activeChapterId, setActiveChapterId] = useState<string>("");
   const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
    </>
-   ) : (
+   ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
-   ) : (
+   ) : (
+     <PythonCompiler />
  )}

  <SimulatorModal vizId={modalVizId} onClose={() => setModalVizId("")} />

```

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-900">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>

  <div className="flex items-center w-full lg:w-auto" >
    >
    <button
      id="tab-guide-btn"
      onClick={() => setActiveTab('guide')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <BookOpen className="w-4 h-4" /> Учебное пособие
    </button>
    <button
      id="tab-simulator-btn"
      onClick={() => setActiveTab('simulator')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
    <button
      id="tab-compiler-btn"
      onClick={() => setActiveTab('compiler')}
      className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
        activeTab === 'compiler'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
      }`}
    >
      <Code className="w-4 h-4" /> Компилятор
    </button>
  </div>
</div>
```

```
-----
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
    >
        {busy ? <Loader2 className="w-4 h-4 animate
    </button>
    </div>
  </div>
</header>
);
};
},

```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
 >
 <Cpu className="w-4 h-4" /> Тренажёры

```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {
 module as PyodideModule,
 }).then((module) => module.loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 }
 }, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
 <section className="rounded-2xl border border-slate-200 p-4">
 <div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center gap-2">


```

```

 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 p-4" >
 <label className="block px-4 pt-3 text-slate-500" >
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border border-slate-800"
 style={{border: '1px solid #333'}}
 />
 </div>
</section>

<section className="rounded-2xl border border-slate-800 p-4" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-2" >
 <Terminal className="w-4 h-4 text-emerald-500" />
 </div>

 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
 {runtimeReady ? "готов" : "не загружен"}

 </div>
 <pre className="min-h-[360px] max-h-[560px] w-full leading-6 text-slate-300" >
 {output}
 </pre>
 <div className="border-t border-slate-800 p-4" >
 </div>
</section>
</div>

```

```

+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+ const [output, setOutput] = useState("Нажмите «Запус
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">
+ <button
+ type="button"
+ onClick={reset}

```

```

+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border-slate-800 p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4">
+ <div className="flex items-center justify-between">
+ <div className="flex items-center gap-2">
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+
+ <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+ {runtimeReady ? "готов" : "не загружен"}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-2">
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+ <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+ Ctrl или Cmd + Enter запускает код. Имя файла не обязательно
+ </div>
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+ <Info className="w-4 h-4 shrink-0 text-amber-500" />
+ Имя файла обязательно
+ </div>
+ </div>

```

```
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
 },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|--------------------|
| --- | --- | --- |
| `npm run export:txt` | код → txt | `public/export/ful |
| `npm run export:png` | txt → png | `tmp/export-pages/ |
| `npm run export:pdf` | png → pdf | `public/export/ful |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`
* **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты/карты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация |
| ... | |

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте следующие теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы `Section` и `SectionTitle`.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика: Используют тег `

` и классы `Text` и `TextTitle`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|---------------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - |
| 22 | 22. Z-функция | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| - | Обзор: Кратчайшие пути и Графы | - | - | - |
| - | Алгоритмы на карте | - | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - | - |
| - | Мосты в графах: код + визуализация | 1 | - | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не...
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш...
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli...
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-50">
```

```
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
```

```
      {activeTab === "guide" ? (
```

```
        <>
```

```
          <MobileToc
```

```
            selectedChapterId={activeChapterId}
```

```
            setSelectedChapterId={goTo}
```

```
            currentTitle={activeChapter.title}
```

```
            index={index}
```

```
            total={chapters.length}
```

```
          />
```

```
        <div className="flex flex-col lg:flex-row max-w-7xl">
```

```
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
```

```
          <div className="hidden lg:block lg:w-80 shrink-0">
```

```
            <Sidebar selectedChapterId={activeChapterId}>
```

```
          </div>
```

```
          <main id="main-content" className="flex-1 min-w-0">
```

```
            {/* Шапка главы с быстрыми переходами */}
```

```
            <div className="flex items-center justify-between">
```

```
              <div className="min-w-0">
```

```
                <div className="text-[10px] uppercase">
```

```
                  {activeChapter.category || "Универсальное пособие"}>
```

```
                </div>
```

```
                <h2 className="text-base sm:text-xl font-medium">
```

```
                  {activeChapter.title}>
```

```
                </h2>
```

```
              </div>
```

```
              <ChapterNav prev={prev} next={next} index={index}>
```

```
            </div>
```

```
          <div className="h-1 w-full bg-slate-800 rounded">
```

```
            <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}])(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
            perTerm.set(id, usedTerm + 1);
            perSurface.set(surface, usedSurface + 1);
        }

        if (m.index > last) frag.appendChild(document.createElement("div"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("div"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
    const run = () => {
        const el = ref.current;
        if (!el) return;
        try {
            annotateTerms(el);
        }
    };
    useLayoutEffect(run, []);
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            зок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
метода: Split и Merge.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Sp
        </div>
        <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
                Аналогия 2: Слияние капель (M
            </div>
            <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
        </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
```

```

border-rose-500 shadow-md">
    Ты пытаешь
</div>
<p class="text-slate-300 text-sm mb">
<p class="text-slate-300 text-sm mb
p>
    <p class="text-slate-300 text-sm mb
на каждом шаге.</p>
    <p class="text-slate-300 text-sm mb
т кривым.</p>
    <p class="text-slate-300 text-sm mb
у делают уникальными (случайными).</p>
    <p class="text-slate-300 text-sm">.
ь детей merge за  $O(h)$ .</p>
</div>
</div>

<div class="bg-slate-900/60 p-4 rounded-lg">
    <p class="text-lg text-amber-300 italic">
    <table class="w-full text-left border-c
    <thead>
        <tr class="border-b border-slate
            <th class="py-2 px-3">Операт
            <th class="py-2 px-3">Средн
            <th class="py-2 px-3">Худши
        </tr>
    </thead>
    <tbody class="font-mono text-slate-1
        <tr class="border-b border-slate
0"> $O(\log n)$ </td><td class="py-2 px-3 text-r
        <tr class="border-b border-slate
-300"> $O(\log n)$ </td><td class="py-2 px-3 tex
        <tr class="border-b border-slate
3 text-emerald-300"> $O(\log n)$ </td><td class=
        <tr><td class="py-2 px-3">Постр
)</td><td class="py-2 px-3 text-rose-300">0
    </tbody>
</table>

```

```

else:
    root = merge(root.left, root.right)    # детей с
return root</pre>
    <p><b class="text-white">Сложность:
    троение –  $O(n \log n)$  в среднем (сортировка
    й любого узла – поэтому убрать можно произв
    </div>
</details>
</div>
</div>
</section>`,
},
{
    id: "splay-tree",
    title: "4. Splay-дерево",
    type: "html",
    description: "Дерево поиска, которое чинит само себя",
    category: "Продвинутые структуры",
    content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

    <div class="space-y-8">

        <!-- 0. Определение -->
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-lg sm:text-xl font-mono">
            поиска <span class="text-slate-500">без</span>
            : серия поворотов, которая поднимает v в кор
                <p class="text-sm text-slate-400 text-c">
            т – поворот.</p>
        </div>

```

```

<ul class="list-disc list-inside text-slate-300">
  <li><span class="text-white font-bold">Важно!</span>
    каждой вставки чинят дерево, чтобы оно <em>не</em>
  </li>
  <li><span class="text-white font-bold">Важно!</span>
    менно быть кривым. Но каждый раз, когда мы
    ровнее становится.</li>
</ul>
</div>

```

<!-- 2. Аналогии -->

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
    Аналогия 1: стопка бумаг на столе
  </div>
  <p class="text-slate-300 text-sm">У тебя
    скиваешь его и, поработав, кладёшь <span class="text-white font-bold">Важно!</span>
    неделю самые нужные бумаги сами собой оказыва
    ет ровно это: «взял → положил наверх».</p>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg border border-rose-500 shadow-md">
    Аналогия 2: тропинка через газон
  </div>
  <p class="text-slate-300 text-sm">Ты идёшь по дорожке, ко
    ода дорожка сама укорачивалась вдвое: чем чаще
    оплачивает все следующие. Пойдёшь один раз
  </p>
</div>

```

<!-- 3. Поворот -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-emerald-500 shadow-md">
  <h3 class="text-lg font-bold text-white mb-4">Поворот
  <p class="text-slate-300 text-sm mb-4">Поворот — это движение, три перевешенные ссылки</span> и

```

```

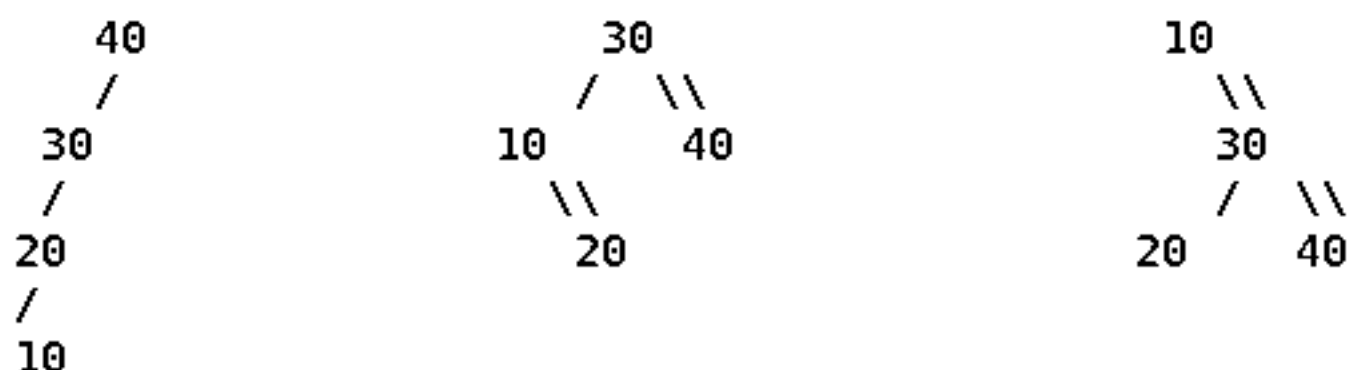
        </tr>
    </thead>
    <tbody class="text-slate-300">
        <tr class="border-b border-slate-200">
            <td class="py-3 pr-3 font-bolder">Зиг</td>
            <td class="py-3 pr-3">деда</td>
            <td class="py-3 pr-3">сам х</td>
            <td class="py-3">х стал королем</td>
        </tr>
        <tr class="border-b border-slate-200">
            <td class="py-3 pr-3 font-bolder">Пег</td>
            <td class="py-3 pr-3">х и п</td>
            <td class="py-3 pr-3">аво-право)</td>
            <td class="py-3">х поднялся</td>
        </tr>
        <tr>
            <td class="py-3 pr-3 font-bolder">Григ</td>
            <td class="py-3 pr-3">х и п</td>
            <td class="py-3 pr-3"><span class="font-bolder">Пег</span></td>
            <td class="py-3">п и г стали</td>
        </tr>
    </tbody>
</table>
</div>

<p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
<p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>
</div>

<!-- 5. Главная ловушка -->
<div class="bg-rose-950/30 p-6 rounded-xl border">
    <h3 class="text-lg font-bold text-rose-300">
    <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
    <div class="grid grid-cols-1 lg:grid-cols-2">

```

```
<p class="text-slate-300 text-sm mb-4">Дерево  
3 шага, а потом делаем Splay(10).</p>  
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-  
0 leading-relaxed">старт           после Zi  
        глубина 3 → 1           10 в корне
```



итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра  
инили дерево по пути</span>. Все узлы, мимо  
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl border-2  
<h3 class="text-lg font-bold text-white mb-3">  
<p class="text-slate-300 text-sm mb-3">Стро  
оддерева) по всем узлам, и Zig-Zig/Zig-Zag  
ах идея такая:</p>  
<div class="grid grid-cols-1 md:grid-cols-3  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">c  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">H  
  </div>  
  <div class="bg-slate-900 rounded-lg p-4  
    <p class="text-white font-bold mb-1">  
    <p class="text-slate-400 text-xs">д
```

```

-----
// один поворот: x встаёт на место своего родителя
function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                          // левый поворот (зеркало)
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                    // родитель уехал вниз
    x.parent = g;                    // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right = x; }
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                    // ZIG: деда нет
            rotate(x);
        } else if ((g.left === p) === (p.left === x)) {
            rotate(p);                // ZIG-ZIG: сносим
            rotate(x);
        } else {
            rotate(x);                // ZIG-ZAG: сносим
            rotate(x);
        }
    }
    return x;                        // теперь x — корень
}

// поиск: спустились и подняли найденное наверх

```

```

<details class="bg-slate-900/40 rounded-lg border-
  <summary class="p-4 font-bold text-slate-400
  open:border-b border-slate-700/50">
    Вопросы, которые задают на экзамене (
  </summary>
  <div class="p-5 text-sm text-slate-300 space-
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Последний
    енный или преемник). Splay делают всегда, да
    бы, и амортизация сломалась бы.</p>
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Потому что
    еворачивают бамбук в другую сторону (см. бл
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Чередовани
    вает один из них в корень, превращая дерево
    ступает только для <em>одной</em> конкретной
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Treap дер
    play не хранит вообще ничего и держит балан
    p>
    </div>
  </div>
</details>

</div>
</section>`,
  },
  {
    id: "prefix-sums-2d",
    title: "5. Двумерная матрица префиксных сумм",
  }

```



```

        content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-blue-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-blue-300 italic m
        <p class="text-xl font-mono text-white"
    ый граф можно раскрасить 4 цветами.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4 l
        rder border-emerald-500 shadow-md">
          Аналогия 1: Карта мира
        </div>
        <p class="text-slate-300 text-sm mb
      . Границы не пересекаются в одной точке по-
      ы не сливались цветами, Всегда можно всего
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        description: "Разбиение орграфа на макро-вершины. А
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        нвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            БЫХ направлениях по односторонним улицам. Как это отразится
            у на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреначивает граф",
        category: "Графы. Продвинутые",
        content: `

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
тей > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>
        <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-indigo-400

```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия: Копаем туннель под .
    </div>
    <p class="text-slate-300 text-sm mb-6">
      ней земли. Но если начать копать одновременно
      в 2 РАЗА меньше работы (геометрически: площ
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          (проверяем через DSU) - берем в ответ.</p>
      </div>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
      id: "mst-boruvka",
      title: "20. Графы. Остовное дерево. Борувка",
      type: "html",
      description: "",
      category: "Графы. Остовные",
      content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
ается с соседом.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
            Аналогия: Племена
          </div>
          <p class="text-slate-300 text-sm mb-6">
изкому племени для объединения. К вечеру пл

```

```

        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300
            <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

description: "",
category: "Теория сложности",
content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">Сложность
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Сложность
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-purple-400">Сведение
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Сведение (Reduction)
        <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс <b>NP</b> — это
          пример сортировка чисел). Класс <b>NP</b> — это
          енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
          </div>
          <p class="text-slate-300 text-sm mb-4">Это отличный переводчик на английский (Сведени
          аче B, то B — это <b>NP-Полная</b> (сосредо
        </div>
      </div>
    </div>
  </div>
`

```

ин раз. Вершины повторять можно.</p>

```
<p><strong class="text-white">Эйлер</strong>
<div class="p-3 bg-slate-950 rounded-lg border-rose-500 shadow-md">
  <p class="text-sm text-blue-300">
    <p><strong class="text-white">Путь:
    <p><strong class="text-white">Цикл:
  </div>
</div>
</div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла)
  <br>• <span class="text-green-400 font-weight-bold">шел-вошёл-вышел).
  <br>• <span class="text-yellow-400 font-weight-bold">финиш).
</p>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-950 border-rose-500 shadow-md">
      Путь: Нормальная прогулка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)</strong>.
      <strong>Теплый</strong> пришёл в бар <strong>(вторая нечётная)</strong>.
      Домой вернуться не смог, потому что
    </p>
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-rose-500 border-rose-500 shadow-md bg-slate-950">
    Цикл: Гамильтонов синдром (болезнь)
  </div>
  <p class="text-slate-300 text-sm mb-4">
```



```
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-4">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Представь, что ты альпинист. Ты спустился в пещеру.
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">tin[v]
        <p><strong class="text-white">low[v]
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font-mono text-sm">
            <p class="text-xl font-bold text-white">Мост: Единственная веревка
            <p class="text-xs text-slate-400">
          </div>
        </div>
      </div>
    </div>
```

```
<p class="text-slate-300 text-sm">
  Если из сына u и всех его потомков <strong>
  Единственная ниточка. Порвётся — граф р-связен.
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры u нет других выходов — то есть u — вершина.
      Если верёвка (ребро v-u) оборвётся — граф р-связен.
    </p>
  </div>
  <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спустился в пещеру.
      Из пещеры u нет других выходов — то есть u — вершина.
      Если верёвка (ребро v-u) оборвётся — граф р-связен.
    </p>
  </div>
</div>
```

```

int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                  // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `class="text-white">V=5, -white">10 > 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забу

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bord
```

```
<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6 xt-white">9 ≤ 12</code> – проходит, но он в`

Почему? Потому что у двудольного гр

Отсюда более строгое ограничение: <

```
<code class="text-white">9 &gt; 8</
```

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg bord
```

```
<summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скры

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 spac
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

Пусть К5 планарен. $V = 5$, $E = 10$.

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
  епятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соединены
      её перекроют (удалят вершину) – районы будут
      <strong>хрупкость</strong> системы.
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
      Телеком: Центральный свитч
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У тебя несколько компьютеров в одной
      абелем (мостом). Сами свитчи – это точки со
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
```

```

        <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
        <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
    </ul>
  </div>
</div>
</section>`,
  },
];

```

ФАЙЛ 19/82: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);

```

```
import type { DemoKind } from "../components/hints/MiniHints";

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» – 1–2 фразы. */
  short: string;
  /** Строгая формулировка / что важно не забыть. */
  formal?: string;
  complexity?: string;
  /** Какая мини-анимация показывается в карточке. */
  demo?: DemoKind;
  /** id полного симулятора (см. vizRegistry) – кнопка «симулятор» */
  simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину"],
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом соседи соседей и так далее.",
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину – если у неё есть неvisited соседи, добавляем их в очередь и помечаем visited.",
  },
];
```

```

    demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
    title: "Стек (LIFO)",
    short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
    formal: "push / pop / top за  $O(1)$ . На стеке живёт рекурсия",
    demo: "stack",
    simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот первым уходит",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS и поиска в ширину",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся множеств"],
    title: "DSU — система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Инициализация за  $O(n)$ ",
    complexity: "почти  $O(1)$  — обратная функция Аккермана",
    demo: "dsu",
    simulator: "mst-kruskal",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
    title: "Бор (trie, префиксное дерево)",
    short:

```

```

    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топосорт"],
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от н
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]
    complexity: "препроцессинг O(n), запрос O(1)",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},

```



```
{
  id: "articulation",
  labels: ["точка сочленения", "точки сочленения", "т"],
  title: "Точка сочленения",
  short:
    "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
  formal: "Для не-корня:  $v$  — точка сочленения. Это ребро",
  demo: "articulation",
  simulator: "graph-articulation",
},
{
  id: "mst",
  labels: [
    "остовное дерево", "остовного дерева", "остовное",
    "минимальное остовное дерево", "MST",
  ],
  title: "Минимальное остовное дерево (MST)",
  short:
    "Связать все точки проводами так, чтобы суммарная",
  formal: " $V-1$  ребро, никаких циклов. Краскал (жадно и",
    "но).",
  demo: "mst",
  simulator: "mst-kruskal",
},
{
  id: "amortized",
  labels: [
    "амортизированная", "амортизированный", "амортизи",
    "амортизированное", "амортизированного", "амортизи",
  ],
  title: "Амортизированная сложность",
  short:
    "Средняя цена операции «по абонементу»: одна опер.",
  formal: "Метод потенциалов: дорогая операция оплачив",
    "ассив.",
  demo: "amortized",
},
{
```

```

        "префикс", "префикса", "суффиксом", "суффикс", "с",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался",
        "от кусок.",
    formal:
        " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
        "возвращая указатель по тексту.",
    complexity: "O(n + m)",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "z-function",
    labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
    title: "Z-функция",
    short:
        "Для каждой позиции: сколько символов подряд, начинающихся",
        "с каждой буквы.",
    formal:
        " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
    complexity: "O(n)",
    demo: "prefix-function",
    simulator: "string-z-func",
},
{
    id: "dp",
    labels: [
        "динамическое программирование", "динамики", "динамическое",
        "мемоизация", "мемоизацию", "подзадач", "подзадачи",
    ],
    title: "Динамическое программирование",
    short:
        "Решаем задачу через ответы на её меньшие копии и",
    formal:
        "Нужны: оптимальная подструктура + перекрывающиеся подзадачи",

```

```

    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрываясь",
    formal:
        "Связный граф: эйлеров цикл    все степени чётные;
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного",
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе бумаги",
    formal:
        " $V - E + F = 2$  для связного планарного графа. Курсив",
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всё",
    ],
    title: "Splay и повороты",
    short:
        "Каждый раз, когда к узлу обратились, его «вытягивают»",
    formal:
        "Три случая: zig (родитель – корень), zig-zig (узлы в одну
        рацию.",
    demo: "amortized",
    simulator: "splay-tree",
}
```

```

    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину k",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки  
        шить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядк  
        ами.",
    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на кажд
    formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prima",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближай
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",

```

```

    short:
      "Точки (вершины) и связи между ними (рёбра). Города и дороги.
formal:
      "G = (V, E). Ребро бывает ориентированным (улицы с односторонним движением).
demo: "graph",
simulator: "graph-dfs-bfs",
},
{
  id: "coord-compress",
  labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
  title: "Сжатие координат",
  short:
    "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде
    порядковым номером.",
  formal:
    "sort + unique + binary search: значение → индекс",
  complexity: "O(n log n)",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова д-во"],
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету.
    Случайность и держит его сбалансированным.",
  formal:
    "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить
    два дерева через них. Ожидаемая высота O(log n).",
  complexity: "ожидаемо O(log n)",
  demo: "heap",
},
{
  id: "bst",
  labels: ["бинарное дерево поиска", "дерево поиска", "дерево поиска"],
  title: "Бинарное дерево поиска",
  short:

```

```

        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с"],
    title: "Строка как массив символов",
    short:
        "Строковые алгоритмы держатся на трёх вопросах: где
        (Z-функция) и где встречаются слова словаря (бордер)",
    formal:
        "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо"],
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт на
        дившемся месту. Порядок ключей при этом не ломается",
    formal:
        "Правый поворот вокруг p:  $x = p.left; p.left = x.$ 
        g и Zig-Zag.",
    complexity: " $O(1)$ ",
    demo: "zig",
    simulator: "splay-rotations",
},
{
    id: "zig",
    labels: ["Zig", "Zag", "зиг"],
    title: "Zig — одиночный поворот",
    short:

```

ФАЙЛ 21/82: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const graphChapters: Chapter[] = [
```

```
{
```

```
  id: "intro",
```

```
  title: "Обзор: Кратчайшие пути и Графы",
```

```
  type: "html",
```

```
  category: "Графы",
```

```
  content: `<section id="intro-content" class="mb-12`
```

```
    <div class="flex items-center mb-6">
```

```
      <span class="bg-indigo-600 text-white px-4 py-1`
```

```
      <h2 class="text-3xl font-bold text-white">Обзор`
```

```
    </div>
```

```
    <div class="space-y-8">
```

```
      <div class="bg-slate-700/50 p-6 rounded-xl border`
```

```
        <h3 class="text-xl font-bold text-blue-400`
```

```
        <p class="text-slate-300 text-sm mb-4">Пред`
```

```
        оезда). Алгоритмы поиска кратчайшего пути п`
```

```
        <div class="mt-8 mb-4">
```

```
          <div id="slot-mnemonic-cards"></div>
```

```
        </div>
```

```
      </div>
```

```
    </div>
```

```
  </section>`
```

```
},
```

```
{
```

```
  id: "dijkstra",
```

```
  title: "Билет 14. Дейкстра",
```

```
  type: "html",
```

```
  category: "Графы",
```

```
  content: `<section id="dijkstra-content" class="mb-
```

```

        <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
            Строгое доказательство (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-
            </p>
        </div>
    </details>

    <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
    id: "bellman-ford",
    title: "Билет 15. Форд-Беллман",
    type: "html",
    category: "Графы",
    content: `<section id="bellman-ford-content" class="flex items-center mb-6">
        <div class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-3xl font-bold text-white">Форд-Беллман
        </div>

        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-rose-400">Описание
            </div>

            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
                <p class="text-xl font-mono text-white">
            </div>

            <div class="grid grid-cols-1 xl:grid-cols-2">
                <!-- Аналогия 1 -->
                <div class="bg-slate-800 p-6 rounded-lg">

```



```

<h3 class="text-xl font-bold text-indigo-400">Аналогия 1: Автомагистрали</h3>

<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">Аналогия 1: Автомагистрали</p>
  <p class="text-xl font-mono text-white">Аналогия 1: Автомагистрали</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
  <!-- Аналогия 1 -->
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Автомагистрали
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Сколько времени потребуется самолету, чтобы долететь из Москвы в Париж, если мы сделаем
    </p>
    <div id="slot-chapter-image-floyd" class="h-400px">
      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
          Время работы
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Сколько времени потребуется самолету, чтобы долететь из Москвы в Париж, если мы сделаем
        </p>
      </div>
    </div>
  </div>
</div>

<div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "aho-corasick",
  title: "Билет 23. Алгоритм Ахо-Корасик",

```

```

        <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        терминальные ссылки</b> (term_link) – прямые мос
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
    <summary class="p-4 font-bold text-slate-400 outl
        pen:border-slate-700/50">
        Внутренняя структура автомата и код BFS (Скры
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
        <p>Каждая вершина имеет таблицу переходов <code>
            ссылку <code>term_link</code> (ближайшая мен
        <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
    map<char, int> go;          // Предвычисленные пер
    int link = 0;              // Суффиксная ссылка: куда
    int term_link = 0;         // Терминальная ссылка: матр
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {

```

```
        </div>
    </div>
</section>`
    }
};
```

ФАЙЛ 22/82: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 106 | РАЗМЕР: 1.2 КБ

```
export interface QuizQuestion {
    question: string;
    options: string[];
    correctIndex: number;
    explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
    "euler-path-vs-cycle": [
        {
            question: "В связном графе ровно 2 вершины с нечётной степенью, если и только если в графе есть цикл.",
            options: [
                "Есть эйлеров цикл (вернусь домой!)",
                "Есть эйлеров путь (из одной нечётной в другую)",
                "Ни пути, ни цикла нет — это полный провал"
            ],
            correctIndex: 1,
            explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных вершин больше 2 — эйлеров цикла нет, но путь может быть. Например, в графе с 3 нечётными вершинами (звезда K3) есть путь, но нет цикла."
        },
        {
            question: "Почему гамильтонов цикл — это NP-полная задача?",
            options: [
                "Потому что Гамильтон — болезнь, и лечить её тяжело.",
                "Потому что для эйлерова цикла есть простой критерий (степени всех вершин чётны).",
                "Потому что Эйлер был умнее Гамильтона"
            ],
            correctIndex: 2,
            explanation: "Задача нахождения гамильтонова цикла является NP-полной. Это означает, что не существует известного алгоритма, который решил бы эту задачу за полиномиальное время. В отличие от эйлерова цикла, для которого есть простой критерий проверки существования."
        }
    ],
    "hamiltonian-cycle": [
        {
            question: "Какая задача является NP-полной?",
            options: [
                "Нахождение эйлерова цикла",
                "Нахождение гамильтонова цикла",
                "Нахождение минимального остовного дерева"
            ],
            correctIndex: 1,
            explanation: "Нахождение гамильтонова цикла является NP-полной задачей. Нахождение эйлерова цикла — задача P, а нахождение минимального остовного дерева — задача P."
        }
    ],
    "graph-theory-basics": [
        {
            question: "Какая теорема утверждает, что в любом связном графе существует путь между любыми двумя вершинами?",
            options: [
                "Теорема Эйлера",
                "Теорема Гамильтона",
                "Теорема о связности графа"
            ],
            correctIndex: 2,
            explanation: "Теорема о связности графа утверждает, что в связном графе существует путь между любыми двумя вершинами. Теорема Эйлера касается циклов, а теорема Гамильтона — гамильтоновых циклов."
        }
    ]
}
```

```

    },
    {
      question: "Какова временная сложность алгоритма поиска мостов в графе?",
      options: [
        "O(V^2) – ищем мосты в квадрате",
        "O(V + E) – каждый элемент один раз",
        "O(2^V) – полный перебор"
      ],
      correctIndex: 1,
      explanation: "DFS обходит каждую вершину и каждое ребро."
    }
  ],
  "planarity-euler-formula": [
    {
      question: "Планарный граф имеет V=6, E=12. Возможно ли?",
      options: [
        "Да, если это двудольный граф",
        "Нет, нарушается неравенство E ≤ 3V - 6",
        "Да, если нет треугольных граней"
      ],
      correctIndex: 1,
      explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 – на грани возможности. Но граф с 6 вершинами и 12 ребрами не может быть планарным. Так что такой граф непланарен."
    },
    {
      question: "Сколько граней у планарного графа с V=7, E=10?",
      options: [
        "3 грани",
        "5 граней",
        "10 граней"
      ],
      correctIndex: 1,
      explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не 10."
    },
    {
      question: "Почему K5 непланарен?",
      options: [
        "Потому что 10 > 3*5 - 6 = 9 – нарушение",

```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-blue-300 italic">
    <p class="text-xl font-mono text-white">
      omise (lazy).</p>
    </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
      rder border-emerald-500 shadow-md">
      Аналогия 1: Корпоративная пре
    </div>
    <p class="text-slate-300 text-sm mb-4">
      ы рассылать 10 000 писем, он пишет одно пис
      счетах сотрудников только тогда, когда сотр
      женное обновление).</p>
    </div>
```

```
    <div class="bg-slate-900 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 l
        border-rose-500 shadow-md">
        Аналогия 2: Матрешки-коробки
      </div>
      <p class="text-slate-300 text-sm mb-4">
        ше, и так далее. Если нам нужно покрасить п
        Внутри всё синее!" на большую коробку. Опус
      </div>
    </div>
```

```
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate-300">
    -b border-slate-700/50">
    Строгое доказательство / Код (Скр
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded
```

```
function push(node) {
```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
        </div>
        <p class="text-slate-300 text-sm mb-2">
            зок длины 7. Ты берешь две заготовленные за
            ь отрезок длины 7 (перекрывая друг друга по
            ничего складывать, просто пересекаем куски.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
                Аналогия 2: Ступени
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ужно покрыть отрезок, мы берем две самые бо
            </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg">
            <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
            </div>
        </details>
    </div>
</div>
</section>`
},
{
    id: "treap",
    title: "3. Декартово дерево (Treap)",
    type: "html",
    description: "Дерево поиска по ключу и Куча по приоритету",
    category: "Продвинутые структуры",

```

```

        <div class="p-5 text-sm text-slate-300"
            <pre class="bg-slate-950 p-4 rounded-2xl text-white"
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}
}
</pre>
    </div>
</details>
</div>
</div>
</section>`
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
        type: "html",
        description: "Дерево поиска, которое самобалансируется",
        category: "Продвинутые структуры",
        content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">

```

```

    }
  }</pre>
</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">
  <h4 class="text-lg font-bold text-indigo-400">
    Корень

    <div class="space-y-4 text-sm text-slate-300">
      <div>
        <strong class="text-white block">
          корень?</strong>
        <p>Оно поднимет узел, на котором
          ска. За счет сдвига этого листа в корень, д
        /p>
      </div>

      <div>
        <strong class="text-white block">
          <p>Если агент постоянно переключ
            находящимися на разных концах дерева, его
            ащая дерево в вытянутый бамбук для другого.
            . Постоянное свинг-переключение превратит п
          </div>

          <div>
            <strong class="text-white block">
              <p>Хотя один поиск может стоять
                емах обладают прекрасным свойством: они не
                по пути. "Палочное" дерево прессуется в сб
                осов.</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
},

```


ФАЙЛ 24/82: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
```

```
{
```

```
  id: "dijkstra",
```

```
  title: "14. Графы. Поиск кратчайшего пути. Дейкстра
```

```
  type: "html",
```

```
  category: "Графы. Пути",
```

```
  content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
  </div>
```

```
  <div class="space-y-8">
```

```
    <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
      <h3 class="text-xl font-bold text-emerald-400">
```

```
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
        <p class="text-lg text-emerald-300 italic">
```

```
        <p class="text-xl font-mono text-white">
```

```
      </div>
```

```
      <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
        <!-- Аналогия 1 -->
```

```
        <div class="bg-slate-800 p-6 rounded-lg">
```

```
          <div class="absolute -top-3 left-4">
```

```
            <div class="border-emerald-500 shadow-md">
```

```
              Аналогия 1: Позитивное планирование
```

```
            </div>
```

```
            <p class="text-slate-300 text-sm mb-4">
```

```
              (нельзя поехать и заработать). Он всегда в
```

```
            </div>
```

```
        <!-- Аналогия 2 -->
```

```
        <div class="bg-slate-900 p-6 rounded-lg">
```

```

        берет ВСЕ возможные дороги V-1 раз подряд.
      </div>
      <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em;">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
          Отрицательный цикл
        </div>
        <p class="text-slate-300 text-sm mb-0">
          попали во временную петлю.</p>
      </div>
    </div>
    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
  },
  {
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы. Пути
    <h2 class="text-2xl sm:text-3xl font-bold text-white">16. Графы. Поиск кратчайшего пути. Флойд
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-indigo-400">16.1. Алгоритм Флойда-Уоршелла
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Алгоритм Флойда-Уоршелла — алгоритм поиска кратчайших путей между всеми парами вершин в взвешенном неориентированном графе.
        <p class="text-xl font-mono text-white">
          1. Инициализация: для каждой пары вершин (i, j) установить кратчайший путь как вес ребра между ними (или бесконечность, если ребра нет).
          2. Итерации: для каждой вершины k (промежуточная вершина) пройтись по всем парам (i, j) и проверить, можно ли улучшить текущий кратчайший путь из i в j, пройдя через k. Если да, обновить значение.
          3. Результат: после завершения всех итераций матрица содержит кратчайшие пути между всеми парами вершин.
        </p>
      </div>
    </div>
    <div class="bg-slate-800 p-6 rounded-lg border border-slate-900">
      <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">

```

```

    </div>
    <div class="grid grid-cols-1 gap-6 mb-6">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 border border-amber-500 shadow-md">
          ⚡ Суть перевзвешивания
        </div>
        <p class="text-slate-300 text-sm mb-6">
          Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать <b>Джонсона</b> пути остались теми же.
        </p>
        <ol class="text-sm text-slate-300 space-y-2">
          <li>Добавляем <b>фиктивную вершину</b>
          <li>Запускаем от неё <b>медленный поиск</b> «потенциалы» <code class="text-pink-400">
          <li>Меняем старые веса: новое расстояние = старое + потенциал
          <li><b>Магия!</b> Все веса теперь неотрицательные, потенциалы сокращаются).</li>
          <li>Теперь смело запускаем <b>Борзовского</b>
        </ol>
      </div>
    </div>
    <div id="slot-johnson-viz" class="mt-8"></div>
  </div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-6">
  <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
      которое торчит из посещенных в непосещенных.
    </p>
  </div>
  <div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        ✦ Аналогия: Заражение вирусом
      </div>
      <p class="text-slate-300 text-sm mb-6">
        ли вирус). Мы стартуем из одного города, и
        Сеть растет только из одного связного куска
      </p>
    </div>
  </div>
</div>
</section>`
},
{
  id: "mst-boruvka",
  title: "20. Графы. Остовное дерево. Борувка",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-6">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          ается с соседом.</p>

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
    сом, оканчивающимся в позиции i.</p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
    rder border-emerald-500 shadow-md">
        ♂ Аналогия 1: Поиск без отка
      </div>
      <p class="text-slate-300 text-sm mb"
        Мы идём по строке слева направо. К
        строки СЕЙЧАС закончился под моим пальцем?»
        Представь, ты ищешь в тексте слова
      </code>. Тупой алгоритм начнет искать заголо
      </code>, а это начало нашего слова! Не надо
    </p>
  </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
  border-rose-500 shadow-md">
      Аналогия 2: LLM стриминг
    </div>
    <p class="text-slate-300 text-sm mb"
      Для LLM, которая стримит токены
      каждом шаге. Если мы частично совпали с зап
      акого места этого слова мы можем продолжить
    </p>
  </div>
</div>

```

```

        </div>
      </details>
    </div>
  </div>
</section>`
},
{
  id: "string-z-func",
  title: "22. Z-функция — Сдвиг и наложение",
  type: "html",
  category: "Строки",
  content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
              <p class="text-xl font-mono text-white">
                са 0) и её суффиксом, начинающимся с i.</p>
            </div>

            <div class="grid grid-cols-1 xl:grid-cols-2">
              <!-- Аналогия 1 -->
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 l
                rder border-emerald-500 shadow-md">
                  Аналогия 1: Линейка-шаблон
                </div>
                <p class="text-slate-300 text-sm mb">
                  Тут мы берём всю строку и «прикр
                  я начну читать строку отсюда, насколько длин
                  Это самый быстрый способ найти
                >, считаешь Z-функцию, и там, где <code>Z[i

```

-b border-slate-700/50">

Код C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounded

```
int l = 0, r = 0;
```

```
for (int i = 1; i < n; i++) {
```

```
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
```

```
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
```

```
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
```

```
}</pre>
```

</div>

</details>

</div>

</div>

</section>

},

{

id: "aho-corasick",

title: "23. Алгоритм Ахо-Корасик",

type: "html",

category: "Строки",

content: `

<section id="aho-corasick" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-indigo-600 text-white px-4 py-1 rounded

<h2 class="text-2xl sm:text-3xl font-bold text-white

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl border-l

<h3 class="text-xl font-bold text-blue-400 mb-4">1

<div class="bg-slate-900 p-4 rounded-lg mb-6 text

<p class="text-sm text-blue-300 italic mb-2">Осн

<p class="text-lg font-mono text-white">Бор (Tr

</div>

<p class="text-slate-300 text-sm">Позволяет найти

```
int go(int v, char c) {
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
    }
    return t[v].go[c];
}
```

```
</div>
</details>
</div>
</section>`
},
{
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded">24</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Классы сложности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Если я умею решать задачу A, то я умею решать задачу B.</p>
        <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею решать задачу A, то я умею решать задачу B.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
        </div>
      </div>
    </div>
  </div>
`
}
```



```
-----
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      { /* База представлений */ }
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-slate-300">
        <div class="bg-slate-900 p-4 rounded-lg mb-1">
          <p class="text-lg text-slate-300 italic">
          <p class="text-xl font-mono text-white">
            (V + E).</p>
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
            rder border-emerald-500 shadow-md">
              Аналогия: Матрица = Таблица
            </div>
            <p class="text-slate-300 text-sm mb-1">
              глупо, но проверка "знает ли Вася Петю" мгно
            </div>
            <div class="bg-slate-900 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 border
              border-rose-500 shadow-md">
                Аналогия: Списки = Контакты
              </div>
              <p class="text-slate-300 text-sm mb-1">
                т. Оптимально для почти всех задач.</p>
            </div>
          </div>
        </div>
      </div>
      { /* DFS vs BFS */ }
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-indigo-400">
```

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate
-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounded

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

</pre>

<pre class="bg-slate-950 p-3 rounded

// BFS (Очередь)

queue<int> q;

q.push(start_node);

vis[start_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

}

}

</pre>

</div>

</details>

</div>

```

{
  id: "graph-components",
  title: "8. Графы. Компоненты связности",
  type: "html",
  description: "Нахождение кусков графа",
  category: "Графы. База",
  content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Нахождение кусков графа
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Граф — это совокупность вершин и ребер.
        <p class="text-xl font-mono text-white">Граф может состоять из нескольких компонент.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm mb-4">Если мы посмотрим на карту — остались ли серые (неисследованные) острова?
          <p class="text-slate-300 text-sm mb-4">Да, остались. И мы можем пройти через океан — столько и компонент.
        </div>
      </div>
    </div>
  </div>
</section>`,
},
{
  id: "top-sort",
  title: "9. Графы. Топологическая сортировка",

```

```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
    Душный мод: Код на C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
    <p class="text-indigo-400 font-bold">
        <div class="bg-slate-950 p-4 rounded">
            <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (post-order)
}

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем
}</pre>
            </div>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)"
    }
}

```

```

        <p class="text-slate-300 text-sm mb-4">
            Сильная связность — это "клуб" вершин, которые можно найти
            ни в одной SCC. Добавить курьера Гошу, который может выйти
            йти наружу (к директору), но обратно в клуб не может.
        </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md">
            Связь с Билетом 12
        </div>
        <p class="text-slate-300 text-sm mb-4">
            <b>SCC (Билет 10)</b> склеивает вершины в одну компоненту
        </p>
        <p class="text-slate-300 text-sm mb-4">
            <b>Точки сочленения (Билет 12)</b> — это вершины, удаление которых
            ость</strong> монолита. Вырвешь точку сочленения — разрушишь
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg">
    <summary class="p-4 font-bold text-slate-300">
        Душный мод: Код Алгоритм Косарайю
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
        <ol class="list-decimal list-inside">
            <li>Запускаем DFS на исходном графе, чтобы найти SCC.
        </li>
            <li>Разворачиваем этот список (то есть делаем reverse-DFS).
        </li>
            <li>Переворачиваем все ребра в графе.
        </li>
            <li>Идем по <code>order</code>: для каждой вершины v, пока не
        </li>
        </ol>
    </div>
</details>

```

```

{
  id: "graph-articulation",
  title: "12. Графы. Точки сочленения",
  type: "html",
  description: "Вершины, удаление которых убивает свя
  category: "Графы. Продвинутое",
  content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m

      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-rose-300 italic m
        <div class="text-left font-mono text-sm
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-40
        </div>
      </div>
    </div>
    <div class="p-3 bg-slate-950 rounde
      <p class="text-slate-300 text-sm">
        <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong
ег - это тупик со степенью 1).
      </p>
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap

```



```
</section>`,  
  },  
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 27/82: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";  
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {  
  id: number;  
  x: number;  
  y: number;  
  label: string;  
}
```

```
interface Edge {  
  u: number;  
  v: number;  
}
```

```
const nodes: Node[] = [  
  { id: 0, x: 250, y: 50, label: "0" },  
  { id: 1, x: 100, y: 150, label: "1" },  
  { id: 2, x: 250, y: 250, label: "2" },  
  { id: 3, x: 400, y: 150, label: "3" },  
  { id: 4, x: 550, y: 150, label: "4" },  
  { id: 5, x: 700, y: 50, label: "5" },  
  { id: 6, x: 700, y: 250, label: "6" },  
];
```



```

let timer = 0;
const visited = new Set<number>();
const ap = new Set<number>();
const currentTin = Array(nodes.length).fill(-1);
const currentLow = Array(nodes.length).fill(-1);

const recordStep = (desc: string, u: number | null) => {
  newSteps.push({
    desc,
    visited: new Set(visited),
    ap: new Set(ap),
    currentNode: u,
    tin: [...currentTin],
    low: [...currentLow],
  });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
  visited.add(v);
  currentTin[v] = currentLow[v] = timer++;
  let children = 0;

  recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

  for (const to of adj[v]) {
    if (to === p) continue;

    if (visited.has(to)) {
      currentLow[v] = Math.min(currentLow[v], currentLow[to]);
      recordStep(
        `Обратное ребро ${v}-${to}. Обновляем low[${v}]`
      );
    } else {
      children++;
      dfs(to, v);
    }
  }
};

```

```

    setSteps(newSteps);
  }, []));

const currentStep = steps[currentStepIndex] || {
  desc: "",
  visited: new Set(),
  ap: new Set(),
  currentNode: null,
  tin: [],
  low: [],
};

useEffect(() => {
  let interval: NodeJS.Timeout;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    interval = setInterval(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

```

```

        key={"edge-" + i}
        x1={u.x}
        y1={u.y}
        x2={v.x}
        y2={v.y}
        stroke={isBridge ? "#f43f5e" : "#475568"}
        strokeWidth={isBridge ? "4" : "2"}
        strokeDasharray={isBridge ? "8,4" : ""}
      />
    );
  }}}

```

```

{/* Nodes */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited.has(node.id);
  const isAp = currentStep.ap.has(node.id);

```

```

    const fill = isCurrent
      ? "#3b82f6"
      : isAp
        ? "#e11d48"
        : isVisited
          ? "#10b981"
          : "#1e293b";

```

```

    const stroke = isCurrent
      ? "#60a5fa"
      : isAp
        ? "#f43f5e"
        : isVisited
          ? "#34d399"
          : "#334155";

```

```

    const r = isAp ? 24 : 20;

```

```

    return (
      <g key={node.id}>
        <circle
          cx={node.x}

```

```

        fill="#94a3b8"
        fontSize="10px"
      >
        low:{" "}
        {currentStep.low[node.id] >= 0
          ? currentStep.low[node.id]
          : "-"}
      </text>
    </>
  })
</g>
);
}}
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

```

```

<div className="bg-slate-900 border border-sl
  <p className="text-white font-medium min-h-
    {currentStep.desc}
  </p>
</div>

```

```

<div className="space-y-4 flex-1 overflow-y-a
  <div className="mb-4">
    <h5 className="text-xs text-slate-500 fon
    <div className="flex items-center gap-2 t
      <div className="w-3 h-3 rounded-full bg
        Точка сочленения
      </div>
    <div className="flex items-center gap-2 t
      <div className="w-3 h-3 rounded-full bg
        Посещена
      </div>

```

```
        </div>
      </div>
    </div>
  </div>
</div>
);
};
```

ФАЙЛ 28/82: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
  ];
```

```

{ line: 4, text: '          for (u, v, weight) in E: '
{ line: 5, text: '              if dist[u] + weight <
{ line: 6, text: '                  dist[v] = dist[u]
{ line: 7, text: '          for (u, v, weight) in E: // П
{ line: 8, text: '              if dist[u] + weight < dist
{ line: 9, text: '                  return "ОБНАРУЖЕН ОТПР
{ line: 10, text: '          return dist' },
];

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 999 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '    Фура заведена. Начинаем с Базы (S). Расст
    activeCodeLine: 2,
  });

```

```

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

```

```

simulationSteps.push({
  iteration: iter, checkingEdge: { u: edge.u, v

```

```

        activeCodeLine: 10,
      });
    }
  }

  setSteps(simulationSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" style={borderStyle}>
    <div className="flex flex-wrap items-center justify-center gap-3">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-blue-600 text-white rounded" style={buttonStyle}>
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">Загрузка...
          <p className="text-sm text-blue-300">Полный цикл
        </div>
      </div>
    </div>

    <div className="flex items-center gap-3">
      <div className="bg-slate-900 p-1 rounded-lg border" style={borderStyle}>

```

```
path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
</defs>
```

```
{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id === u);
  const vNode = nodes.find((n) => n.id === v);
  const isChecking = currentStep.checkingEdges;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

  let strokeColor = '#475569';
  let marker = 'url(#arrow-bf-normal)';
  if (isChecking) { strokeColor = '#f59e0b'; }
  else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#f43f5e'; }

  return (
    <g key={idx}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={isChecking ? 4 : 2} marker={marker} />
      <circle cx={(uNode.x + vNode.x) / 2} cy={(uNode.y + vNode.y) / 2} r={isChecking ? 4 : 2} stroke={strokeColor} stroke-width={isChecking ? 4 : 2} />
      <text x={(uNode.x + vNode.x) / 2} y={(uNode.y + vNode.y) / 2} text={edge.w} font-size="11" font-weight="bold" />
    </g>
  );
})}
```

```
{nodes.map((node) => {
  const dist = currentStep.distances[node.id];
  const isCheckingNode = currentStep.checkingNodes;

  return (
    <g key={node.id} className="transition-...>
      <circle cx={node.x} cy={node.y} r={isCheckingNode ? 4 : 2} stroke={strokeColor} stroke-width={isCheckingNode ? 4 : 2} />
    </g>
  );
})}
```



```
</div>
```

```
<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        Таблица расстояний
      </h4>
      <div className="grid grid-cols-4 gap-2 text-rose-950">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded-lg
              ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`}>
              <div className="text-xs text-slate-400">
                {n.id}
              </div>
              <div className={`font-mono font-bold text-rose-950`}
                {d}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
</div>
```

```
{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60">
  <div>
    <h4 className="text-lg font-bold text-slate-400">
      Код алгоритма
    </h4>
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => {
        <div key={item.line} className={`py-1.5 px-4`}
          {item.line === currentStep.startLine ? 'bg-blue-900/80 text-amber-400' : ''}
        <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
        <span className="whitespace-pre flex-grow-1">{item.code}</span>
      </div>
    </div>
  </div>
</div>
```

```
    visited: number[];
    currentNode: number | null;
    logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0).",
  });

  queue.push(0);
  visited.add(0);
  steps.push({
    activeLine: 2,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Добавляем стартовую вершину 0 в очередь и по",
  });

  while (queue.length > 0) {
    steps.push({
      activeLine: 4,
      queue: [...queue],
      visited: Array.from(visited),
      currentNode: null,
      logs: `Очередь не пуста, элементов: ${queue.length}`,
    });
  }
}
```

```

        currentNode: curr,
        logs: `Соседей нет или все уже посещены.`
    });
  }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
    }
  }, [stepIdx, isPlaying]);
}

```

```

        </button>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
      {/** Визуал графа */}
      <div className="bg-slate-800 border border-slate-600 border-radius-50px">
        <svg className="w-full h-full min-h-[350px]" >
          {/** Рёбра */}
          {EDGES.map((e, idx) => {
            const n1 = NODES.find(n => n.id === e.from)
            const n2 = NODES.find(n => n.id === e.to)
            const isVisited = step.visited.includes(n1.id)
            return (
              <line
                key={idx}
                x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
                className={`transition-all duration-500 ${
                  isVisited ? 'stroke-slate-600' : 'stroke-blue-400'
                }`}
              />
            )
          })}

          {/** Вершины */}
          {NODES.map(n => {
            const isCurrent = step.currentNode === n.id
            const isInQueue = step.queue.includes(n.id)
            const isVisited = step.visited.includes(n.id)

            let fill = '#1e293b'; // slate-800
            let stroke = '#475569'; // slate-600
            let scale = 1;
            void scale;

            if (isCurrent) {
              fill = '#3b82f6'; // blue-500
              stroke = '#60a5fa'; // blue-400
            } else if (isInQueue) {
              fill = '#f962b2'; // pink-500
              stroke = '#f4791a'; // orange-500
            } else if (isVisited) {
              fill = '#2d3748'; // slate-700
              stroke = '#475569'; // slate-600
            }

            return (
              <circle
                key={n.id}
                cx={n.x} cy={n.y}
                r={n.radius}
                fill={fill}
                stroke={stroke}
                stroke-width={n.strokeWidth}
                style={{ transform: `scale(${scale})` }}
              />
            )
          })}
        </svg>
      </div>
    </div>
  )}
)

```

```

<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono font-normal tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty():' },
        { line: 5, code: '    node = queue.pop()' },
        { line: 6, code: '    for neighbor in graph[node]:' },
        { line: 7, code: '        if neighbor not in visited:' },
        { line: 8, code: '            visited.add(neighbor)' },
        { line: 9, code: '            queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line
        return (
          <div key={cl.line} className={`px-2 py-2 border border-l-2 border-blue-400 -ml-0.5` : 'text-blue-400'}>
            <span className="opacity-40 w-6 inline-block"> </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>

  <div className="bg-slate-800/80 border border-slate-700 p-4">
    <div className="text-sm font-bold text-emerald-400">
      <span>Состояние Очереди (Queue):</span>
      <span className="text-xs text-slate-500">{queue.size}</span>
    </div>
    <div className="flex items-center gap-2 mt-2">

```

```
      captionColor: 'text-blue-400',
    },
    'bellman-ford': {
      src: bellmanImg,
      alt: 'Фура по болоту',
      caption: ' Тяжёлая, но ей пофиг на ямы!',
      borderColor: 'border-rose-500/30',
      captionColor: 'text-rose-400',
    },
    floyd: {
      src: floydImg,
      alt: 'Все ко Всем Флойд',
      caption: ' Все связаны со всеми!',
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-gray-200`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```

```

        className="flex items-center gap-1 px-2.5 py-1
        :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  );
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
        transition-all"
      >
        <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
        </div>
        <div className="text-sm font-bold text-slate-700">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-white
        transition-all sm:col-start-2"
      >
        <div className="flex items-center justify-end text-indigo-400 mb-1">
          Следующая тема <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-700">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);

```

```

const [saving, setSaving] = useState<"idle" | "work">("idle");

useTermHints(contentRef, [chapter.id]);

const cancelHide = useCallback(() => {
  if (hideTimer.current) {
    window.clearTimeout(hideTimer.current);
    hideTimer.current = null;
  }
}, []);

const scheduleHide = useCallback(() => {
  cancelHide();
  hideTimer.current = window.setTimeout(() => setAnchor(
), [cancelHide]);

useEffect(() => {
  setAnchor(null);
  setSaving("idle");
}, [chapter.id]);

const saveHtml = useCallback(async () => {
  setSaving("work");
  try {
    await downloadChapterHtml(chapter);
    setSaving("done");
    window.setTimeout(() => setSaving("idle"), 2500);
  } catch {
    setSaving("idle");
  }
}, [chapter]);

const open = useCallback(
  (el: HTMLElement) => {
    const termId = el.dataset.termId;
    if (!termId) return;
    cancelHide();
    setAnchor({ termId, rect: el.getBoundingClientRect() });
  },
);

```



```

disabled={saving === "work"}
title="Сохранить эту тему одним автономным I
className="flex items-center gap-1.5 text-x
300 hover:text-white hover:border-emerald-5
>
{saving === "work" ? (
  <Loader2 className="w-3.5 h-3.5 animate-s
) : saving === "done" ? (
  <Check className="w-3.5 h-3.5 text-emeral
) : (
  <Download className="w-3.5 h-3.5" />
)}
{saving === "done" ? "Файл сохранён" : "Ска
</button>
<button
  onClick={() => window.print()}
  title="Распечатать или сохранить в PDF сред
  className="flex items-center gap-1.5 text-x
  300 hover:text-white hover:border-indigo-50
>
  <Printer className="w-3.5 h-3.5" /> Печать
</button>
<span className="text-[11px] text-slate-500">
</div>

<div
  ref={contentRef}
  className="prose prose-invert max-w-none"
  onMouseOver={handlePointerOver}
  onMouseOut={handlePointerOut}
  onClick={handleClick}
  onFocus={handleFocus}
  dangerouslySetInnerHTML={{ __html: chapter.co
/>

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2">
  <MousePointerClick className="w-4 h-4 shrink-

```

```
        />
      </>
    );
  };
};
```

ФАЙЛ 33/82: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";
```

```
interface CodeLine {
  line: string;
  highlight: string;
}
```

```
const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    ", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (visited[to]) {", highlight: "normal" },
  { line: "            low[v] = min(low[v], tin[to]);", highlight: "normal" },
  { line: "        } else {", highlight: "normal" },
  { line: "            dfs(to, v);", highlight: "normal" },
  { line: "            low[v] = min(low[v], low[to]);", highlight: "normal" },
  { line: "        ", highlight: "normal" },
  { line: "        if (low[to] > tin[v]) {", highlight: "normal" },
  { line: "            // ЭТО МОСТ!", highlight: "normal" },
  { line: "        }", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];
```

```

{
  currentNode: 0, visitedIndices: [0],
  tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
  activeEdge: null, backEdge: null,
  log: "Входим в A. tin[A]=1, low[A]=1.",
  activeCodeLine: [0,1,2]
},
{
  currentNode: 1, visitedIndices: [0,1],
  tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
  activeEdge: [0,1], backEdge: null,
  log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 6, visitedIndices: [0,1,6],
  tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
  activeEdge: [1,6], backEdge: null,
  log: "Из B идём в G. tin[G]=3, low[G]=3.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 1, visitedIndices: [0,1,6],
  tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
  activeEdge: null, backEdge: null,
  log: "У G нет соседей (кроме B). Возврат. low[G]=3.",
  activeCodeLine: [13,14,15]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
  activeEdge: [1,2], backEdge: null,
  log: "Из B идём в C. tin[C]=4, low[C]=4.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,6,2],
  activeEdge: null, backEdge: null,
  log: "Из C идём в B. Впервые. tin[B]=2, low[B]=2.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 0, visitedIndices: [0],
  tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
  activeEdge: null, backEdge: null,
  log: "Выходим из A. tin[A]=1, low[A]=1.",
  activeCodeLine: [0,1,2]
}

```

```

    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост!
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост!
        activeCodeLine: [13,14,15,16,17]
    },
    {
        currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост!
        activeCodeLine: [11,13,16,17]
    },
    {
        currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D).
        activeCodeLine: [11,13,16,17,18]
    },
    {
        currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
        activeEdge: null, backEdge: null,
        log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм DFS завершён!
        activeCodeLine: []
    },
];

```

```

        <SkipForward className="h-3.5 w-3.5" />
      </button>
      <button onClick={() => { setDfsAuto(false); setDfsAuto(
        className="p-1.5 hover:bg-slate-700 rounded
        <RefreshCw className="h-3.5 w-3.5" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-5 gap-4" >
    { /* Graph visualization */ }
    <div className="md:col-span-3 bg-slate-950 rounded" >
      <div className="absolute inset-0 bg-[radial-gradient(circle,
        >
      <svg className="w-full h-[280px] z-10 relative" >
        {GRAPH_EDGES.map(e => {
          const u = GRAPH_NODES.find(n => n.id === e.u)
          const v = GRAPH_NODES.find(n => n.id === e.v)
          const bridge = step.bridges.includes(e.key)
          const activeTree = step.activeEdge && (
            (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
            (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u)
          );
          const activeBack = step.backEdge && (
            (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
            (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
          );
          let sc = "#475569", sw = 2, dash = "none"
          if (bridge) { sc = "#eab308"; sw = 4; }
          else if (activeTree) { sc = "#10b981"; sw = 2; }
          else if (activeBack) { sc = "#f43f5e"; sw = 2; }
          else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) {
            return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
              stroke={sc} strokeWidth={sw} strokeDash={dash}
              className="transition-all duration-500"
            />
          }
        })}
        {step.currentNode !== null && step.currentNode.id === step.currentNode.id}
        const n = GRAPH_NODES.find(x => x.id === step.currentNode.id)

```

```

    { /* Code panel */ }
    <div className="bg-slate-950 rounded-lg border"
      <div className="bg-slate-900 px-3 py-1.5 text-slate-400"
        >
        <div className="p-2 overflow-x-auto">
          <pre className="text-[10px] font-mono leading-relaxed"
            {CODE_LINES.map(({cl, i}) => {
              const hl = step.activeCodeLine.includes(cl);
              return <div key={i}
                className={`px-1 transition-all duration-200
                  ${hl ? "bg-indigo-600/20 text-indigo-400" : "text-slate-400"}
                }`>
                <span className="text-slate-600 mr-1 font-mono"
                  {cl.line} || ' '
                </span>
              </div>;
            })}
          </pre>
        </div>
      </div>
    </div>

    { /* Stack */ }
    <div className="bg-slate-900/50 rounded-xl border"
      <div className="text-[10px] font-bold text-slate-400"
        <span>Стек DFS</span>
        <span className="text-indigo-400">{step.stack.length}</span>
      </div>
      <div className="flex flex-col-reverse gap-1"
        {step.stack.length === 0
          ? <div className="text-[10px] text-slate-400"
            : step.stack.map((id, i) => {
              <div key={id} className={`px-2 py-0.5
                ${i === step.stack.length - 1
                  ? "bg-emerald-600/20 text-emerald-400"
                  : "bg-slate-900 text-slate-400"}
                }`>Вершина {GRAPH_NODES.find(n=>n.id === id).name}</div>
              <span className="text-slate-500 ml-1"
                {step.stack[id]}</span>
            </div>
          )}
      </div>
    </div>
  </div>

```

```

}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
    { u: 'C', v: 'B', w: 1 },
    { u: 'C', v: 'G', w: 3 },
    { u: 'C', v: 'E', w: 8 },
    { u: 'B', v: 'G', w: 4 },
    { u: 'B', v: 'D', w: 5 },
    { u: 'G', v: 'D', w: 1 },
    { u: 'G', v: 'E', w: 2 },
    { u: 'D', v: 'F', w: 3 },
    { u: 'E', v: 'F', w: 1 },
  ];

  const adjList: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (adjList[n.id] = []));
  edges.forEach(e => {
    adjList[e.u].push({ v: e.v, w: e.w });
  });

  const codeLines = [
    { line: 1, text: 'function dijkstra(graph, start):' },
    { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
    { line: 3, text: '    pq = PriorityQueue(); pq.push(start)' },
    { line: 4, text: '    while not pq.empty():' },
  ]
}

```

```

    visited[u] = true;
    simulationSteps.push({
      currentNode: u, checkingEdge: null,
      distances: { ...dist }, visited: { ...visited },
      log: `  Выбираем вершину ${u} с мин. непосещёнными соседями`,
      activeCodeLine: 5,
    });

    const neighbors = edges.filter(e => e.u === u);
    for (const edge of neighbors) {
      const v = edge.v;
      if (visited[v]) continue;

      simulationSteps.push({
        currentNode: u, checkingEdge: { u, v },
        distances: { ...dist }, visited: { ...visited },
        log: `  Смотрим соседа ${v}. Текущее расстояние: ${dist[u] + edge.w}.`,
        activeCodeLine: 8,
      });

      if (dist[u] + edge.w < dist[v]) {
        dist[v] = dist[u] + edge.w;
        prev[v] = u;
        simulationSteps.push({
          currentNode: u, checkingEdge: { u, v },
          distances: { ...dist }, visited: { ...visited },
          log: `  Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]}.`,
          activeCodeLine: 9,
        });
      }
    }
  }
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: `  Дейкстра всё доставил! Все возможные кратчайшие пути найдены.`,
  activeCodeLine: 10,
});

```



```

        onClick={() => { setIsPlaying(false); setCu
        className="flex items-center gap-1 bg-slate
        ion-colors"
    >
        C6poc
    </button>
    <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1 px-4 py
            isPlaying ? 'bg-amber-600 hover:bg-amber-!
            `}
    >
        {isPlaying ? '    Паўза' : '▶ Пуск'}
    </button>
    <button
        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length
        className="flex items-center gap-1 bg-indigo
        px-3 py-2 rounded-lg text-sm font-medium tr
    >
        ⏏
    </button>
</div>
</div>

```

```

        <text
          x={uNode.x + vNode.x} / 2} y={uNode.y + vNode.y} / 2}
          fill={isChecking ? '#f59e0b' : '#f8d7da'}
          fontSize="11" fontWeight="bold" text={edge.w}
        >
      </text>
    </g>
  );
}
}

{/* Вершины */}
nodes.map((node) => {
  const isCurrent = currentStep.currentNode === node.id;
  const isVisited = currentStep.visited[node.id];
  const dist = currentStep.distances[node.id];

  return (
    <g key={node.id} className="transition-all duration-300">
      <circle
        cx={node.x} cy={node.y}
        r={isCurrent ? 22 : 18}
        fill={isCurrent ? '#4f46e5' : isVisited ? '#f8d7da' : '#fff2cc'}
        stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f8d7da' : '#fff2cc'}
        strokeWidth={isCurrent ? 4 : 2}
        className="transition-all duration-300"
      />
      <text x={node.x} y={node.y + 5} fill="black">
        {node.id}
      </text>
      <text x={node.x} y={node.y - 28} fill="white" className="bg-indigo-950 px-1 py-0">
        {dist === 999 ? '∞' : `d=${dist}`}
      </text>
      <text x={node.x} y={node.y + 35} fill="black">
        {node.name.split(' ')[0]}
      </text>
    </g>
  );
});

```

```

        </span>
      );
    }}}
  </div>
</div>
);
}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        <div className="overflow-x-auto">
          <table className="w-full text-left border">
            <thead>
              <tr className="border-b border-rose-950">
                <th className="py-2 px-3">Вершина</th>
                <th className="py-2 px-3">Расстояние</th>
                <th className="py-2 px-3">Из (пред. шаг)</th>
              </tr>
            </thead>
            <tbody className="text-sm">
              {nodes.map((n) => {
                const dist = currentStep.distances[n.id];
                const prev = currentStep.previous[n.id];
                const vis = currentStep.visited[n.id];
                const isCurr = currentStep.currentNode === n.id;

                return (
                  <tr key={n.id} className={`border-b border-rose-950 ${
                    isCurr ? 'bg-indigo-950/60 font-medium' : ''
                  }`}>
                    <td className="py-2.5 px-3 flex items-center">
                      <span className={`w-2 h-2 rounded-full ${

```

```
        </div>
      </div>
    );
  }
}
```

ФАЙЛ 35/82: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

interface DPStep {
  id: number;
  n: number;
  action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'return';
  desc: string;
  codeLine: number;
  memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "  if (n in memo) return memo[n];",
  /* 3 */ "  if (n <= 2) return 1;",
  /* 4 */ "  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
  /* 5 */ "  return memo[n];",
  /* 6 */ "}"
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState<boolean>(false);
  const [speed] = useState<number>(800);
```

```

        return 1;
    }

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
      codeLine: 4,
      memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'calc',
      desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
      codeLine: 5,
      memoState: { ...memoMap }
    });

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

  generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `Расчет завершен! fibМемо(${targetN}) = ${finalRes}`,
    codeLine: 6,
    memoState: { ...memoMap }
  });

```

```

<div className="flex items-center gap-2 bg-slate-950" >
  <span>N:</span>
  <select
    value={targetN}
    onChange={(e) => setTargetN(Number(e.target.value))}
    disabled={isPlaying}
    className="bg-transparent text-white font-mono"
  >
    <option value={4}>N = 4</option>
    <option value={5}>N = 5</option>
    <option value={6}>N = 6</option>
    <option value={7}>N = 7</option>
  </select>
</div>

```

```

<div className="flex items-center bg-slate-950" >
  <button
    onClick={() => { setIsPlaying(false); setTargetN(4); }}
    className="p-2 text-slate-400 hover:text-white"
    title="Сбросить"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
  <button
    onClick={() => { setIsPlaying(false); setTargetN(4); }}
    disabled={currentStepIndex === 0}
    className="p-2 text-slate-400 hover:text-white"
    title="Шаг назад"
  >
    <ChevronLeft className="w-5 h-5" />
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1.5 px-2 py-2
      ${isPlaying
        ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
        : 'bg-emerald-600 text-white hover:bg-emerald-700'}
    `}
  >
    {isPlaying ? 'Пауза' : 'Воспроизвести'}
  </button>
</div>

```

```

        {isCached ? val : '0'}
      </span>
      {num <= 2 && <span className="text-[9px]
    </div>
  );
  }}}
</div>

{currentStep && (
  <div className="mt-6 pt-4 border-t border-slate-400" >
    <div className="text-sm font-medium text-slate-600" >
      <span className="inline-block w-2 h-2 rounded-full bg-current" />
      <span>{currentStep.desc}</span>
    </div>
    <div className="text-xs font-mono bg-slate-400 text-white" >
      Текущий N: <strong className="text-emerald-400" />
    </div>
  </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-400" >
  <button
    onClick={() => setActiveTab('table')}
    className={`flex items-center gap-2 px-6 py-3
      ${activeTab === 'table'
        ? 'border-emerald-500 text-emerald-400 bg-white'
        : 'border-transparent text-slate-400 hover:bg-slate-400'}
    `}
  >
    <Eye className="w-4 h-4" />
    Шаг-бай-шаг состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      ${activeTab === 'code'

```

```

        return (
          <div
            key={lineNum}
            className={`flex items-center px-4
              isActive
                ? 'bg-emerald-600/20 border-l-4
                : 'text-slate-300 hover:bg-slate-500
              `}
          >
            <span className="w-8 text-xs text-slate-500">
              <span>{line}</span>
            </div>
          );
        })}
      </pre>
      <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500/20">
        <span className="p-1.5 bg-emerald-500/20 text-white">
          <span>{currentStep?.desc}</span>
        </div>
      </div>
    )}
  </div>

  {/* Progress Bar */}
  <div className="mt-8 pt-6 border-t border-slate-800">
    <div className="flex items-center justify-between">
      <span>Прогресс выполнения</span>
      <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
    </div>
    <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
        style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
      ></div>
    </div>
  </div>
</div>
);

```



```

    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы пр
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
    const nextEdges = [...c1VisitedEdges, key];
    const nextPath = [...c1Path, vId];
    setC1VisitedEdges(nextEdges);
    setC1Path(nextPath);

    if (nextEdges.length === C1_EDGES.length) {
      const startDegree = C1_EDGES.filter(e => e.u === c1Path[0]).length;
      const endDegree = C1_EDGES.filter(e => e.u === vId).length;
      const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
      if (isCycle && nextPath[0] === vId) {
        setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф
      } else {
        setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
      }
      setC1Done(true);
    } else {
      setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
    }
  };

  return (
    <div className="space-y-4">

```

```

        Ребёр: {c1VisitedEdges.length}/{C1_EDGES.length}
      </div>
    </div>

    <div className={`p-3 rounded-xl border text-sm ${
      c1Done && c1VisitedEdges.length === C1_EDGES.length
        ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
        : c1Done
        ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
        : "bg-slate-900/50 border-slate-700 text-slate-50"}
    }`}>
      {c1Msg}
    </div>
  </div>
);
}

```

ФАЙЛ 37/82: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
  id: number;
  question: string;
  options: string[];
  correctAnswer: number;
  explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если

```

```

    {
      id: 4,
      question: 'В чем главная суть Динамического программирования?',
      options: [
        'Писать код очень быстро, динамично нажимая на клавиши.',
        'Запоминать (кешировать) результаты уже решенных задач.',
        'Использовать динамическую типизацию как в Python.',
        'Постоянно менять требования к проекту в процессе разработки.'
      ],
      correctAnswer: 1,
      explanation: 'Точно! Главный принцип ДП – мемоизация.'
    },
    {
      id: 5,
      question: 'Какая структура данных идеально помогает реализовать алгоритм быстрого поиска?',
      options: [
        'СНМ (Система непересекающихся множеств / Disjoint Set Structure)',
        'Стек (LIFO).',
        'Красно-черное дерево.',
        'Хэш-таблица со списками коллизий.'
      ],
      correctAnswer: 0,
      explanation: 'Блестяще! СНМ (DSU) позволяет за практический момент времени обрабатывать запросы на проверку принадлежности к множеству и объединение множеств.'
    }
  ];

```

```

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<number>(0);
  const [selectedOption, setSelectedOption] = useState<number>(-1);
  const [isAnswered, setIsAnswered] = useState<boolean>(false);
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>(false);

  const currentQ = quizQuestions[currentQIndex];

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;

```

```

    <p className="text-5xl font-black text-indigo-500">
    <p className="text-slate-300 text-sm">
      {score === quizQuestions.length
        ? ' Идеально! Ты настоящий сеньор-раскра
        : score >= 3
        ? ' Отличный результат! Главные концепции
        : ' Стоит повторить главы пособия и еще
    </p>
  </div>

  <button
    onClick={handleRestart}
    className="inline-flex items-center gap-2 px-4 py-2 rounded-xl shadow-lg shadow-indigo-600/30"
  >
    <RotateCcw className="w-5 h-5" />
    <span>Пройти тест заново</span>
  </button>
</div>
);
}

return (
  <div className="bg-slate-900/90 border border-slate-800" >
    <div className="flex items-center justify-between" >
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-4 py-2 rounded">
          Экспресс-тест
        </span>
        <h3 className="text-xl font-bold text-white">
          </h3>
        </div>
        <span className="text-sm font-mono text-slate-400">
          Вопрос {currentQIndex + 1} / {quizQuestions.length}
        </span>
      </div>

      <div className="mb-8">

```

```

        <div className="flex-1 text-sm md:text-l
            {option}
        </div>
    </div>
    );
    }}}
</div>
</div>

{isAnswered && (
    <div className="bg-slate-950 border border-slate
        <div className="flex items-center gap-2 mb-2
            <AlertCircle className="w-4 h-4" />
            <span>Объяснение:</span>
        </div>
        <p className="text-slate-300 text-sm leading-
            {currentQ.explanation}
        </p>
    </div>
)}

<div className="flex justify-end pt-4 border-t bo
    <button
        onClick={handleNext}
        disabled={!isAnswered}
        className={
            "px-8 py-3.5 font-bold rounded-xl transition
            (!isAnswered
                ? "bg-slate-800 text-slate-500 border bor
                : "bg-indigo-600 hover:bg-indigo-500 acti
            }
        >
        {currentQIndex + 1 < quizQuestions.length ? '
    </button>
</div>
</div>
);
};

```

```

    [999, 1, 999, 999, 999, 999, 0]
  ];

  const adjList: { [key: number]: { v: number; w: number }[] } = {
    0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
    1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
    2: [{ v: 4, w: 2 }],
    3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
    4: [{ v: 6, w: 1 }],
    5: [{ v: 6, w: 5 }],
    6: [{ v: 1, w: 1 }],
  };

  const codeLines = [
    { line: 1, text: 'function floyd_warshall(matrix, V)' },
    { line: 2, text: '  dist = copy(matrix)' },
    { line: 3, text: '  for k from 0 to V - 1: // Пос' },
    { line: 4, text: '    for i from 0 to V - 1:' },
    { line: 5, text: '      for j from 0 to V - 1' },
    { line: 6, text: '        if dist[i][k] + d' },
    { line: 7, text: '          dist[i][j] = ' },
    { line: 8, text: '  return dist' },
  ];

  useEffect(() => {
    const simulationSteps: Step[] = [];
    const dist = initialMatrix.map(row => [...row]);

    simulationSteps.push({
      k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
      log: '  Инициализация матрицы прямых путей, ∞ (999)',
      activeCodeLine: 2,
    });

    const N = 7;
    for (let k = 0; k < N; k++) {
      simulationSteps.push({
        k, i: -1, j: -1, matrix: dist.map(r => [...r]),

```

```

    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border"
    <div className="flex flex-wrap items-center justify"
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white"
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white"
            <p className="text-sm text-emerald-300">Полн
          </div>
        </div>

        <div className="flex items-center gap-2">
          <button onClick={() => { setIsPlaying(false);
            text-sm"> Сброс</button>
          <button onClick={() => setIsPlaying(!isPlaying
            -amber-600' : 'bg-emerald-600')}`}>{isPlaying
          <button onClick={() => { setIsPlaying(false);
            ; }} className="px-3 py-2 bg-emerald-600 te
          </div>
        </div>

    <div className="flex flex-col lg:flex-row gap-6 m
      {/* Граф */}
      <div className="w-full lg:w-2/3 bg-indigo-950/2
        stiffy-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indi
          ont-bold shadow-sm">

```

```

    let fill = '#1e293b'; let stroke = '#64748f';
    if (isK) { fill = '#4f46e5'; stroke = '#a8a8a8'; }
    else if (isI) { fill = '#b45309'; stroke = '#b45309'; }
    else if (isJ) { fill = '#047857'; stroke = '#047857'; }
    return (
      <g key={node.id} className="transition-colors">
        <circle cx={node.x} cy={node.y} r={isJ ? 4 : 2} className="transition-colors duration-200" />
        <text x={node.x} y={node.y + 5} fill={fill} />
        <text x={node.x} y={node.y + 32} fill={fill} />
      </text>
    </g>
  );
}
}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 rounded">
  <p className="font-semibold text-amber-400">
    <p className="min-h-[40px] flex items-center">
  </div>
</div>

```

```

  /* Список смежности */
  <div className="w-full lg:w-1/3 bg-emerald-950/40">
    <h4 className="text-lg font-bold text-emerald-400">
      <div className="space-y-2 text-sm font-mono">
        {Object.keys(adjList).map((uStr) => {
          const u = parseInt(uStr);
          const isI = currentStep.i === u;
          const isK = currentStep.k === u;
          return (
            <div key={u} className={`flex flex-wrap ${
              isI ? 'bg-amber-900/40 border-amber-400' :
              isK ? 'bg-indigo-900/40 border-indigo-400' :
            }`} >
              <span className="font-bold text-white text-sm">
                e-400">></span>
              {adjList[u].map((edge, idx) => {

```



```

        const isUpd = currentStep.i === i
        const isVia = currentStep.k !== -
        tep.j === j));
        let bg = 'bg-slate-900/40 text-slate-100';
        if (isUpd) bg = 'bg-emerald-900/80 text-white';
        else if (isVia) bg = 'bg-indigo-900/80 text-white';
        else if (i === j) bg = 'bg-slate-900/40 text-slate-100';
        return (<td key={j} className={bg}
        /td>);
    }
  }
</tr>
    )
  }
</tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-100">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map(item => {
        <div key={item.line} className={item.i === currentStep.i ? 'bg-blue-900/80 text-amber-400' : 'bg-slate-900/40 text-slate-100'}>
        <span className="text-slate-600 w-5 shrink-0">{item.i}</span>
        <span className="whitespace-pre flex-grow-1">{item.v}</span>
        </div>
      })}
    </div>
  </div>
</div>
</div>
</div>
</div>
);
}

```

```

    type: 'visit' | 'process' | 'backtrack';
    node: string;
    desc: string;
    queueOrStack?: string[];
    visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const stack: string[] = []; // for visualization, just for visualization

    function dfs(v: string) {
        vis.add(v);
        stack.push(v);
        steps.push({ type: 'visit', node: v, desc: `Зависит от`,
            visitedNodes: Array.from(vis) });

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                steps.push({ type: 'process', node: v, desc: `Процесс`,
                    queueOrStack: [...stack], visitedNodes: Array.from(vis) });
                dfs(u);
            }
        }

        stack.pop();
        steps.push({ type: 'backtrack', node: v, desc: `Отвязываем`,
            queueOrStack: [...stack], visitedNodes: Array.from(vis) });
    }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();

```

```

        setStepIdx(0);
        setAutoPlay(false);
    }, [mode]));

    useEffect(() => {
        if (autoPlay) {
            const t = setInterval(() => {
                setStepIdx(p => {
                    if (p >= steps.length - 1) {
                        setAutoPlay(false);
                        return p;
                    }
                    return p + 1;
                });
            }, 1200);
            return () => clearInterval(t);
        }
    }, [autoPlay, steps.length]);

    const step = steps[stepIdx] || steps[0];

    const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
    const isCurrent = (nodeId: string) => step.node === nodeId;

    return (
        <div className="space-y-4">
            <div className="flex bg-slate-900 border border-slate-800 p-4">
                <button onClick={() => setMode("dfs")}
                    className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                    DFS (В глубину)
                </button>
                <button onClick={() => setMode("bfs")}
                    className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
                    BFS (В ширину / Волна)
                </button>
            </div>
        </div>
    );

```

```

        textColor = "fill-amber-500"
        radius = 22;
    }

    return (
      <g key={n.id} className="graph" style={{position: 'relative', width: '100%', height: '100%'}}>
        <circle cx={n.x} cy={n.y} r={radius} fill="white" stroke="black" stroke-width={1}
          className={`stroke-${n.type} ${n.type === 'edge' ? 'stroke-amber-500' : 'stroke-emerald-500'}`} />
        <text x={n.x} y={n.y} text-align="center" font-size={14} font-weight="bold"
          className={`text-${n.type} ${n.type === 'edge' ? 'text-amber-500' : 'text-emerald-500'}`}>
          {n.label}
        </text>

        {/* Ripple for BFS mode */}
        {current && mode === 'bfs' && n === current ? (
          <circle cx={n.x} cy={n.y} r={radius} fill="white" stroke="black" stroke-width={1}
            className="stroke-emerald-500" />
        ) : null}
      </g>
    )
  )}
</svg>
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-emerald-500 p-10" style={{width: '100%', height: '100%'}}>
  <div className={`absolute -top-3 left-3 right-3 bottom-3 border border-emerald-500
    ${mode === 'dfs' ? 'border-emerald-500' : 'border-amber-500'}
    ${mode === 'dfs' ? 'text-emerald-200' : 'text-amber-200'}`}>
    {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
  </div>

  <div className="flex flex-col gap-2 mt-10" style={{width: '100%'}}>
    {step.queueOrStack && step.queueOrStack.length > 0 ? (
      <div key={idx} className={`w-full p-2 border border-emerald-500
        ${mode === 'dfs' ? 'bg-emerald-900 text-emerald-200' : 'bg-amber-900 text-amber-200'}`}>
        {step.queueOrStack[idx]}
      </div>
    ) : null}
  </div>
</div>

```

```
        {/* Description Log */}
        <div className="flex-1 bg-slate-900/50
ow-hidden">
            <div className="text-[10px] text-slate-100" data-bbox="274 194 1000 214">
{steps.length}</div>
            <div className="text-sm text-slate-100" data-bbox="408 244 1000 264">
                {step.desc}
            </div>
        </div>
    </div>
</div>
);
}
```

ФАЙЛ 40/82: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
```

```
interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
```

```

    return { x, y };
  }

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
  });
  return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog] = useState<string>('');
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);

```

```

        setBusy(false);
    }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
    e.preventDefault();
    if (!customName.trim()) return;
    insertPatient(customName, customPriority);
    setCustomName('');
};

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
    if (busy || heap.length === 0) {
        setShake(true);
        addLog('⚠ Нет пациентов для экстренной госпитализации');
        setTimeout(() => setShake(false), 400);
        return;
    }
    setBusy(true);
    const topPatient = heap[0];
    setHealing(topPatient);

    addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

    // Step 1: Root lights up as winner
    setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));

    setTimeout(() => {
        // Step 2: Root exits the tree, tree reorganizes
        setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'healed' } : x));
    }, 1000);
}, [busy, heap]);

```

```

    if (l < heap.length) {
      const lp = nodePos(l);
      res.push(
        <line key={`el${idx}`}
          x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
          x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    if (r < heap.length) {
      const rp = nodePos(r);
      res.push(
        <line key={`er${idx}`}
          x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
          x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
          stroke="#475569" strokeWidth="2"
        />
      );
    }
    return res;
  });

return (
  <div className="flex flex-col gap-6">
    /* МНЕМОНИКА / ПРАВИЛО */
    <div className="bg-emerald-950/40 border border-emerald-500">
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-emerald-400 text-center">
          Priority Queue</h3>
        <p className="text-xs text-slate-300 mt-1 leading-relaxed">
          В реанимации не важно, кто пришел раньше, а кто сейчас в
          остояние</b> (максимальный приоритет).
          Куча (Heap) автоматически перестраивает пацков, которые
          оказывался на самом верху дерева (в корне).
        </p>
      </div>
    </div>
  )

```



```

        <button
          type="submit"
          disabled={busy || !customName.trim() || heap.length === 0}
          className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 py-4 px-4 rounded-xl shadow-lg active:scale-95 transition-all"
        >
          Везти
        </button>
      </form>

      {/* Забрать самого тяжелого */}
      <button
        onClick={extractMax}
        disabled={busy || heap.length === 0}
        className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
      >
        <span> В реанимацию (EXTRACT MAX)</span>
      </button>

      <button
        onClick={reset}
        className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
      >
        Сброс
      </button>
    </div>

    {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
    <div className="grid grid-cols-1 lg:grid-cols-12 gap-4">

      {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
      <div className={`lg:col-span-8 bg-slate-900 rounded-2xl shadow-2xl p-4 [380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
        <div className="absolute top-4 left-4 text-[18px] font-medium">
          Сортировка по тяжести (Двоичная куча / Max-Heap)
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        <div className="text-emerald-400">
        </div>
    </div>
</div>
    );
    }}}
</div>

    <div className="w-full h-3 bg-gradient-to-r f
</div>

    { /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
    <div className="lg:col-span-4 bg-slate-900 round
        <div className="absolute top-4 left-4 text-[1
            Операционная койка
        </div>

    <div className="flex-1 flex flex-col justify-
        {healingPatient ? (
            <div className="flex flex-col items-cente
                <div className="relative">
                    <span className="text-6xl block anim-l
                    <span className="absolute -top-4 -right
                    <span className="absolute -bottom-2 -
                </div>
            <div>
                <h4 className="text-white font-black
                    <span>Пациент: {healingPatient.name
                </h4>
                <p className="text-emerald-400 text-x
                    Состояние: Стабилизация ({healingPa
                </p>
                <div className="flex justify-center g
                    <span className="w-2 h-2 rounded-fu
                    <Heart className="w-4 h-4 text-rose
                </div>
            </div>
        </div>
    </div>

```

```
/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу — мож
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
  const [step, setStep] = useState(0);
  const paused = useContext(DemoPauseContext);
  const period = Math.round(ms * SPEED);

  useEffect(() => {
    if (steps <= 1 || paused) return;
    const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
    return () => clearInterval(t);
  }, [steps, period, paused]);

  return step;
}

export const C = {
  idle: "#334155",
  idleStroke: "#475569",
  text: "#e2e8f0",
  dim: "#94a3b8",
```

```

        {label}
      </text>
    })
  </g>
);

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    strokeWidth={width}
    strokeDasharray={dashed ? "4 3" : undefined}
    style={{ transition: MOVE }}
  />
);

```

ФАЙЛ 42/82: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
  const step = useLoop(3, 1200);
  // i -> j напрямую 9; через k=1 получаем 3+4=7

```

```

    [ ["D", "E"] ],
    [ ["F", "G"], ["G", "H"], ["F", "H"] ],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) => e === v);

  return (
    <Svg caption={step === 0 ? "Граф распался на острова" : ""}>
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => (
          <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]} />
        ))
      )}
      {Object.entries(pos).map(([k, [x, y]]) => {
        const ci = compOf(k);
        return <Node key={k} x={x} y={y} r={12} label={k} color={colors[ci]} />
      })}
    </Svg>
  );
};

```

/** Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
  const step = useLoop(3, 1300);
  const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50], D: [350, 50], E: [450, 50], F: [550, 50], G: [650, 50], H: [750, 50], I: [850, 50], J: [950, 50] };
  const edges: [string, string, number][] = [
    ["A", "B", 1], ["B", "C", 1], ["C", "D", 1], ["D", "E", 1], ["E", "F", 1], ["F", "G", 1], ["G", "H", 1], ["H", "I", 1], ["I", "J", 1],
    ["A", "D", 2], ["B", "E", 2], ["C", "F", 2], ["D", "G", 2], ["E", "H", 2], ["F", "I", 2], ["G", "J", 2],
    ["A", "E", 3], ["B", "F", 3], ["C", "G", 3], ["D", "H", 3], ["E", "I", 3], ["F", "J", 3],
  ];
  return (
    <Svg>
      <caption>{
        step === 0
          ? "Вершины — объекты (города, слова, состояния)"
          : step === 1
            ? "Рёбра — связи (дороги, слова, переходы)"
            : "Граф — совокупность вершин и рёбер"
      }</caption>
      {edges.map(([u, v, w]) => (
        <Edge key={`${u} - ${v}`} a={pos[u]} b={pos[v]} weight={w} />
      ))}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} r={12} />
      ))}
    </Svg>
  );
};

```

```

const startX = (260 - raw.length * (boxW + 4)) / 2;

return (
  <Svg
    caption={
      step === 0
        ? "Исходные координаты – огромные и с дырами"
        : step === 1
          ? "Сортируем уникальные значения"
          : "Каждое число заменяем его номером → плотн
    }
  >
    {raw.map((v, i) => {
      <g key={i}>
        <rect x={startX + i * (boxW + 4)} y={20} width
        <text x={startX + i * (boxW + 4) + boxW / 2}
          {v}
        </text>
      </g>
    })}

    {step >= 1 &&
      sorted.map((v, i) => {
        <g key={v}>
          <rect x={30 + i * 52} y={62} width={46} hei
          <text x={53 + i * 52} y={77} textAnchor="mi
            {v}
          </text>
        </g>
      })}

    {step >= 2 &&
      raw.map((v, i) => {
        <g key={`c${i}`}>
          <rect x={startX + i * (boxW + 4)} y={98} wi
          <text x={startX + i * (boxW + 4) + boxW / 2
            {sorted.indexOf(v)}
          </text>

```

```

        {id}
      </text>
    </g>
  )))
</Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
  <SplayFrames
    captions={["р – корень, х под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
    ]}
  />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верш
export const ZigZigDemo: React.FC = () => {
  <SplayFrames
    captions={["Линия g → р → х («бамбук»)", "Поворот В
    frames={[
      { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
      { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
      { pos: { x: [102, 24], p: [152, 70], g: [202, 112
    ]}
  />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
  <SplayFrames
    captions={["Зигзаг: g влево, х вправо", "Поворот НИ
    frames={[
      { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
      { pos: { g: [176, 24], x: [110, 70], p: [64, 112]

```

```
};
```

```
export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={k === head ? "red" : "gray"} />
      ))}
    </Svg>
  );
};
```

```
export const ToposortDemo: React.FC = () => {
  const nodes = [
    { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
    { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
  ];
  const edges: [string, string][] = [
    ["труссы", "штаны"], ["труссы", "носки"], ["штаны", "боты"], ["носки", "боты"]
  ];
  const order = ["труссы", "носки", "штаны", "боты"];
  const step = useLoop(order.length + 1, 800);
  const placed = order.slice(0, step);
  const pos = Object.fromEntries(nodes.map((n) => [n.id, [n.x, n.y]]));
  return (
    <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={placed.indexOf(v) < placed.indexOf(u) ? "red" : "gray"} />
      ))}
      {nodes.map((n) => (
        <Node key={n.id} x={n.x} y={n.y} label={n.id} fill={placed.indexOf(n.id) < step ? "red" : "gray"} />
      ))}
    </Svg>
  );
};
```



```

const step = useLoop(2, 1300);
const pos: Record<string, [number, number]> = {
  A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
};
const edges: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
const cut = step === 1;
return (
  <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}
    {edges.map(([u, v], i) => {
      const isBridge = u === "C" && v === "D";
      if (isBridge && cut) return null;
      return <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "blue"} />
    ))}
  </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}
      {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />))}
      {Object.entries(pos).map(([k, [x, y]]) => (
        cut && k === "C" ? null : (
          <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "blue"} />
        )
      ))}
    </Svg>
  );
};

```

```

const edges: [string, string][] = [["A", "B"], ["B",
const step = useLoop(2, 1400);
const scc = ["A", "B", "C"];
const inScc = (u: string, v: string) => step === 1 &&
return (
  <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
    {edges.map(([u, v], i) => (
      <Edge key={i} a={pos[u]} b={pos[v]} color={inScc
    ))}
    {Object.entries(pos).map(([k, [x, y]]) => (
      <Node key={k} x={x} y={y} label={k} fill={step :
    ))}
  </Svg>
);
};

```

ФАЙЛ 44/82: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
  const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
  const step = useLoop(states.length, 1000);
  const heap = states[step];
  const pos: [number, number][] = [[130, 24], [80, 68],
  const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
  return (
    <Svg caption="Новый элемент всплывает наверх, пока
      <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
      <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
      {heap.map((v, i) => (
        <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
      ))}
    )
  );
};

```

```

    <g key={v} style={{ transition: "all .3s" }}>
      <rect x={30 + i * 54} y={56} width={46} height={18}>
        <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
      </rect>
    </g>
  )}
  <line x1={24} y1={71} x2={10} y2={71} stroke={C.d} />
</Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];
  const root = (i: number): number => (p[i] === i ? i : root(p[i]));
  return (
    <Svg caption="find = «кто твой староста», union = 0" >
      {p.map((par, i) =>
        par !== i ? (
          <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
            fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
        ) : null
      )}
      {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
    </Svg>
  );
};

```

```

const TRIE_NODES = [
  { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
  { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
  { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
  { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
  { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
  { id: 5, x: 188, y: 104, ch: "c", parent: 2 },

```

```

return (
  <Svg caption="Суффиксная ссылка: «куда откатиться,
    {TRIE_NODES.filter((n) => n.parent !== null).map(
      const p = TRIE_NODES[n.parent as number];
      return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
    ]}}
    {links.slice(0, step).map(([from, to], i) => {
      const a = TRIE_NODES[from];
      const b = TRIE_NODES[to];
      return (
        <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
          fill="none" stroke={C.warn} strokeWidth={1.
        )};
      )}}
    {TRIE_NODES.map((n) => (
      <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
    )}}
  </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const nodes = [
    { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
    { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
    { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
    { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
    { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
    { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
    { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
  ];
  return (
    <Svg caption="Запрос собирается из  $O(\log n)$  готовых
      {nodes.map((n, i) => (
        <g key={i}>
          <rect x={n.x - n.w / 2} y={n.y - 10} width={n
            fill={n.lvl === step ? C.active : C.idle} s

```

```
export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы дл
      {Array.from({ length: 8 }).map((_, i) => {
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={18} />
          <text x={25 + i * 30} y={29} textAnchor="middle" />
        </g>
      })}
      {Array.from({ length: count }).map((_, i) => {
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 13} width={26} height={18}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
      })}
    </Svg>
  );
};
```

```
export const AmortizedDemo: React.FC = () => {
  const costs = [1, 1, 1, 8, 1, 1, 1, 1];
  const step = useLoop(costs.length + 1, 550);
  const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
  const avg = step ? (total / step).toFixed(2) : "-";
  return (
    <Svg caption={`Сделано ${step} операций, суммарно $
      {costs.map((c, i) => {
        <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
          fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
      })}
      <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
      <text x={130} y={20} textAnchor="middle" fontSize={14} />
    </Svg>
  );
};
```

```

         $\pi[i]$  — длина самого длинного «префикс = суффикс»
      </text>
    </Svg>
  );
};

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>
              <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
                fill={idx === step - 1 ? C.active : fill}
              {filled && (
                <text x={57 + c * 38} y={36 + r * 32} textAnchor="middle"
                  fontSize={10} fontWeight="700" fill="black">
                  {idx}
                </text>
              )}
            </g>
          );
        )}
      )}
    </Svg>
  );
};

<text x={130} y={122} textAnchor="middle" fontSize={14}>
  каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

```

    mst: MstDemo,
    amortized: AmortizedDemo,
    np: NpDemo,
    scc: SccDemo,
    "prefix-function": PrefixFunctionDemo,
    dp: DpDemo,
    floyd: FloydDemo,
    components: ComponentsDemo,
    graph: GraphBasicsDemo,
    "coord-compress": CoordCompressDemo,
    zig: ZigDemo,
    "zig-zig": ZigZigDemo,
    "zig-zag": ZigZagDemo,
  };

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
  const [paused, setPaused] = useState(false);
  const Cmp = REGISTRY[kind];
  if (!Cmp) return null;
  return (
    <DemoPauseContext.Provider value={paused}>
      <div
        className="bg-slate-950/70 rounded-lg border bo
        onMouseEnter={() => setPaused(true)}
        onMouseLeave={() => setPaused(false)}
      >
        <Cmp />
      </div>
    </DemoPauseContext.Provider>
  );
};

```

```

        </div>
        <button
          onClick={onClose}
          className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
          aria-label="Закрыть симулятор"
        >
          <X className="w-5 h-5" />
        </button>
      </div>
    <div className="overflow-y-auto overscroll-contain" />
  </div>
</div>,
document.body
);
};

```

ФАЙЛ 47/82: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
  termId: string;
  rect: DOMRect;
}

interface Props {
  anchor: HintAnchor | null;
}

```



```

window.addEventListener("scroll", onClose, true);
return () => {
  window.removeEventListener("keydown", onKey);
  window.removeEventListener("scroll", onClose, true);
};
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500"
      <div className="flex items-start gap-2 px-3 pt-3"
        <h4 className="text-sm font-bold text-indigo-400"
          <button
            onClick={onClose}
            className="text-slate-500 hover:text-white"
            aria-label="Заккрыть подсказку"
          >
            <X className="w-4 h-4" />
          </button>
        </div>
      </div>
  >

```

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
  ];

  const isNodeVisible = (id: string) => {
    if (id === 'S' && (step === 0 || step === 7)) return true;
    return false;
  };

  const getPotential = (id: string) => {
    if (step >= 2 && step < 7) {
      if (id === 'S') return 0;
      if (id === 'A') return 0;
      if (id === 'B') return -2;
      if (id === 'C') return -1;
    }
    return null;
  };
}
```

```

    if (e.id === 'CA' && step === 6) return "url(#a";
    if (e.id === 'CA' && step > 6) return "url(#arr";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

  if (step < 4) return e.weight;

  if (e.id === 'AB') {
    if (step === 4) return `${e.weight} + (${0}`;
    return e.newWeight;
  }
  if (e.id === 'BC') {
    if (step < 5) return e.weight;
    if (step === 5) return `${e.weight} + (${-2}`;
    return e.newWeight;
  }
  if (e.id === 'CA') {
    if (step < 6) return e.weight;
    if (step === 6) return `${e.weight} + (${-1}`;
    return e.newWeight;
  }
  return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
  if (e.id === 'AB' && step === 4) return true;
  if (e.id === 'BC' && step === 5) return true;
  if (e.id === 'CA' && step === 6) return true;
  return false;
};

const isEdgeTextEmerald = (e: any) => {
  if (e.id === 'AB' && step > 4) return true;

```

```

        <div>
            <b>Шаг 3.1: Пересчет ребра A → B.</b>
            Старый вес: <span className="text-emerald-300">0</span>
            Вычисляем:  $-2 + 0 - (-2) =$  <span cla
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span cla
        </div>
    );
    case 6: return (
        <div>
            <b>Шаг 3.3: Пересчет ребра C → A.</b>
            Старый вес: 2. <br/>
            Вычисляем:  $2 + (-1) - 0 =$  <span cla
        </div>
    );
    case 7: return (
        <div className="text-emerald-300">
            <b>Готово!</b> Фиктивную вершину S
            Все новые веса ребер в графе стали
            При этом старые кратчайшие пути оста
        </div>
    );
    default: return "";
}
};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">
        <h4 className="text-sm md:text-base font-bo">
            <span className="w-2.5 h-2.5 bg-amber-400">
                Визуализация: Перевзвешивание алгоритмом
            </h4>

```

```

const edgeColorClass = getEdgeColorClass(i)
const textHigh = isEdgeTextHigh(i)
const textEmer = isEdgeTextEmer(i)

return (
  <g key={i}>
    <line
      x1={u.x} y1={u.y}
      className={`stroke-${edgeColorClass}`}
      markerEnd={getMarkerEnd(i)}
    />

    <text
      x={midX} y={midY}
      textAnchor="middle"
      dominantBaseline="bottom"
      className={`text-${edgeColorClass}`}
      style={{textHigh: textHigh, textEmer: textEmer}}
      isS ? 'font-size: 1.2em' : ''
    />
    {getEdgeContent(i)}
  </text>
</g>
)
})
})

```

```

{/* Nodes */}
{nodes.map(n => {
  if (!isNodeVisible(n.id)) return null
  const pot = getPotential(n.id)

  return (
    <g key={n.id} className={`node-${pot}>
      <circle
        cx={n.x} cy={n.y}
        className={`stroke-${pot}>

```

```

        <RotateCcw strokeWidth={3}
      </button>
      <button onClick={() => setStep(1)}
        className="p-3 bg-slate-800
        -slate-400 transition-colors" title="Шаг на
        <Undo strokeWidth={3} size=
      </button>
      <button onClick={() => setStep(
        className={`flex-1 font-bol
          ${step === MAX_STEP
            ? 'bg-emerald-600/50'
            : 'bg-indigo-600 ho
          {step === MAX_STEP ? "Готов
          {step !== MAX_STEP && <Skip
      </button>
    </div>

    {/* Progress Dots */}
    <div className="flex justify-center
      {Array.from({length: MAX_STEP +
        <div key={i} className={`w-1
      125' : i < step ? 'bg-indigo-900' : 'bg-sla
        )}}
      </div>
    </div>
  </div>
</div>
);
}

```

ФАЙЛ 49/82: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import {

```

```

    visited: Set<number>;
    currentNode: number | null;
    order: number[];
    transposed: boolean;
    sccMap: Record<number, number>; // node -> sccId
    currentSccId: number;
    activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
    const visited = new Set<number>();
    const order: number[] = [];
    const sccMap: Record<number, number> = {};

    const recordStep = (
      phase: AlgoStep["phase"],
      desc: string,
      currentNode: number | null,
      transposed: boolean,
      currentSccId: number,
    ) => {
      generatedSteps.push({
        phase,
        desc,
        visited: new Set(visited),
        currentNode,
        order: [...order],
        transposed,
        sccMap: { ...sccMap },
        currentSccId,
        activeEdges: transposed
      });
    };
  });
};

```

```

        u,
        false,
        -1,
    );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологическ
    ебра.`,
    null,
    true,
    -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
    visited.add(u);
    sccMap[u] = currentScc;
    recordStep(
        "dfs2",
        `DFS 2: заходим в ${u} на транспонированном гра
        u,
```



```

        null,
        true,
        sccId,
    );

    setSteps(generatedSteps);
    setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
  phase: "init",
  desc: "Загрузка...",
  visited: new Set(),
  currentNode: null,
  order: [],
  transposed: false,
  sccMap: {},
  currentSccId: -1,
  activeEdges: initialEdges,
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

```

```

<div className="grid grid-cols-1 lg:grid-cols-12" >
  { /* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] relative" >
    <svg className="w-full h-full max-w-[650px]" >
      { /* Markers for directed arrows */}
      <defs>
        <marker
          id="arrow"
          viewBox="0 0 10 10"
          refX="22"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#66BB6A" />
        </marker>
        <marker
          id="arrow-active"
          viewBox="0 0 10 10"
          refX="22"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#FFCDD2" />
        </marker>
        <marker
          id="arrow-transposed"
          viewBox="0 0 10 10"
          refX="22"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >

```

```

        {edge.u === 4 && edge.v === 3});
const dy = isBidi ? (edge.u === 3 ? -10 :

return (
  <line
    key={`edge-${idx}`}
    x1={x1}
    y1={y1 + dy}
    x2={x2}
    y2={y2 + dy}
    stroke={stroke}
    strokeWidth="2.5"
    markerEnd={`url(#${markerId})`}
    className="transition-all duration-300"
  />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited =
    step.visited.has(node.id) ||
    (step.phase === "dfs1" && step.currentNode === node.id);
  const sccId = step.sccMap[node.id];

  let classes = "fill-slate-800 stroke-slate-800";
  if (sccId) {
    classes = getSccColor(sccId);
  } else if (isCurrent) {
    classes = "fill-amber-600 stroke-amber-600";
  } else if (isVisited && step.phase === "dfs2") {
    classes = "fill-emerald-700 stroke-emerald-700";
  }

  return (
    <g key={node.id}>
      <circle

```

```

        1 rounded">
          Граф транспонирован (Ребра обратные)
        </div>
      })
    </div>

    { /* Logs & Stacks */ }
    <div className="lg:col-span-4 bg-slate-950 p-5
      00 overflow-y-auto">
      <h4 className="text-xs font-bold text-slate-400">
        Порядок обхода и стек
      </h4>

      <div className="bg-slate-900 border border-slate-800">
        <p className="text-xs text-white leading-relaxed">
          {step.desc}
        </p>
      </div>

      <div className="flex-1 space-y-4 overflow-y-auto">
        { /* Top-Sort output / Order stack representation */ }
        <div>
          <span className="text-[10px] text-slate-500">
            СТЕК ВЫХОДА (DFS 1 ORDER):
          </span>
          <div className="flex flex-wrap gap-1 bg-slate-900">
            {step.order.length === 0 ? (
              <span className="text-slate-600 text-sm">
                Нет элементов в стеке
              </span>
            ) : (
              [...step.order].reverse().map((val, idx) => {
                <div
                  key={idx}
                  className="bg-indigo-950 border border-indigo-800
                    px-2 py-1 text-xs text-white flex items-center gap-1"
                >
                  {val}
                </div>
              })
            )}
          </div>
        </div>
      </div>
    </div>
  )}
</div>

```

```
        <button
          disabled={currentStepIdx >= steps.length}
          onClick={() => setCurrentStepIdx((prev) => prev + 1)}
          className="bg-slate-900 border border-slate-500 text-white px-4 py-2 rounded"
        >
          Вперед
        </button>
      </div>
    </div>
  </div>
</div>
);
};
```

ФАЙЛ 50/82: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';

interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
}
```

```
interface Edge {
  id: string;
  u: number;
  v: number;
  weight: number;
```

```

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal')
  const [vertices, setVertices] = useState<Vertex[]>([])
  const [edges, setEdges] = useState<Edge[]>([])
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>([])
  const [activeCheatTab, setActiveCheatTab] = useState<string>('')

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Представь, что тебе нужно раскрасить граф, чтобы не было ребер между вершинами одного цвета. Опиши, как ты это сделаешь.',
      type: 'info',
    },
  ])

  // --- KRUSKAL STEPS ---
  const kruskalSteps = [
    // Step 1: Inspect A-B (2)
    () => {
      setEdges(prev => prev.map(e => e.id === '0-1' ? { ...e, color: 'red' } : e))
      setLogs(prev => [
        {
          title: 'Шаг 1: Берем самое дешевое ребро A-B (в данном случае это ребро 0-1)',
          description: 'Оно соединяет две бесцветные вершины. Оно самое дешевое, поэтому мы его берем.',
          type: 'info'
        },
        ...prev
      ])
    },
    // Step 2: Include A-B
    () => {
      setVertices(prev => prev.map(v => v.id === 0 || v.id === 1 ? { ...v, color: 'red' } : v))
      setEdges(prev => prev.map(e => e.id === '0-1' ? { ...e, color: 'red' } : e))
      setLogs(prev => [
        {
          title: 'Успех: Ребро A-B в остова',
          description: 'Вершины A and B окрашены в красный цвет. Ребро A-B включено в остова.',
          type: 'success'
        },
        ...prev
      ])
    },
  ]

```

```

    },
    // Step 7: Inspect A-D (5)
    () => {
        setEdges(prev => prev.map(e => e.id === '0-3' ? {
            setLogs(prev => [{
                title: 'Шаг 4: Берем ребро A-D (вес 5)',
                description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
                    ю – мы перекрашиваем всё в один цвет!',
                type: 'warning'
            }, ...prev]));
    },
    // Step 8: Include A-D (Merge to purple)
    () => {
        setVertices(prev => prev.map(v => v.color === 'blue' ? {
            setEdges(prev => prev.map(e => e.status === 'included' ? {
            setLogs(prev => [{
                title: 'Слияние кланов: Ребро A-D добавлено',
                description: 'Мы объединили красный и синий кланы',
                type: 'success'
            }, ...prev]));
    },
    // Step 9: Inspect B-E (6) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '1-4' ? {
            setLogs(prev => [{
                title: 'Шаг 5: Берем ребро B-E (вес 6)',
                description: 'НО: это ребро соединяет вершины B
                    type: 'warning'
            }, ...prev]));
    },
    // Step 10: Reject B-E
    () => {
        setEdges(prev => prev.map(e => e.id === '1-4' ? {
            setLogs(prev => [{
                title: 'Отказ: Цикл обнаружен!',
                description: 'Мы выбрасываем ребро B-E. Иначе по
                    type: 'error'
            }, ...prev]));
    },

```

```

    },
    // Step 15: Inspect C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Шаг 8: Берем ребро C-F (вес 9)',
                description: 'Оно соединяет вершины C и F, кото
                type: 'warning'
            }, ...prev]));
    },
    // Step 16: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Отказ: Цикл обнаружен!',
                description: 'Выбрасываем ребро C-F.',
                type: 'error'
            }, ...prev]));
    },
    // Step 17: Inspect G-H (10)
    () => {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
            setLogs(prev => [{
                title: 'Шаг 9: Берем ребро G-H (вес 10)',
                description: 'Соединяет фиолетовую G и бесцветн
                type: 'info'
            }, ...prev]));
    },
    // Step 18: Include G-H
    () => {
        setVertices(prev => prev.map(v => v.id === 7 ? {
            setEdges(prev => prev.map(e => e.id === '6-7' ? {
            setLogs(prev => [{
                title: 'Успех: Ребро G-H в осто
                description: 'Вершина H перекрашена в фиолетовы
                type: 'success'
            }, ...prev]));
    },

```



```

];

// --- PRIM STEPS ---
const primSteps = [
  // Step 1: Start at E
  () => {
    setVertices(prev => prev.map(v => v.id === 4 ? {
      setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-A (5)'],
      setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5']),
      setLogs(prev => [{
        title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
        description: 'Мы выбрали вершину E как стартовую точку и поместили ее в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7)',
        type: 'info'
      }, ...prev]));
  },
  // Step 2: Pop D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      setLogs(prev => [{
        title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
        description: 'Плесень тянется по ребру D-E к вершине D',
        type: 'info'
      }, ...prev]));
  },
  // Step 3: Include D-E, add D's edges to heap
  () => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
      setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-C (4)'],
    setLogs(prev => [{
      title: 'Успех: Плесень захватила вершину D',
      description: 'В кучу добавлены новые пути из D: A-D(5), D-C(4)',
      type: 'success'
    }, ...prev]));
  },
  // Step 4: Pop A-D (5)

```

```

        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
        description: 'ВНИМАНИЕ! Плесень проверяет вершину E.',
        type: 'warning'
    }, ...prev]);
},
// Step 11: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
    setLogs(prev => [{

```

```

    setEdges(prev => prev.map(e => e.id === '3-6' ? {
      eap' } : e));
    setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H (11)']);
    setLogs(prev => [{
      title: 'Успех: Вершина G захвачена',
      description: 'В кучу добавлено G-H(10).',
      type: 'success'
    }, ...prev]);
  },
  // Step 16: Pop C-F (9) - Cycle!
  () => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
      eap' } : e));
    setLogs(prev => [{
      title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
      description: 'Вершины C и F уже заражены плесенью',
      type: 'warning'
    }, ...prev]);
  },
  // Step 17: Reject C-F
  () => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
      eap' } : e));
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)']);
    setLogs(prev => [{
      title: 'Отказ: Игнорируем ребро C-F',
      description: 'Выбрасываем из кучи.',
      type: 'error'
    }, ...prev]);
  },
  // Step 18: Pop G-H (10)
  () => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
      eap' } : e));
    setLogs(prev => [{
      title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
      description: 'Плесень ползет к вершине H.',
      type: 'info'
    }, ...prev]);
  },
  // Step 19: Include G-H, add H's edges

```

```

    },
    // Step 23: Include H-I
    () => {
        setVertices(prev => prev.map(v => v.id === 8 ? {
        setEdges(prev => prev.map(e => e.id === '7-8' ? {
        setHeapQueue(['F-I (13 - цикл)']));
        setLogs(prev => [{
            title: '      Успех: Плесень захватила весь граф
            description: 'Все 9 вершин поглощены. Итоговый
                иной – мы разрастались из одной точки с помо
            type: 'success'
        }, ...prev]));
    },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
        setEdges(prev => prev.map(e =>
            ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
        ]));
        setLogs(prev => [{
            title: 'Первая пятилетка: Каждая деревня выбира
            description: 'Каждая из 9 деревень одновременно
                А выбирает A-B(2), D выбирает D-E(3), а G вы
                !',
            type: 'info'
        }]);
    }
];

```

```

    }, ...prev]));
  },
  // Step 4: Concurrently merge into single Kolkhoz
  () => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
      e.status === 'included' || e.id === '0-3' ? { .
    }));
    setLogs(prev => [{
      title: 'Объединение завершено: Один гигантский
      description: 'Оба колхоза объединились по мосту
      type: 'success'
    }, ...prev]));
  },
  // Step 5: Check remaining edges & mark as rejected
  () => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
      title: ' Успех: MST построен Борувкой за 2 шаг
      description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
        вес: 51. Благодаря параллельности, мы потрати
      type: 'success'
    }, ...prev]));
  }
];

const currentSteps =
  activeAlgo === 'kruskal' ? kruskalSteps :
  activeAlgo === 'prim' ? primSteps :
  boruvkaSteps;

const handleNextStep = () => {
  if (stepIndex < currentSteps.length) {
    currentSteps[stepIndex]();
    setStepIndex(stepIndex + 1);
  }
};

```

```

    case 'blue': return 'bg-blue-500 border-blue-300';
    case 'purple': return 'bg-purple-600 border-purple-300';
    case 'mold': return 'bg-emerald-500 border-emerald-300';
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIndex === 0)
        return 'bg-slate-700 border-emerald-500 text-emerald-500';
      return 'bg-slate-700 border-slate-500 text-slate-500';
  }
};

```

```

const getEdgeStroke = (status: string, color: string) => {
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

```

```

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800 p-4">
      <div className="flex flex-col lg:flex-row justify-between items-center">
        <div>
          <div className="flex items-center gap-2 mb-2">
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-bold uppercase tracking-wider">
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white">
              MST Simulator
            </h3>
          </div>

```

```

+
    (activeAlgo === 'boruvka'
      ? 'bg-rose-600 text-white shadow-md'
      : 'text-slate-400 hover:text-slate-500')
  }
>
  <Users className="w-3.5 h-3.5" />
  <span>Борувка (Билет 20)</span>
</button>
</div>

<button
  onClick={() => handleReset(activeAlgo)}
  className="flex items-center justify-center gap-2
    -200 font-semibold text-xs rounded-xl border-1"
>
  <RotateCcw className="w-3.5 h-3.5" />
  <span>Сбросить</span>
</button>

<button
  onClick={handleNextStep}
  disabled={stepIndex >= currentSteps.length}
  className={
    "flex items-center justify-center gap-2
    initial cursor-pointer " +
    (stepIndex >= currentSteps.length
      ? "bg-slate-800 text-slate-500 border-1"
      : activeAlgo === 'kruskal'
      ? "bg-indigo-600 hover:bg-indigo-500 border-1"
      : activeAlgo === 'prim'
      ? "bg-emerald-600 hover:bg-emerald-500 border-1"
      : "bg-rose-600 hover:bg-rose-500 text-white")
  }
>
  <SkipForward className="w-4 h-4" />
  <span>{stepIndex === 0 ? 'Начать' : stepIndex + 1}</span>
</button>

```



```

</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom-6
    shadow-xl z-20">
    <p className={"font-bold text-base " +
      'ld-300' : 'text-rose-300'}>
      MST построено с помощью {activeAlgo}
    </p>
    <p className="text-slate-300 text-xs mt-2">
      Общий вес минимального остова: 51. Всего
    </p>
  </div>
)}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border-b
  <div className="p-4 bg-slate-900 border-b
    <h4 className="font-bold text-slate-200">
      Ход выполнения
    </h4>
  </div>

  {activeAlgo === 'prim' && (
    <div className="p-3 bg-slate-900/60 border
      <p className="text-[11px] font-bold text-slate-200">
        ✂ Приоритетная очередь (Min-Heap):
      </p>
      <div className="flex flex-wrap gap-1.5">
        {heapQueue.length === 0 ? (
          <span className="text-xs text-slate-400">
            Пусто
          </span>
        ) : (
          heapQueue.map(({item, idx}) => (
            <span key={idx} className={"px-2 py-1
              0 shadow-md animate-pulse" : "bg-slate-800
                {item}
              </span>
            )
          )
        )
      </div>
    )
  )
}

```

```

<div className="flex flex-col sm:flex-row items
  <div className="flex items-center gap-2.5">
    <BookOpen className="w-5 h-5 text-indigo-400">
    <h4 className="text-xl font-bold text-white">
  </div>

  <div className="flex bg-slate-950 p-1 rounded
    <button
      onClick={() => setActiveCheatTab('kruskal')}
      className={"px-3 py-1.5 rounded text-xs font-medium"}
      == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-950 text-white"
    >
      Краскал (Билет 18)
    </button>
    <button
      onClick={() => setActiveCheatTab('prim')}
      className={"px-3 py-1.5 rounded text-xs font-medium"}
      == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-white"
    >
      Прима (Билет 19)
    </button>
    <button
      onClick={() => setActiveCheatTab('boruvka')}
      className={"px-3 py-1.5 rounded text-xs font-medium"}
      == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-white"
    >
      Борувка (Билет 20)
    </button>
  </div>
</div>

{activeCheatTab === 'kruskal' && (
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-lg">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-400">
        </p>

```

```
edges.sort() # O(E log E)
```

```
mst = []
```

```
for w, u, v in edges:
```

```
    if union(u, v):
```

```
        mst.append((u, v, w))`}
```

```
    </pre>
```

```
    </div>
```

```
    </div>
```

```
    </div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
  <div className="grid grid-cols-1 lg:grid-cols-
```

```
    <div className="lg:col-span-1 space-y-4">
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
          <Clock className="w-3.5 h-3.5 text-em
```

```
        </p>
```

```
        <p className="text-2xl font-black text-v
```

```
        <p className="text-slate-400 text-xs mt
```

```
      </div>
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
          <Award className="w-3.5 h-3.5 text-em
```

```
        </p>
```

```
        <p className="text-2xl font-black text-v
```

```
        <p className="text-slate-400 text-xs mt
```

```
      </div>
```

```
      <div className="bg-slate-950 p-4 rounded-
```

```
        <p className="text-slate-400 text-xs for
```

```
        ложении:</p>
```

```
        <p className="text-xs text-slate-300 le
```

```
        ие чистые вершины через самое дешевое досту
```

```
      </div>
```

```
    </div>
```

```
  <div className="lg:col-span-2">
```

```
    <div className="bg-slate-950 p-4 rounded-
```

```
      <p className="text-slate-400 text-xs for
```

```

<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs fo
    <Award className="w-3.5 h-3.5 text-ro
  </p>
  <p className="text-2xl font-black text-r
  <p className="text-slate-400 text-xs mt
</div>
<div className="bg-slate-950 p-4 rounded-1
  <p className="text-slate-400 text-xs fo
ении:</p>
  <p className="text-xs text-slate-300 le
, организовав массовое распараллеленное слияни
  </div>
</div>
<div className="lg:col-span-2">
  <div className="bg-slate-950 p-4 rounded-1
    <p className="text-slate-400 text-xs fo
      <Code className="w-3.5 h-3.5 text-ros
    </p>
    <pre className="text-xs text-emerald-400
80 flex-1">
{`def boruvka(n, edges):
    parent = list(range(n))
    mst = []
    while len(mst) < n - 1:
        # Для каждого колхоза ищем самый дешёвый мост н
        cheapest = [-1] * n
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2]
                if cheapest[rv] == -1 or cheapest[rv][2]
        # Объединяем все колхозы по выбранным мостам
        for bridge in cheapest:
            if bridge != -1 and union(bridge[0], bridge
                mst.append(bridge)`}
    </pre>
  </div>

```

```

    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '    Все ко Все (Флойд)',
    desc: 'Матрица + три цикла (',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: ')). Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

```

```

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover group-hover:scale-110"/>
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.highlightColor}`}>{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.highlightColor}`}>{c.complexity}</span></p>
            <div className={`mt-3 text-xs font-mono px-2`}

```

```

    }, [open]));

useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
}, []);

return (
    <>
    <div className="lg:hidden sticky top-[57px] z-40" >
        <button
            onClick={() => setOpen(true)}
            className="w-full flex items-center gap-2.5 p-2 text-slate-400"
            aria-label="Открыть содержание"
        >
            <span className="p-1.5 rounded-lg bg-indigo-600 text-white" >
                <List className="w-4 h-4" />
            </span>
            <span className="min-w-0 flex-1" >
                <span className="block text-[10px] uppercase" >
                    Содержание • {index + 1} из {total}
                </span>
                <span className="block text-[13px] font-bold" >
                    {currentSection}
                </span>
            </span>
        </button>
    </div>

    {open && (
        <div className="lg:hidden fixed inset-0 z-[90]" >
            <div className="absolute inset-0 bg-slate-950" >
                <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
                    te-[tocIn_.18s_ease-out]" >
                    <div className="flex items-center justify-between" >
                        <span className="font-bold text-white text-[12px]" >
                            Содержание
                        </span>
                        <button
                            onClick={() => setOpen(false)}
                            className="p-2 -mr-2 text-slate-400 hover:text-white" >
                                Закрыть
                            </button>
                    </div>
                </div>
            </div>
        )}
    </>
)

```

```

const saveBook = async () => {
  setBusy(true);
  try {
    await downloadBookHtml(chapters);
  } finally {
    setBusy(false);
  }
};

return (
  <header className="bg-slate-900 border-b border-slate-800">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex items-center gap-3 w-full">
        <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-10 h-10 rounded-full flex items-center justify-center">
          <Sparkles className="w-6 h-6" />
        </div>
        <div className="min-w-0 flex-1">
          <h1 className="text-xl font-extrabold text-white">
            Универсальное пособие
          </h1>
          <p className="text-xs text-indigo-400 font-medium">
            Подготовка к экзаменам: Алгоритмы, Структуры данных, Математика
          </p>
        </div>
      </div>
    </div>

    <div className="flex items-center w-full lg:w-auto">
      <button
        id="tab-guide-btn"
        onClick={() => setActiveTab('guide')}
        className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-white font-medium duration-200 whitespace-nowrap ${
          activeTab === 'guide'
            ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        Гид
      </button>
    </div>
  </div>
);

```

```
        <FileText className="w-4 h-4" /> TXT
      </a>
      <button
        id="download-html-btn"
        onClick={saveBook}
        disabled={busy}
        className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
      >
        {busy ? <Loader2 className="w-4 h-4 animate
      </button>
    </div>
  </div>
</header>
);
};
```

ФАЙЛ 54/82: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border
          <div className="absolute top-2 left-2 bg-slate-
            00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0
            {(() => {
              const pts = [
```



```

        const labels = ["A", "B", "C", "D", "E"];
        for(const p of pts) {
            circles.push(<g key={`c${p.id}`}>
                <circle cx={p.x} cy={p.y} r={8} fill=
                <text x={p.x} y={p.y+3} textAnchor="m
            </g>);
        }
        return <>{lines}{circles}</>;
    }}())}
</svg>
<div className="absolute bottom-2 left-2 right
    z-20">
    V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
    <div className="absolute top-2 left-2 bg-slate
        00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
    <svg className="w-full h-full absolute inset-0
        {(() => {
            const pts = [
                {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
                {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
            ];
            const edges = [
                [0,3],[0,4],[0,5],
                [1,3],[1,4],[1,5],
                [2,3],[2,4],[2,5]
            ];
            function findPt(id:number) { return pts.f
            function intersect(a:number,b:number,c:nu
                if (a===c||a===d||b===c||b===d) return
                const p1=findPt(a),p2=findPt(b),p3=find
                if (!p1||!p2||!p3||!p4) return false;
                function ccw(ax:number,ay:number,bx:num
                    return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)

```

```
        </div>
      </div>
    );
  }
}
```

ФАЙЛ 55/82: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';

interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue', bubbleText: 'Матана!' },
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet', bubbleText: 'Рацию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'Плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', bubbleText: '' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', bubbleText: 'Пиво для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '👔', color: 'from-slate', bubbleText: 'Тартапы.' },
];
```

```

    setQueue(prev => [...prev, newGuest]);
    addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

    setTimeout(() => {
      setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c), 450);
    }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (isServing) return;
  if (queue.length === 0) {
    setShake(true);
    addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsServing(true);
  const headGuest = queue[0];

  addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

  // Step 1: Head guest gets eating animation
  setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '🍲' } : c));

  setTimeout(() => {
    // Step 2: Guest leaves with happy face, queue shrinks
    setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, emoji: '😊' } : c));

    setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

    setTimeout(() => {
      setQueue(prev => prev.slice(1));
      setIsServing(false);
      addLog(`- ${headGuest.name} поел и ушел сытым. 🍴`);
    }, 400);
  }, 900);
}, [queue, isServing]);

```

```

        onClick={enqueue}
        disabled={isServing}
        className="flex-1 min-w-[150px] bg-gradient-to-r
          from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
          font-mono text-sm py-2 px-3 rounded shadow-lg hover:scale-95 transition-all cursor-pointer"
      >
        <span> Встать в очередь (ENQUEUE)</span>
      </button>

    <button
      onClick={dequeue}
      disabled={isServing || queue.length === 0}
      className="flex-1 min-w-[150px] bg-gradient-to-r
        from-slate-800 to-slate-900 opacity-40 disabled:cursor-not-allowed text-white
        font-mono text-sm py-2 px-3 rounded shadow-lg hover:scale-95 transition-all cursor-pointer flex
        items-center justify-center"
    >
      <span> Налить суп первому (DEQUEUE)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono text-sm
        cursor-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12 gap-4" >

    { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
    <div className={`md:col-span-9 bg-slate-900 rounded-2xl p-4
      min-h-[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
      <div className="absolute top-4 left-4 text-white font-mono text-sm" >
        Очередь голодных гостей (Буфер FIFO / BFS)
      </div>

```

```

const isTail = idx === queue.length
return (
  <div
    key={customer.id}
    className={`
      flex flex-col items-center gap-
      ${customer.animState === 'in'
        ? '10px'
        : '20px'}
      ${customer.animState === 'eat'
        ? 'animate-eat'
        : ''}
      ${customer.animState === 'out'
        ? 'animate-out'
        : ''}
    `}
  >
    {/* Облачко мыслей */}
    <div className={`
      absolute -top-16 left-1/2 -tr
      8 text-center text-slate-200 leading-tight
      ${isHead ? 'opacity-100 border-1px solid black'
        : 'opacity-0'}
    `}>
      <div className="font-bold text-slate-200">
        {customer.bubbleText}
      <div className="absolute -bottom-16 left-1/2 -right-1/2
        it rotate-45"></div>
    </div>

    {/* Персонаж */}
    <div className={`
      relative
      w-14 h-14 rounded-2xl border-2
      ${customer.color}
      ${isHead ? 'ring-4 ring-amber-500'
        : ''}
    `}>
      <span className="text-3xl">{customer.name}</span>

      {/* Индексы HEAD/TAIL */}
      {(isHead || isTail) && (
        <span className={`
          absolute -bottom-2 text-[0.8em]
          ${isHead ? 'bg-amber-500 text-black'
            : 'bg-slate-200 text-slate-200'}
        `}>

```

```

        {servedHistory.map(({item, idx}) => {
            <div
                key={idx}
                className="w-full py-1.5 px-3 rounded
0 flex items-center gap-2 text-[11px] text-
            >
                <span>{item}</span>
            </div>
        })}
    </div>
    </div>

    {/* Пол */}
    <div className="w-full h-3 bg-gradient-to-r f
    </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
        {log.map(({entry, idx}) => {
            <div
                key={idx}
                className={`text-xs font-mono flex items-ce
            >
                {idx === 0 ? <span className="text-indigo-5
                <span>{entry}</span>
            </div>
        })}
    </div>
</div>
});
};

```

```

        setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
}, []));

// Эффект разговора при смене текста
useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false),
    return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
    const newNodes: { [key: number]: TrieNode } = {
        0: { id: 0, char: 'ROOT', parent: 0, children: {}
    };
    let count = 0;

    wordList.forEach((word) => {
        let curr = 0;
        for (let i = 0; i < word.length; i++) {
            const c = word[i].toUpperCase();
            if (newNodes[curr].children[c] === undefined) {
                count++;
                newNodes[curr].children[c] = count;
                newNodes[count] = {
                    id: count,
                    char: c,
                    parent: curr,
                    children: {},
                    fail: 0,
                    term_link: 0,
                    is_terminal: false,
                    words: [],
                    depth: newNodes[curr].depth + 1
                };
            }
        }
    });
}

```

```

    setSpeechText(
      `Отлично! Я построил базовое префиксное дерево (БПД)
      S, чтобы расставить fail-ссылки!`
    );
  };

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем построить БПД');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};

// Занульк BFS
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];

```



```

    while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
      v = updatedNodes[v].fail;
    }
    const failTarget = updatedNodes[v].children[char] !== undefined ?
      updatedNodes[v].children[char] : updatedNodes[v].fail;
    updatedNodes[u].fail = failTarget;

    // Рассчитываем терминальную ссылку (term_link)
    if (updatedNodes[failTarget].is_terminal) {
      updatedNodes[u].term_link = failTarget;
    } else {
      updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
    }

    logs += `Для #${u} ('${char}') fail = #${failTarget}`;
  }

  if (childrenIds.length === 0) {
    logs += 'У этой вершины нет детей, просто переходим к следующему';
  }

  setNodes(updatedNodes);
  setBfsQueue(newQueue);
  setCurrentNode(r);
  setSpeechText(logs);
};

return (
  <div className="bg-slate-800/90 rounded-2xl p-6 border" >
    {/* Шапка виджета */}
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center space-x-3" >
        <span className="text-3xl" > </span>
        <div>
          <h4 className="text-2xl font-bold text-white" >
            Интерактивный конструктор: Говорящий Эхо-
          </h4>
          <p className="text-sm text-slate-400" >
            Построение дерева и расчет суффиксных ссы
          </p>
        </div>
      </div>
    </div>
  </div>
);

```

```

    { /* Плавники */ }
    <path d="M 90 55 L 70 30 L 110 55 Z" fill="black" />
    <path d="M 90 145 L 70 170 L 110 145 Z" fill="black" />
    <path
      d="M 120 115 L 100 135 L 130 130 Z"
      fill="#f87171"
      className="origin-[120px_115px] animate"
    />

    { /* Глаз (с морганием) */ }
    <circle cx="145" cy="85" r="12" fill="black" />
    { isBlinking ? (
      <line x1="133" y1="85" x2="157" y2="85" />
    ) : (
      <>
        <circle cx="148" cy="85" r="6" fill="black" />
        <circle cx="151" cy="82" r="2" fill="black" />
      </>
    ) }

    { /* Рот (анимация разговора) */ }
    { isTalking ? (
      <path d="M 165 105 Q 155 120 165 125 Q" />
    ) : (
      <path d="M 165 110 Q 155 115 168 118" stroke="black" />
    ) }

    { /* Улыбка / жабры */ }
    <path d="M 130 80 Q 125 100 130 120" stroke="black" />
    <path d="M 120 85 Q 115 100 120 115" stroke="black" />
  </svg>

  { /* Декоративные пузыри */ }
  <div className="absolute -top-1 right-4 text-white" />
  <div className="absolute top-8 right-1 text-white" />
</div>
<span className="mt-3 bg-rose-600/30 border border-rose-600/30" />

```

```

        <button
          onClick={stepBFS}
          className="bg-blue-600 hover:bg-blue-500
            shadow-blue-900/40 transition-transform"
        >
          <span>Шаг BFS</span>
          <span className="bg-blue-800 px-1.5 py-0.5">
            </span>
        </button>
      )}
    {simulationStep === 2 && (
      <button
        onClick={() => buildInitialTrie(words)}
        className="bg-indigo-600 hover:bg-indigo-500
          transition-transform active:scale-95"
      >
        <RotateCcw className="w-4 h-4" /> Перезапустить
      </button>
    )}
  </div>
</div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
  <h5 className="text-white font-bold mb-3 flex items-center">
    <span>Дерево и Суффиксные (fail) ссылки</span>
  </h5>

```

```

  {(() => {
    let maxX = 0;
    let maxY = 0;
    Object.values(nodes).forEach(n => {
      if (n.x && n.x > maxX) maxX = n.x;
      if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

```

```

        {node.char}
      </text>
    </g>
  );
}}

{/* Draw Fail Links */}
{simulationStep > 0 && Object.values(nodes).map(node => {
  if (node.id === 0 || node.fail === 0) return null;
  // Wait, all nodes get fail link. Let's draw it at simulationStep
  // To avoid too many lines, let's draw it at simulationStep
  const target = nodes[node.fail];
  if (!target || !node.x || !node.y || !target.x || !target.y) return null;

  // Simple bezier curve for fail link
  const isCurrent = currentNode === node.id;

  return (
    <path
      key={`fail-${node.id}`}
      d={`M ${node.x} ${node.y - 10} Q ${node.x + 16} ${node.y - 16} ${node.x + 16} ${node.y - 10}`}
      fill="none"
      stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
      strokeWidth={isCurrent ? 2 : 1.5}
      strokeDasharray="4 4"
      markerEnd="url(#fail-arrow)"
    />
  );
})}

{/* Draw Nodes */}
{Object.values(nodes).map(node => {
  if (!node.x || !node.y) return null;
  const isRoot = node.id === 0;
  const isCurrent = currentNode === node.id;

```

```

        x={node.x + 12}
        y={node.y - 12}
        fill="#94a3b8"
        fontSize="9"
      >
        {node.id}
      </text>
    })
  </g>
);
}}}
</svg>
);
}}({})
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700
              px-2 py-1 rounded text-white font-medium"
          >
            {w}
          </span>
          <button
            onClick={() => removeWord(w)}
            className="text-slate-500 hover:text-slate-400
              font-medium text-sm"
            title="Удалить"
          >
            ✕
          </button>

```

```

<thead>
  <tr className="border-b border-slate-700">
    <th className="py-2 px-3">ID Вершины</th>
    <th className="py-2 px-3">Символ</th>
    <th className="py-2 px-3">Родитель</th>
    <th className="py-2 px-3">fail-ссылка</th>
    <th className="py-2 px-3">term_link</th>
    <th className="py-2 px-3">Дети</th>
    <th className="py-2 px-3">Слова (dict)
  </tr>
</thead>
<tbody className="divide-y divide-slate-800">
  {Object.values(nodes).map((node) => {
    const isCurrent = currentNode === node
    const isInQueue = bfsQueue.includes(node)

    return (
      <tr
        key={node.id}
        className={`transition-colors ${
          isCurrent
            ? 'bg-blue-900/40 font-bold text-white'
            : isInQueue
            ? 'bg-slate-800/80'
            : 'hover:bg-slate-800/40'
          }`}
      >
        <td className="py-2.5 px-3 flex items-center">
          <span
            className={`w-2 h-2 rounded-full ${
              node.id === 0
                ? 'bg-purple-500'
                : isCurrent
                ? 'bg-blue-400 animate-pulse'
                : isInQueue
                ? 'bg-amber-400'
                : 'bg-slate-600'
            }`}
          />
        </td>
      </tr>
    )
  })}

```

```

        <td className="py-2.5 px-3">
          {node.words.length === 0 ? (
            <span className="text-slate-600">
              {node.words.join(', ')}
            </span>
          ) : (
            <span className="bg-emerald-900 text-white">
              {node.words.join(', ')}
            </span>
          )}
        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 57/82: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```
// ===== TYPES =====
```

```

interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
}

```

```

    };
    void CODE;

    function go(v: number, l: number, r: number) {
        if (l === r) {
            steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
                , highlightNodes: [v], arrayHighlight: [l, r] });
            tree[v] = arr[l];
            steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}, r: ${r}` });
            return;
        }
        const m = (l + r) >> 1;
        steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
            { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
        go(2 * v, l, m);
        steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
            highlightNodes: [v], arrayHighlight: [l, r] });
        go(2 * v + 1, m + 1, r);
        tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
        steps.push({ id: id++, desc: `Объединяем: tree[${v}] = ${tree[2 * v]} + ${tree[2 * v + 1]}`,
            tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
    }
    go(1, 0, n - 1);
    steps.push({ id: id++, desc: `Дерево построено! Корень: ${tree[1]}`,
        Highlight: [0, n - 1] });
    return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

```

```

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {

```



```

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l]`});

while (l < r) {
  const highlights: number[] = [];
  if (l & 1) {
    res += tree[l] ?? 0;
    steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res });
    highlights.push(l);
    l++;
  }
  if (r & 1) {
    r--;
    res += tree[r] ?? 0;
    steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res });
    highlights.push(r);
  }
  l >>= 1;
  r >>= 1;
  if (l < r) {
    steps.push({ id: id++, desc: `Поднимаемся: l = $`, codeLine: 7, treeState: tree, highlightNodes: [l, r], arrayHighlight: [qL, qR], result: res });
  }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], ${qL}, ${qR})`, codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? 0 };
  const m = (l + r) >> 1;

```

```
];
```

```
// ===== PLAYER COMPONENT =====
```

```
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);
```

```
  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);
```

```
  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);
```

```
  const step = steps[idx] || steps[0];
  if (!step) return null;
```

```
  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, slate: { btn: 'bg-slate-600 hover:bg-slate-500 border-slate-500 bg-slate-600', childRing: 'ring-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } }[color];
```

```
  return { step, idx, setIdx, playing, setPlaying, speed };
}
```

```
// ===== SIMULATION PANEL =====
```

```
function SimPanel({ title, icon, steps, code, color, arr }: { title: string; icon: string; steps: Step[]; code: string; color: string; arr: number[] }) {
  | 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed } = p;
  const n = arr.length;
```

```

    -emerald-950/50' : 'bg-amber-950/50'} border-b
    <h4 className="font-bold text-sm text-white fle
    <span className="text-[10px] font-mono text-sla
    </div>

```

```

{/** Array strip */}
<div className="px-3 py-2 bg-slate-950 border-b b
  {arr.map((v, i) => {
    const inRange = step.arrayHighlight && i >= s
    return (
      <div key={i} className={`flex flex-col item
      go' ? 'bg-indigo-950 border-indigo-500' : c
      er-500'} -translate-y-0.5` : 'bg-slate-900 l
      <span className="text-[7px] text-slate-50
      <span className="text-[11px] font-bold for
      </div>
    );
  })}
</div>

```

```

{/** Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
  <div className="scale-[0.85] origin-top min-w-m
  </div>

```

```

{/** Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10p
  x">
  {code.map((line, i) => {
    const ln = i + 1;
    const active = step.codeLine === ln;
    return (
      <div key={ln} className={`px-2 py-0.5 round
      2 border-indigo-500 text-indigo-200' : colo
      ' : 'bg-amber-600/20 border-l-2 border-ambe
      <span className="inline-block w-4 text-sl
      </div>
    );
  })}

```

```

        </div>

        {/* Progress bar */}
        <div className="h-1 bg-slate-950"><div className=
          + 1) / total) * 100 : 0}%` }} /></div>
      </div>
    );
  }

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]
  const [customInput, setCustomInput] = useState('5, 8,
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => pars
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  };

  const randomize = () => {
    const size = arr.length;
    const a = Array.from({ length: size }, () => Math.f
    setArr(a);
    setCustomInput(a.join(', '));
  };
}

```

```

        <span className="text-xs text-slate-500 font-medium">
        </div>
    </form>

    {/* Query range */}
    <div className="bg-slate-950 p-4 rounded-xl border border-slate-800">
        <label className="text-[10px] font-bold uppercase">
        <div className="flex items-center gap-2">
            <div className="flex items-center gap-1">
                <span className="text-xs text-slate-400">
                <input
                    type="number"
                    min={0}
                    max={arr.length - 1}
                    value={qL}
                    onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
                    className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
                />
            </div>
        </div>
        <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400">
            <input
                type="number"
                min={0}
                max={arr.length - 1}
                value={qR}
                onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
                className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
            />
        </div>
    </div>
    <div className="text-[10px] text-slate-500 font-medium">
        Запрос:  $\text{sum}(A[\{qL..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((acc, val) => acc + val, 0))\}$ 
    </div>
</div>
</div>

```

```

        title={`Запрос sum([${qL}..${qR}]) – Сверху`}
        icon="↑"
        steps={queryTDSteps}
        code={QUERY_TD_CODE}
        color="emerald"
        arr={arr}
      />
    <SimPanel
      key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу`}
      icon="↓"
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
      color="amber"
      arr={arr}
    />
  </div>
)}

{view === 'theory' && (
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl b...
      <h4 className="text-base font-bold text-whi...
        во разбилось именно так для N={arr.length}?
      <p className="text-sm text-slate-300 leadin...
        Каждый узел берёт <code className="bg-sla...
        к получает <code className="bg-slate-900 px...
        -slate-900 px-1 rounded font-mono text-emerg...
        евая часть забирает на 1 больше (округление
      </p>
      <div className="bg-slate-900 p-4 rounded-lg...
        <div className="text-slate-400">Корень: L
        <div className="text-slate-300">mid = l(0
        <div className="text-indigo-300">→ Левый:
        <div className="text-emerald-300">→ Правы
        h - 1) >> 1)) эл.)</div>
        <div className="text-slate-500 mt-2">Всег

```

```

        <div className="flex justify-between"><
          00 font-bold">x сложно</span></div>
        </div>
      </div>
    </div>

    {/* Verdict */}
    <div className="bg-slate-950 p-5 rounded-xl b
      <h4 className="font-bold text-blue-400 mb-2
        <p className="text-sm text-slate-300 leadin
          <strong className="text-white">Итеративны
            кальности. Но если нужны <strong className=
              ame="text-white">рекурсивный (сверху-вниз)<
            </p>
          </div>
        </div>
      </div>
    )}
  </div>
);
}

```

ФАЙЛ 58/82: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Мобильный режим: список живёт в выдвижной панели */
  onNavigate?: () => void;
}

```

```

        </button>
      })
    </div>
  </div>

  <div className="flex-1 overflow-y-auto px-3 pb-4"
    {groups.length === 0 && <p className="text-sm text-gray-500">
      No groups found.
    </p>
    {groups.map(({category, items}) => (
      <div key={category}>
        <div className="text-[10px] font-bold uppercase">
          {category}
        </div>
        <div className="space-y-1">
          {items.map((chapter) => {
            const isSelected = chapter.id === selectedChapterId
            const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
            return (
              <button
                key={chapter.id}
                onClick={() => {
                  setSelectedChapterId(chapter.id);
                  onNavigate?.();
                  window.scrollTo({ top: 0, behavior: 'smooth' });
                }}
                className={`w-full text-left px-3 py-2 ${
                  isSelected
                    ? "bg-indigo-600/15 border-indigo-600/20"
                    : "bg-slate-800/40 border-transparent"
                }`}
              >
                {chapter.title}
              </button>
            )
          })}
        <div
          {hasViz && (
            <span
              className="w-1.5 h-1.5 rounded-full"
              title="Есть интерактивная визуализация"
            />
          )}
        </div>
        <span className="font-semibold text-gray-500">
          {chapter.count}
        </span>
        <ChevronRight
          className={`w-4 h-4 shrink-0 mt-0.5`}
        />
      </div>
    )}
  </div>

```



```

    icon: string;
  }

interface Person {
  id: string;
  name: string;
  icon: string;
}

interface Hypothesis {
  id: string;
  text: string;
  probability: number;
}

export const Simulator: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'stack' | 'queue' | 'heap'>('stack')

  // Stack state
  const [stack, setStack] = useState<Plate[]>([
    { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
    { id: '3', name: 'Блюдец от торта', icon: '🍰' },
  ]);
  const [popMessage, setPopMessage] = useState<string | null>(null)

  // Queue state
  const [queue, setQueue] = useState<Person[]>([
    { id: '1', name: 'Студент Вася', icon: '👨' },
    { id: '2', name: 'Голодный кодер', icon: '💻' },
    { id: '3', name: 'Кот Борис', icon: '🐈' },
  ]);
  const [dequeueMessage, setDequeueMessage] = useState<string | null>(null)

  // Heap state
  const [heap, setHeap] = useState<Hypothesis[]>([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.5 },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусные блинчики"', probability: 0.5 }
  ])

```

```

    { id: '2', name: 'Тарелка из-под пиццы', icon: '
    { id: '3', name: 'Блюде от торта', icon: ' ' },
  });
  setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
  const presets = [
    { name: 'Любитель борща', icon: ' ' },
    { name: 'Курьер доставки', icon: ' ' },
    { name: 'Усталый админ', icon: ' ' },
    { name: 'Турист с рюкзаком', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() *
  const newPerson = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue(prev => [...prev, newPerson]);
  setDequeueMessage(` Встал в конец очереди: ${newPe
};

const servePerson = () => {
  if (queue.length === 0) {
    setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
    return;
  }
  const firstPerson = queue[0];
  setQueue(prev => prev.slice(1));
  setDequeueMessage(` Выдан суп первому в очереди (F
};

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },

```

```

    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
    { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
  ]));
  setHeapMessage("  Гипотезы восстановлены.");
};

```

```

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

```

```

return (
  <div className="bg-slate-900/90 border border-slate-800 p-3">
    <div className="border-b border-slate-800 pb-6 mb-6">
      <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-purple-500">
        Интерактивный симулятор структур данных
      </h2>
      <p className="text-slate-400 text-sm mt-2">
        Проверь работу принципов LIFO, FIFO и Приоритетной очереди
      </p>
    </div>
  </div>

```

```

  { /* Tabs */ }
  <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
    <button
      onClick={() => setActiveTab('stack')}
      className={`flex items-center justify-between gap-2 px-4 py-2 rounded-md
        ${activeTab === 'stack'
          ? 'bg-blue-600/20 border-blue-500 text-blue-400'
          : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
      `}
    >

```

```

    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'bg-blue-600' : 'bg-slate-800'} rounded-sm` />
      <div className="text-left">
        <div className="font-bold text-sm text-white">
          <div className="text-xs opacity-80">LIFO
        </div>

```



```

        const isTop = index === 0;
        return (
          <div
            key={plate.id}
            className={`w-full flex items-center
              isTop
                ? 'bg-gradient-to-r from-blue
le-102'
                : 'bg-slate-900/90 border-sla
            `}
          >
            <div className="flex items-center"
              <span className="text-2xl">{pla
              <span className={`font-bold tex
                {plate.name}
              </span>
            </div>
            <span className={`text-xs font-mo
              isTop ? 'bg-blue-500 text-white
            `}>
              {isTop ? 'Готово к помывке' : `
            </span>
          </div>
        );
      }
    </div>
  )}
  <div className="mt-6 text-xs text-slate-500
    ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
  </div>
</div>
</div>
)}

{/* QUEUE TAB */}
{activeTab === 'queue' && (
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x

```

```

    <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
      <span className="inline-block w-2 h-2 rounded
        {dequeueMessage}
      </div>
    })
  }

  { /* Visualization */ }
  <div className="bg-slate-950/60 p-6 rounded-2xl
    overflow-x-auto">
    {queue.length === 0 ? (
      <div className="text-center py-12 text-slate-
        <div className="text-5xl mb-3"> </div>
        <p className="text-base font-bold">Очередь
        <p className="text-xs mt-1">Добавьте ро
      </div>
    ) : (
      <div className="w-full flex items-center justify-
        <div className="flex flex-col items-center justify-
300">
        <span className="text-3xl"> </span>
        <span className="text-xs font-bold font-mono">
      </div>

      <div className="text-indigo-500 flex items-center justify-
        <ArrowRight className="w-6 h-6 animate-pulse">
      </div>

      {queue.map((person, index) => {
        const isFirst = index === 0;
        return (
          <div
            key={person.id}
            className={`flex flex-col items-center justify-
              isFirst
                ? 'bg-gradient-to-b from-indigo-950/60 to-
relative'
                : 'bg-slate-900/90 border-slate-950/60'
          >

```

```

<div className="bg-slate-800/80 p-5 rounded-x
tify-between gap-4">
  <div>
    <h3 className="text-lg font-bold text-whi
      <span> Приоритетная очередь / Куча</sp
      <span className="text-xs font-mono bg-e
x-Heap</span>
    </h3>
    <p className="text-slate-400 text-xs mt-1
      Здесь не важно, кто пришел первым. Важн
    </p>
  </div>
  <div className="flex items-center gap-3 w-fr
    <button
      onClick={popHeap}
      className="flex-1 md:flex-none flex item
py-2.5 rounded-xl text-sm font-bold shadow-
    >
      Взять кандидата с макс. вероятностью
    </button>
    <button
      onClick={resetHeap}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
  <div className="md:col-span-6">
    <label className="block text-xs font-bold
    <input
      type="text"

```

```

        {heapMessage}
      </div>
    })

    {/* Visualization */}
    <div className="bg-slate-950/60 p-6 rounded-2
    >
      {sortedHeap.length === 0 ? (
        <div className="text-center py-12 text-slate-400">
          <div className="text-5xl mb-3"> </div>
          <p className="text-base font-bold">Куча
          <p className="text-xs mt-1">Добавьте ги
        </div>
      ) : (
        <div className="w-full max-w-2xl space-y-4">
          <div className="flex items-center justify-between">
            <span>КАНДИДАТ / ГИПОТЕЗА</span>
            <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
          </div>

          {sortedHeap.map(({item, index}) => {
            const isTop = index === 0;
            const percent = (item.probability * 100).toFixed(
            return (
              <div
                key={item.id}
                className={`flex items-center justify-between
                  ${isTop
                    ? 'bg-gradient-to-r from-emerald-500 to-emerald-600/10 scale-[1.01]'
                    : 'bg-slate-900/80 border-slate-800'}
                `}
              >
                <div className="flex items-center justify-between">
                  <span className={`flex items-center justify-between
                    ${isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'}
                    `}>
                      #{index + 1}
                    </span>
                    <span>#{item.id}</span>
                    <span>#{item.probability}</span>
                  </div>
                <div>#{item.message}</div>
              </div>
            )
          })}
        </div>
      )}
    </div>
  )}

```



```
    }}  
  </div>  
);  
};
```

ФАЙЛ 60/82: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useMemo, useState } from "react";  
import { BookOpen, Play, Search, X } from "lucide-react";  
import { VIZ_REGISTRY } from "../vizRegistry";  
import { chapters } from "../data/content";
```

```
interface Props {  
  /** Открыть тренажёр в модальном окне. */  
  onOpen: (vizId: string) => void;  
  /** Перейти к теме, к которой привязан тренажёр. */  
  onGoChapter: (chapterId: string) => void;  
}
```

```
/**  
 * Каталог всех интерактивных тренажёров.  
 *  
 * Раньше вкладка «Симулятор» показывала один случайный  
 * теперь это витрина: видно всё, что вообще можно потыкать.  
 */
```

```
export const SimulatorHub: React.FC<Props> = ({ onOpen,  
  const [query, setQuery] = useState("");
```

```
  const items = useMemo(() => {  
    const titleById = new Map(chapters.map((c) => [c.id, c.title]));  
    const all = Object.entries(VIZ_REGISTRY).map(([id, entry],  
      id,  
      entry,  
      chapterTitle: titleById.get(id),
```

```

    }}
  </div>

  {items.length === 0 && <p className="text-slate-500">
    Нет данных
  </p>}

  <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3" >
    {items.map(({ id, entry, chapterTitle }) => (
      <div
        key={id}
        className="flex flex-col bg-slate-900 border border-slate-800"
      >
        <h3 className="font-bold text-white text-sm">
          {entry.hint && <p className="text-[12px] text-white">
            {entry.hint}
          && <div className="flex-1" />
        }
        <div className="flex items-center gap-2 mt-2">
          <button
            onClick={() => onOpen(id)}
            className="flex items-center gap-1.5 text-white transition-colors"
          >
            <Play className="w-3.5 h-3.5" /> Закрыть
          </button>
          {chapterTitle && (
            <button
              onClick={() => onGoChapter(id)}
              title={chapterTitle}
              className="flex items-center gap-1.5 text-white transition-colors min-w-[150px]"
            >
              <BookOpen className="w-3.5 h-3.5 shrink-0" />
              <span className="truncate max-w-[9rem]">
                {chapterTitle}
              </span>
            </button>
          )}
        </div>
      </div>
    ))}
  </div>
</div>
</div>

```

```

    target: 1,
    build: () => {
      const n1 = leaf(1);
      const n2: TNode = { key: 2, left: n1, right: leaf(3) };
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Змейка",
    hintCase: "Zig-Zag",
    target: 3,
    build: () => {
      const n3 = leaf(3);
      const n2: TNode = { key: 2, left: leaf(1), right: leaf(4) };
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Один уровень",
    hintCase: "Zig",
    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(1), right: leaf(3) }, right: leaf(5) }),
  },
  {
    name: "Смешанный",
    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,
    build: () => {
      const n5 = leaf(5);
      const n6: TNode = { key: 6, left: n5, right: leaf(7) };
      const n4: TNode = { key: 4, left: leaf(3), right: leaf(5) };
      return { key: 8, left: n4, right: leaf(9) };
    },
  },
];

```

```

const path = findPath(root, key);
if (path.length < 2) return null;
const x = path[path.length - 1];
const p = path[path.length - 2];
const g = path.length >= 3 ? path[path.length - 3] : null;

if (!g) return { node: x.key, kind: "Zig" };

const xLeft = p.left === x;
const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
return { node: x.key, kind: "Zig-Zag (сначала нижняя ветвь)" };
}

/* ----- раскладка ----- */

interface Placed {
  key: number;
  x: number;
  y: number;
  depth: number;
}

function layout(root: TNode | null): { nodes: Placed[]; edges: [number, number][] } {
  const nodes: Placed[] = [];
  const edges: [number, number][] = [];
  let order = 0;

  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
    if (n.left) edges.push([n.key, n.left.key]);
    if (n.right) edges.push([n.key, n.right.key]);
    walk(n.right, depth + 1);
  };
  walk(root, 0);
}

```

```

const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonicalPath(target)));

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с кем

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.length === p.length }]);
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
  if (history.length === 0) return;
  setTree(history[history.length - 1]);
  setHistory((h) => h.slice(0, -1));
  setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
  <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
    <div className="flex items-start justify-between p-4">
      <div>
        <h4 className="text-sm sm:text-base font-bold text-slate-400">
          <p className="text-[12px] text-slate-400 lead-4">
            Клик по узлу = один поворот с его родителем
            <span className="font-mono font-bold text-inherit">
              поворотов выбираешь сам, разбор покажем в к
            </span>
          </p>
        </div>
      </div>
      <button
        onClick={() => reset((presetIdx + 1) % PRESET.length)}
        className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-medium text-slate-400 hover:text-white"
      >
        </button>
    </div>
  </div>
);

```

```

        y2={nb.y}
        stroke={onPath ? "#6366f1" : "#475569"}
        strokeWidth={onPath ? 3 : 1.8}
        style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
      />
    );
  }
}

{nodes.map((n) => {
  const isTarget = n.key === target;
  const clickable = !solved && n.depth > 0;
  return (
    <g
      key={n.key}
      onClick={() => clickable && clickNode(n)}
      style={{
        transform: `translate(${n.x}px, ${n.y}px)`,
        transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
        cursor: clickable ? "pointer" : "default",
      }}
    >
      {isTarget && (
        <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
      )}
      <circle
        r={16}
        fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475569"}
        stroke={isTarget ? "#a5b4fc" : "#475569"}
        strokeWidth={2}
      />
      <text textAnchor="middle" dominantBaseline="middle">
        {n.key}
      </text>
    </g>
  );
})}
</svg>
</div>

```

```

        </div>
    })

    {solved && (
      <div
        className={`mt-3 rounded-lg border px-3 py-2.5`}
        wrong === 0
          ? "bg-emerald-950/40 border-emerald-700/60"
          : "bg-rose-950/30 border-rose-800/60 text-rose-50"
      >
        <div className="flex items-center gap-2 font-mono text-sm">
          {wrong === 0 ? <CheckCircle2 className="w-4 h-4 text-emerald-500"/> : <X className="w-4 h-4 text-rose-500"/>}
          {wrong === 0
            ? `Узел ${target} в корне за ${moves.length} ходов`
            : `Узел ${target} в корне за ${moves.length} ходов`
          }
        </div>
        {wrong > 0 && (
          <ul className="list-disc pl-5 space-y-0.5">
            {moves.map(
              (m, i) =>
                !m.correct && (
                  <li key={i}>
                    ход {i + 1}: крутили <span className="font-mono text-sm">{m.expected}</span>
                  </li>
                )
            )}
          </ul>
        )}
      </div>
    )}
    {wrong === 0 && moves.length > minimal && (
      <div className="opacity-80">Порядок верный,
    </div>
    )}
  </div>
);
};

```

```

    ms: number;
}

interface Case {
    key: "zig" | "zig-zig" | "zig-zag";
    label: string;
    when: string;
    rotations: string;
    /** Номера узлов: роль → число. */
    keys: Partial<Record<NodeId, number>>;
    /** Обход слева направо – одинаковый на всех кадрах. */
    inorder: string[];
    /** Что означают поддеревья A..D. */
    ranges: string;
    frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
    pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
    edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
    key: "zig",
    label: "Zig",
    when: "родитель x – уже корень, деда нет",
    rotations: "1 поворот",
    keys: { x: 20, p: 40 },
    inorder: ["A", "20", "B", "40", "C"],
    ranges: "A < 20 · B между 20 и 40 · C > 40",
    frames: [
        {

```



```
const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
  rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
      text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
      ms: RESULT_MS,
    },
    {
      ...ZZ_S0,
      kind: "aim",
      title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
      text: "Главная хитрость этого случая: первым крутит 60, а не 40",
      focus: ["g", "p"],
      ms: AIM_MS,
    },
    {
      ...ZZ_S1,
      kind: "result",
      title: "Поворот 1: 40 поднялось над 60",
      text: "40 встало в корень, 60 опустилось к нему по  
убине 1, а не 2.",
      moved: ["p", "g", "C"],
      ms: RESULT_MS,
    },
    {
      ...ZZ_S1,
      kind: "aim",
      title: "Прицел 2: ребро 40 – 20",

```

```

    ...ZG_S0,
    kind: "start",
    title: "Было: 60 → влево на 20 → вправо на 40",
    text: "Узлы идут «змейкой»: сначала шаг влево, по
        ",
    ms: RESULT_MS,
},
{
    ...ZG_S0,
    kind: "aim",
    title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
    text: "Дед (60) стоит на месте и в этом повороте
    focus: ["p", "x"],
    ms: AIM_MS,
},
{
    ...ZG_S1,
    kind: "result",
    title: "Поворот 1: змейка выпрямилась",
    text: "40 заняло место 20 и забрало его к себе ле
    moved: ["x", "p", "C"],
    ms: RESULT_MS,
},
{
    ...ZG_S1,
    kind: "aim",
    title: "Прицел 2: ребро 60 – 40",
    text: "Остался одиночный поворот вокруг деда – и
    focus: ["g", "x"],
    ms: AIM_MS,
},
{
    pos: { x: [180, 50], p: [90, 130], g: [275, 130],
    edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
    kind: "result",
    title: "Стало: 20 и 60 – два сына сорока",
    text: "Дерево стало идеально симметричным: все че
        очку». ",

```

```

    const from = prev[id]!;
    const to = pos[id]!;
    const r = isSubtree(id) ? 15 : 20;
    const dx = to[0] - from[0];
    const dy = to[1] - from[1];
    const len = Math.hypot(dx, dy) || 1;
    const pad = r + 8;
    return (
      <g key={`ghost-${id}`} opacity={0.55}>
        <circle cx={from[0]} cy={from[1]} r={r} fill="white">
          {len > pad * 2 + 6 && (
            <line
              x1={from[0] + (dx / len) * pad}
              y1={from[1] + (dy / len) * pad}
              x2={to[0] - (dx / len) * pad}
              y2={to[1] - (dy / len) * pad}
              stroke="#a5b4fc"
              strokeWidth={1.5}
              strokeDasharray="4 3"
              markerEnd="url(#splay-arrow)"
            />
          )}
        </g>
      );
    )}
  )}

  {edges.map(([a, b], i) => {
    const pa = pos[a];
    const pb = pos[b];
    if (!pa || !pb) return null;
    const hot = Boolean(focus && focus.includes(a));
    return (
      <line
        key={`-${a}-${b}-${i}`}
        x1={pa[0]}
        y1={pa[1]}
        x2={pb[0]}
        y2={pb[1]}
      />
    );
  })}

```

```

        </text>
      })
    </g>
  );
  }}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

  const active = CASES[caseIdx];
  const total = active.frames.length;
  const idx = Math.min(frame, total - 1);
  const current = active.frames[idx];
  const prevPos = active.frames[Math.max(idx - 1, 0)].p;
  const duration = useMemo(() => Math.round(current.ms - prevPos), [current]);
  const last = idx === total - 1;

  useEffect(() => {
    setFrame(0);
  }, [caseIdx]);

  useEffect(() => {
    if (!playing) return;
    const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
    return () => clearTimeout(t);
  }, [playing, frame, duration, total]);

  return (
    <div className="w-full bg-slate-950 rounded-2xl border border-slate-800 p-4">
      <div className="flex gap-2 mb-4 overflow-x-auto">
        {CASES.map((c, i) => (
          <button

```

```

<Tree frame={current} prev={prevPos} keys={activeKeys}
  {/* обход слева направо – одинаковый на всех кадрах */}
  <div className="mt-2 pt-2 border-t border-slate-500">
    <div className="flex items-center gap-1.5 flex-wrap">
      <span className="text-slate-500 mr-1">Обход
        {active.inorder.map((t, i) => (
          <span
            key={i}
            className={`px-1.5 py-0.5 rounded font-mono text-sm
              /\d/.test(t) ? "bg-emerald-500/15 text-emerald-500" : ""
            }`}
            >
              {t}
            </span>
          <span key={i} className="text-slate-500 mr-1">
            {i}
          </span>
        ))}
      <span className="text-emerald-400/80 ml-1">
        {current}
      </span>
    </div>
    <p className="text-[11px] text-slate-500 mt-1">
      {current}
    </p>
  </div>

  {/* индикатор автопрокрутки */}
  <div className="h-1 w-full bg-slate-800 rounded-f"
    <div
      key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
      className="h-full bg-indigo-500"
      style={
        playing
          ? { animation: `demoProgress ${duration}ms linear 0s 1` }
          : { width: "100%", background: "#334155" }
      }
    </div>
  </div>

  <div className="flex flex-wrap items-center gap-2">
    <button
      onClick={() => {

```

```

        <button
          onClick={() => {
            setFrame(0);
            setPlaying(false);
          }}
          className="flex items-center gap-1.5 px-3 py-1
            ext-xs font-bold transition-colors"
        >
          <RotateCcw className="w-3.5 h-3.5" /> Сброс
        </button>

        <div className="flex items-center gap-3 ml-auto"
          {[["x", "p", "g"] as const].map((id) =>
            active.keys[id] === undefined ? null : (
              <span key={id} className="flex items-center"
                <span className="w-3 h-3 rounded-full"
                  </span>
                </span>
              )
            )}
        </div>
      </div>

      <p className="mt-2 text-[11px] text-slate-500">
        Листай кнопкой «Дальше» в своём темпе – кадры не
      </p>
    </div>
  );
};

```

ФАЙЛ 63/82: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from "react";
import {
  RotateCcw,

```

```

    return y;
  };

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};

export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };

  const [tree, setTree] = useState<SplayNode>(createInitialTree());
  const [inputValue, setInputValue] = useState<string>('');
  const [log, setLog] = useState<string[]>([
    'Дерево инициализировано. Попробуйте кликнуть на узел.',
  ]);
  const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

  // Splay operation that tracks steps!
  const splayStepByStep = {
    root: SplayNode | null,
    key: number,
  };

```

```
// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,
): SplayNode | null => {
  if (!node || node.key === k) return node;

  if (k < node.key) {
    if (!node.left) return node;

    if (k < node.left.key) {
      // Zig-Zig (Left-Left)
      node.left.left = splayAction(node.left.left,
        node = rightRotate(node));
      steps.push(
        `Вращение Zig-Zig (Правый поворот родителя)`
      );
    } else if (k > node.left.key) {
      // Zig-Zag (Left-Right)
      node.left.right = splayAction(node.left.right,
        if (node.left.right) {
          node.left = leftRotate(node.left);
          steps.push(`Компонент Zig-Zag: левый поворот`);
        }
      );
    }

    if (!node.left) return node;
    steps.push(
      `Финальный поворот Zig (Правое вращение корня)`
    );
    return rightRotate(node);
  } else {
    if (!node.right) return node;

    if (k < node.right.key) {
      // Zag-Zig (Right-Left)
      node.right.left = splayAction(node.right.left,
```



```

    if (isNaN(val)) return;

    // Standard BST insert, then splay!
    let newTree = cloneTree(tree);
    newTree = insertBST(newTree, val);
    const { newRoot, steps } = splayStepByStep(newTree,
    if (newRoot) {
        setTree(newRoot);
    }
    setLog([`Вставили вершину [${val}].`, ...steps]);
    setInputValue("");
    setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;
    handleSplay(val);
    setInputValue("");
};

const resetTree = () => {
    setTree(createInitialTree());
    setHighlightedNode(null);
    setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = {
    node: SplayNode | null,
    x: number,
    y: number,
    levelWidth: number,
}: void => {
    if (!node) return;
    node.x = x;
    node.y = y;

```

```

        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.right));
  }
  return lines;
};

const renderNodes = (node: SplayNode | null): React.R
  if (!node) return [];
  let circles: React.ReactNode[] = [];
  const isHighlighted = highlightedNode === node.key;

  circles.push(
    <g
      key={`n-${node.key}`}
      className="cursor-pointer group"
      onClick={() => handleSplay(node.key)}
      data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
    >
      <circle
        cx={node.x}
        cy={node.y}
        r="18"
        fill={isHighlighted ? "#4f46e5" : "#1e293b"}
        stroke={isHighlighted ? "#818cf8" : "#64748b"}
        strokeWidth={isHighlighted ? "3" : "2"}
        className="transition-all duration-300 group-hover:stroke-4"
      />
      <text
        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
        fontSize="12px"
        fontWeight="bold"
        className="select-none pointer-events-none"
      />
    </g>
  );

```

```

    <input
      type="number"
      placeholder="Поиск..."
      value={inputValue}
      onChange={(e) => setInputValue(e.target
00 className="w-24 bg-slate-950 text-white
focus:outline-none"
    />
    <button
      type="submit"
      onClick={handleFind}
      className="bg-indigo-600 hover:bg-indigo
    >
      Поиск / Splay
    </button>
    <button
      type="button"
      onClick={(e) => {
        e.preventDefault();
        const val = parseInt(inputValue);
        if (!isNaN(val)) {
          let newTree = cloneTree(tree);
          newTree = insertBST(newTree, val);
          const { newRoot, steps } = splaySte
          if (newRoot) setTree(newRoot);
          setLog([`Вставили [${val}].`, ...st
          setInputValue("");
          setHighlightedNode(val);
        }
      }}
      className="bg-indigo-950 hover:bg-slate
0"
    >
      Вставить
    </button>
  </form>

  <button

```

```

        <p>
            <span className="font-semibold text-slate-300">
                колено. Вращаем узел в разные стороны.
            </span>
        </p>
    </div>
</div>
</div>

{/* Answers Section inside the visualizer widget */}
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
    <h4 className="text-indigo-400 font-bold text-slate-300">
        <HelpCircle className="w-4 h-4 text-indigo-400" />
        Разбор экзаменационных вопросов (Без нитья):
    </h4>

    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        <div className="bg-slate-900/60 p-4 rounded-lg">
            <div>
                <p className="font-bold text-slate-300 mb-2">
                    1. Отсутствующий элемент
                </p>
                <p className="text-slate-400 leading-relaxed">
                    Допустим, мы ищем{" "}
                    <code className="text-indigo-400 font-sans">{<code>
                        его нет. Алгоритм спустится к листу, на котором
                        неудачей {например, {" "}}
                    </code>
                    <code className="text-indigo-400">[20]<code>
                    <code className="text-indigo-400">[30]<code>
                    поднимет в корень{" "}
                    <strong className="text-white">
                        последний успешно просмотренный узел
                    </strong>
                    , на котором он споткнулся! Это оставляем
                    сверху.
                </p>
            </div>
            <div className="mt-3 text-[10px] text-indigo-400">
                Поиск настраивает локальный кэш под реалии

```

```
        <strong className="text-white">O(log n)</strong>
      </p>
    </div>
    <div className="mt-3 text-[10px] text-emerald-500">
      Каждое "растяжение" компенсируется "сжатием"
    </div>
  </div>
</div>
</div>
</div>
);
};
```

ФАЙЛ 64/82: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
```

```
interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pink-500 to-pink-700' },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-orange-500 to-orange-700' },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-purple-500 to-purple-700' },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-yellow-500 to-yellow-700' },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-teal-500 to-teal-700' },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-gray-500 to-gray-700' },
];
```

```

        isDirty: true,
        animState: 'in',
    };

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id === newP
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${to

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id === topPla
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and
  const cleanPlate: Plate = {
    ...topPlate,
    isDirty: false,
    animState: 'clean',
  };
};

```

```
    </p>
  </div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap" >
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white
      font-medium rounded-xl px-5 py-3 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-700 opacity-40 disabled:cursor-not-allowed text-white
      font-medium rounded-xl px-5 py-3 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
      cursor-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12">
```

```

        className="absolute right-8 flex items-
        style={{ bottom: `${24 + topIndex * 44}`
    >
    <span className="text-xs font-mono text-
er gap-1">
        <ArrowUp className="w-3.5 h-3.5" />
        <span>ВЕРШИНА (TOP)</span>
    </span>
    <span className="text-blue-400 animate-p
</div>

    /* Тарелки (отрисовываем в обратном поряд
    {dirtyStack.slice().reverse().map((plate,
    const idx = dirtyStack.length - 1 - rev
    const isTop = idx === topIndex;
    return (
        <div
            key={plate.id}
            className={`
                w-full max-w-[280px] h-10 rounded
shadow-md select-none transition-all duration
                ${plate.color}
                ${plate.animState === 'in' ? 'anim
                ${plate.animState === 'washing' ?
                ${isTop ? 'ring-4 ring-blue-500/50
            `}
        >
            <span className="text-lg">{plate.em
            <span className="text-slate-800 tex
            {isTop && (
                <span className="text-[9px] bg-bl
                    LIFO
                </span>
            )}
        </div>
    );
    }}}
</div>

```



```

    }}
  </div>

  { /* Столешница сушилки */ }
  <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500" >
  </div>
</div>

{ /* ЛОГ ДЕЙСТВИЙ */ }
<div className="bg-slate-900 border border-slate-700 p-4" >
  <div className="text-[10px] font-mono text-slate-200" >
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between p-2`} >
        {idx === 0 ? <span className="text-blue-500 font-weight-bold">Действие</span> : <span></span>
        <span>{entry}</span>
      </div>
    ))}
  </div>
</div>
);
};

```

ФАЙЛ 65/82: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;

```

```

const s = pattern + "#" + text;
const n = s.length;
const z = new Array(n).fill(0);
const steps: any[] = [];

let l = 0, r = 0;
steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

for (let i = 1; i < n; i++) {
  steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

  if (i <= r) {
    steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
    z[i] = Math.min(r - i + 1, z[i - l]);
    steps.push({ i, l, r, z: [...z], desc: `Предел не падает. Копипаста сработала.` });
  }

  steps.push({ i, l, r, zTarget: z[i], zSource: i });
  while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
    steps.push({ i, l, r, zTarget: z[i], zSource: i });
    падают! Окно растёт.` });
    z[i]++;
  }

  if (i + z[i] < n) {
    steps.push({ i, l, r, zTarget: z[i], zSource: i });
    [i]]}' разные. Стоп.` });
  } else {
    steps.push({ i, l, r, z: [...z], desc: `Упёрся.` });
  }

  steps.push({ i, l, r, z: [...z], desc: `Фиксируем.` });

  if (i + z[i] - 1 > r) {
    l = i;
    r = i + z[i] - 1;
  }
}

```

```

        setStepIdx(prev => {
          if (prev >= steps.length - 1) { set
            return prev + 1;
          });
      }, 1200);
    }
    return () => { if (timer.current) clearInterval
  }, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
      </div>

```

```

    }
  }

  return (
    <div key={index} className=
      {/* L/R markers for Z */
        {mode === "z" && step.l
          <div className="abs
        }}
        {mode === "z" && step.r
          <div className="abs
        }}

      {/* Index */}
      <span className={`text-

        {/* Char Box */}
        <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
          {char}
        </div>

        {/* Value array box */}
        <div className="text-[1
slate-700/50 font-mono font-bold">
          {mode === "kmp" ? s
        </div>

        {/* KMP fingers */}
        {mode === "kmp" && step
          <div className="abs
            z-20">
              <span classNam
            </div>
          }}
        {mode === "kmp" && step
          <div className="abs
            <span classNam

```

```

        }}}
      </div>
    </div>

    <div className="flex flex-col sm:flex-row gap-4" >
      { /* Controls */ }
      <div className="flex bg-slate-900 border border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-50" >
        <button onClick={prevStep} disabled={!steps.length} >
          <Undo strokeWidth={3} size={16} />
        </button>
        <button onClick={playPause} className="flex-grow-1 items-center justify-center min-w-[80px]" >
          {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
        </button>
        <button onClick={nextStep} disabled={steps.length === 1} >
          <SkipForward strokeWidth={3} size={16} />
        </button>
        <button onClick={reset} className="border border-slate-800" >
          <RefreshCw strokeWidth={3} size={16} />
        </button>
      </div>

      { /* Description Log */ }
      <div className="flex-1 bg-slate-900/50 border border-slate-800 overflow-hidden" >
        <div className="absolute top-0 right-0 bottom-0 left-0" >
          <span className="font-mono text-sm text-slate-400" >
            {steps.map((step) => {
              <div className="text-[10px] text-slate-400" >
                {step.desc}
              </div>
            })}
          </span>
        </div>
      </div>
    </div>
  </div>

```

```
        </div>
    );
}
```

ФАЙЛ 66/82: src/components/TopologicalSortViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];
```

```

    });
};

recordStep(
  "Начало топологической сортировки. Используем кла
  null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
  visited.add(u);
  dfsStack.push(u);
  recordStep(
    `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
    u,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs(to);
    } else if (!fullyProcessed.has(to)) {
      // Explaining potential cycle detected (not in topological order)
      recordStep(
        `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
        u,
      );
    }
  }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in reverse order
recordStep(
  `DFS: закончили обработку '${dagNodes[u].name}'`,
  u,
);
};

```

```

    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучени
      </h3>

      <div className="flex items-center gap-2 bg-slate-
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
        </button>
      </div>
    </div>
  );
);

```



```

        <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
    </marker>
</defs>

```

```

{/* Edges */}
{dagEdges.map((edge, idx) => {
    const u = dagNodes.find((n) => n.id === e
    const v = dagNodes.find((n) => n.id === e

    const isActive =
        (step.currentNode === edge.u || step.cu
        step.visited.has(edge.v);

    return (
        <line
            key={`edge-${idx}`}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isActive ? "#818cf8" : "#334155"}
            strokeWidth={isActive ? "3" : "2"}
            markerEnd={`url(#${isActive ? "dag-ar
            className="transition-all duration-300
        />
    );
}})

```

```

{/* Nodes */}
{dagNodes.map((node) => {
    const isCurrent = step.currentNode === no
    const isVisited = step.visited.has(node.id
    const isProcessed = step.fullyProcessed.h

    let fill = "#1e293b";
    let stroke = "#334155";
    let textColor = "text-slate-400";
    void textColor;

```

```

        </text>
        <text
          x={node.x}
          y={node.y + 12}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="9px"
          className="pointer-events-none"
        >
          {node.name.split(" ").slice(1).join(" ")}
        </text>
      </g>
    );
  }
}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 rounded-00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

```

```

<div className="bg-slate-900 border border-slate-800">
  <p className="text-xs text-indigo-300 min-h-[100px]">
    {step.desc}
  </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
  {/* Topological List output */}
  <div>
    <span className="text-[10px] text-slate-500">
      РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):
    </span>
    <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">

```

```

        className="bg-slate-900 border border-slate-800 text-white"
      >
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length}
        onClick={() => setCurrentStepIdx((prev) => prev - 1)}
        className="bg-slate-900 border border-slate-800 text-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 67/82: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useEffect, useMemo, useState } from "react";

/**
 * Интерактивное построение декартова дерева в координатах
 *
 * Идея страницы: каждая пара (x, y) – точка, прибитая к доске.
 * (x = ключ, y = приоритет). Точки НЕ двигаются никогда.
 * значит правильно соединить их линиями: ребро всегда идет от
 * меньшему (куча), а «левее/правее» решает x (BST). По сути,
 * прямо в своих координатах – никакой отдельной «деревяшки».
 */

```

```

    { key: 9, grade: 3 },
    { key: 7, grade: 2 },
    { key: 1, grade: -4 },
];

const CODE_LINES = [
  { line: 1, text: "root = None" },
  { line: 2, text: "for key, pri in pairs_sorted_by_pri" },
  { line: 3, text: "    cur, parent = root, None" },
  { line: 4, text: "    while cur:" },
  { line: 5, text: "        parent = cur" },
  { line: 6, text: "        cur = cur.left if key <= cu" },
  { line: 7, text: "        <пустое место под parent> = Nod" },
  { line: 8, text: "# pri приходят по убыванию – куча н" },
];

/** Перевод координат точки в пиксели SVG. Клетка: 1 ед
const VIEW_W = 820;
const VIEW_H = 470;
const X = (key: number) => 50 + (key - 1) * ((VIEW_W -
const Y = (grade: number) => 34 + (8 - grade) * ((VIEW_

function collect(n: TNode | null): { points: Placed[];
  const points: Placed[] = [];
  const edges: { from: number; to: number }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    walk(n.left);
    points.push({ key: n.key, grade: n.grade, px: X(n.k
    if (n.parentKey !== null) edges.push({ from: n.pare
    walk(n.right);
  };
  walk(n);
  return { points, edges };
}

function buildSteps(): Step[] {
  const steps: Step[] = [];

```

```

while (cur) {
  const atNode = cur;
  const dir: "L" | "R" = hire.key <= atNode.key ? "L" : "R";
  pathKeys.push(atNode.key);
  steps.push({
    ...snap(
      `⌘ Точка ${label} против (${atNode.key}, ${atNode.key}
        ? "левее" : "правее").`,
      6,
      { candidate: hire, pathKeys: [...pathKeys] },
    ),
  });
  parent = atNode;
  side = dir;
  cur = dir === "L" ? atNode.left : atNode.right;
}

const node: TNode = {
  key: hire.key,
  grade: hire.grade,
  left: null,
  right: null,
  parentKey: parent ? parent.key : null,
};
if (!root) root = node;
else if (side === "L") parent!.left = node;
else parent!.right = node;
doneKeys.push(hire.key);

steps.push({
  ...snap(
    parent
      ? `Проводим линию (${parent.key}, ${parent.key}
        по y – куча цела.`
      : `Точка ${label} первая – пока ни с кем не`,
    7,
    { candidate: hire, pathKeys: [...pathKeys], jus

```

```

    ? posByKey.get(`${step.candidate.key}:${step.cand
    : undefined;

```

```

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    {/* Шанка */}
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white
          <span className="text-2xl"> </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white
          <p className="text-sm text-emerald-300">
            Точки постоянны: x = ключ, y = приоритет.
          </p>
        </div>
      </div>
    </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => {
          setPlaying(false);
          setIdx(0);
        }}
        className="flex items-center gap-1 bg-slate
          ion-colors"
      >
        Сброс
      </button>
      <button
        onClick={() => setPlaying(!playing)}
        className={`flex items-center gap-1 px-4 py
          playing ? "bg-amber-600 hover:bg-amber-500
          }`}
      >
        {playing ? " Пауза" : "▶ Пуск"}
      </button>
      <button

```

```

    { /* Клетчатая бумага */
    <rect x="40" y="24" width={VIEW_W - 60} height={VIEW_H - 24} fill="#f0f0f0" />

    { /* Оси и шкалы */
    <line x1="40" y1={Y(0)} x2={VIEW_W - 20} y2={Y(0)} />
    <line x1={X(1) - 10} y1="24" x2={X(1) - 10} y2={Y(0)} />
    {[0, 5, 10, 15, 20, 25, 30, 35, 40, 45].map((k) => {
      <g key={`x1-${k}`} >
        <line x1={X(k)} y1={Y(0) - 4} x2={X(k)} y2={Y(0)} />
        <text x={X(k)} y={Y(0) + 18} fill="#64719d" />
        {k}
      </text>
    </g>
    )})
    { [-4, -2, 0, 2, 4, 6, 8].map((g) => {
      <g key={`y1-${g}`} >
        <line x1={X(1) - 14} y1={Y(g)} x2={X(1) - 10} y2={Y(g)} />
        <text x={X(1) - 20} y={Y(g) + 4} fill="#64719d" />
        {g}
      </text>
    </g>
    )})
    <text x={VIEW_W - 24} y={Y(0) - 10} fill="#64719d" />
    x (ключ) →
  </text>
  <text x={X(1) - 34} y="20" fill="#94a3b8" />
  y (приоритет)
</text>

  { /* Линии (рёбра) */
  {step.edges.map((e) => {
    const a = step.points.find((p) => p.key === e.to)
    const b = posByKey.get(`${e.to}:${step.points.find((p) => p.key === e.to).key}`)
    if (!a || !b) return null
    const onPath = step.pathKeys.includes(e.from)
    const isFresh = freshEdge && freshEdge.to === e.to
    return (

```

```

        {/* Кандидат: бледное пунктирное кольцо на
        {step.candidate && !candidatePlaced && {
            <g className="transition-all duration-500"
                <circle
                    cx={X(step.candidate.key)}
                    cy={Y(step.candidate.grade)}
                    r={13}
                    fill="none"
                    stroke="#818cf8"
                    strokeWidth={2.5}
                    strokeDasharray="4,3"
                />
                <text
                    x={X(step.candidate.key)}
                    y={Y(step.candidate.grade) + 4}
                    fill="#c7d2fe"
                    fontSize="11"
                    fontWeight="bold"
                    textAnchor="middle"
                    fontFamily="monospace"
                >
                    {step.candidate.key}
                </text>
            </g>
        }}
    </svg>

    <div className="w-full bg-slate-950/80 p-4 rounded"
        <p className="font-semibold text-amber-400"
        <p className="min-h-[40px] flex items-center"
    </div>
</div>

{/* Порядок соединения */}
<div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <p className="text-xs text-slate-400 mb-3">To
    <div className="space-y-2.5 flex-1 overflow-y

```



```

    {HIRES.map((h) => {
      const placed = step.points.find((p) => p.key === h.key)
      const isCur = step.justPlacedKey === h.key
      const parent = placed && placed.parentKey === h.key
      return (
        <tr key={`c-${h.key}-${h.grade}`} className={`font-semibold ${
          placed ? "bg-slate-900/20" : ""
        }`} >
          <td className="py-2.5 px-3 text-white" >
            <span className={`inline-block w-100px h-1.5em`} style={{
              background: `linear-gradient(to right, transparent 49%, #ccc 49%, #ccc 51%, transparent 51%)`,
              backgroundSize: "10px 10px",
            }} >
              {h.grade}
            </span>
          </td>
          <td className="py-2.5 px-3 text-slate-600" >
            {placed && parent ? (placed.px < parent.px ? "←" : "→") : ""}
          </td>
        </tr>
      )
    })
  </tbody>
</table>
</div>

{/* Код */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-4" >
  <h4 className="text-lg font-bold text-slate-200" >Код
  <div className="bg-slate-950/80 rounded-lg p-4" >
    {CODE_LINES.map((item) => {
      const isActive = step.codeLine === item.line
      return (
        <div
          key={item.line}
          className={`py-1.5 px-3 rounded flex items-center justify-between`}
          style={isActive ? "bg-indigo-900/80 text-white" : "text-slate-400"} >
          {item.line}
          <span>
            {step.codeLine === item.line ? "Активная строка" : ""}
          </span>
        </div>
      )
    })}
  </div>
  <div>
    <span className="text-slate-600 select-none" >Строка
    <span className="whitespace-pre flex-grow-1" >
      {CODE_LINES.map((item) => {
        const isActive = step.codeLine === item.line
        return (
          <div
            key={item.line}
            className={`py-1.5 px-3 rounded flex items-center justify-between`}
            style={isActive ? "bg-indigo-900/80 text-white" : "text-slate-400"} >
            {item.line}
            <span>
              {step.codeLine === item.line ? "Активная строка" : ""}
            </span>
          </div>
        )
      })}
    </span>
  </div>

```

```
-----
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";
```

```
/** Разбор поворота и песочница всегда идут парой: посмотри, как это выглядит. */
```

```
* Анимация и песочница идут друг под другом, а не в две колонки.
* колонке дерево сжималось до нечитаемого размера, а песочница была для
* компактности. Между ними – подпись, объясняющая перемену ролей.
*/
```

```
const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево, но
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);
```

```
export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  description: string;
}
```

```

    Component: SparseTableViz,
  },
  treap: {
    title: "Строим Treap по точкам: x — ключ, y — приоритет",
    hint: "Точки {x = ключ, y = приоритет} прибиты к кл...
      код синхронизированы.",
    Component: TreapBuildViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число — увидите, из чего оно сло...",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay-дерево",
    hint: "Покадровый разбор трёх поворотов, песочница",
    Component: () => (
      <div className="space-y-10">
        <SplayRotationsPair />
        <SplayTreeViz />
      </div>
    ),
  },
  "splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Сверху — покадровый разбор поворота, снизу —",
    Component: () => <SplayRotationsPair />,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: ... },
  "top-sort": { title: "Топологическая сортировка", Component: ... },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: ... },
  "graph-articulation": { title: "Точки сочленения", Component: ... },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: ... },

```

```
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
  intro: { title: "Мнемокарточки", Component: MnemonicCards },
  "everyday-basics": {
    title: "Бытовой тренажёр: стек, очередь, куча",
    hint: "Тарелки, очередь в столовой и мешок гипотез",
    Component: Simulator,
  },
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 69/82: `src/components/WaterfallAnimationWidget.tsx`
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для визуализации
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
  term link: number;
```

```
// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»');
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/ /g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};

const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
    setActiveSearchCodeLine(4);
  }
};
```

```

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) => {
            currentMatches.push({ index: currentIndex, word: word });
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
    logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
    setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMatches]);
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Остальные вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
    },

```

```

desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
      Н" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'I',
checkingBranchesFrom: 0,
},
{
  targetNodeId: 8,
  title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
      ). Рыба плывет в ROOT и ищет ветку "H". У корня
      суффикс "H"!',
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'H',
checkingBranchesFrom: 0,
},
{
  targetNodeId: 3,
  title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
      е подходят. fail[HER] = ROOT.',
codeLine: 3,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'R',
checkingBranchesFrom: 0,
},
{
  targetNodeId: 6,
  title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель :
      но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
codeLine: 4,
scoutPos: 0,

```

```

    <div>
      <h4 className="text-2xl font-bold text-white">
        Анимация водопада: Полное обучение автома
      </h4>
      <p className="text-sm text-slate-400">
        Визуализация пошагового построения всех f
      </p>
    </div>
  </div>

  {/* Переключатель режимов */}
  <div className="bg-slate-900 p-1.5 rounded-xl b
    <button
      onClick={() => setActiveMode('training')}
      className={`flex items-center space-x-1.5 p-
        activeMode === 'training'
          ? 'bg-indigo-600 text-white shadow-lg sl
          : 'text-slate-400 hover:text-white'
        }`
    >
      <GitBranch className="w-4 h-4" />
      <span>Режим 1: Полное обучение (BFS от корн
    </button>
    <button
      onClick={() => setActiveMode('search')}
      className={`flex items-center space-x-1.5 p-
        activeMode === 'search'
          ? 'bg-blue-600 text-white shadow-lg sha
          : 'text-slate-400 hover:text-white'
        }`
    >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>
</div>

  {/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}

```



```

        >
        Назад
    </button>
    <button
        onClick={() => setTrainStep((prev) => M
        disabled={trainStep === TRAINING_STEPS_
        className="flex-1 bg-indigo-600 hover:b
        ition-all shadow-lg shadow-indigo-900/50"
    >
        {trainStep === TRAINING_STEPS_INFO.leng
    </button>
</div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
    >
    <div>
        <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
                <span></span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950
                Вершина: #{targetTrainingNode.id} ({t
            </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-500 text-indigo-300 font
                <span>1. let u = trie[r][char]; // Те
                {currentTrainInfo.codeLine === 1 && <
            тут</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-400 text-indigo-200 font
                <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail].label}}</span>
                {currentTrainInfo.codeLine === 2 && <

```

```

<div className="bg-slate-900/60 p-5 rounded-x
  <div>
    <h5 className="text-white font-bold mb-2
      <span>» </span> Текст для сканирования
    </h5>
    <p className="text-xs text-slate-400 mb-4
      Введи текст, чтобы лосось просканировал
    </p>
    <input
      type="text"
      value={textToScan}
      onChange={handleTextChange}
      maxLength={15}
      placeholder="ВВЕДИТЕ ТЕКСТ..."
      className="w-full bg-slate-800 border b
    cus:outline-none focus:border-blue-500"
    />
  </div>

  <div>
    <div className="mb-4 flex flex-wrap gap-1
      <span className="text-xs text-slate-400
        {textToScan.split('').map((char, idx) =>
          <span
            key={idx}
            className={`w-7 h-8 flex items-centr
              idx === currentIndex
                ? 'bg-blue-600 text-white shadow
                : idx < currentIndex
                ? 'bg-emerald-950/60 text-emera
                : 'bg-slate-800 text-slate-500'
            }`}
          >
            {char}
          </span>
        )}
      >
    </div>
    {currentIndex >= textToScan.length && (
      <span className="bg-emerald-600 text-

```

```

        </div>
        <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span <
ем по течению</span>}
        </div>
        <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span <
овпадение!</span>}
        </div>
    </div>
</div>

<div>
    <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounde
tems-center">
        {searchLog}
    </div>
</div>
</div>
</div>
))

```

```

/* SVG Анимация водопада */
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
    /* Водопадные фоновые эффекты */
    <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
    <div className="absolute top-0 left-1/2 -transl
        <div className="w-8 bg-blue-400 h-full animat
        <div className="w-12 bg-blue-300 h-full anima

```

```

        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="12"
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" font-size="12"
      {level.label}
    </text>
  </g>
)}}

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([childId, child]) => {
    const toNode = WATERFALL_NODES[toId];

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ?
    type === 'normal';

    // Подсветка в режиме обучения (когда ищем слово)
    let isTrainingExamined = activeMode === 'train' ?
    let isMatchBranch = isTrainingExamined && child.label === word;

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`}
        {/* Линия течения воды */}
        <line
          x1={fromNode.x}
          y1={fromNode.y}
          x2={toNode.x}
          y2={toNode.y}
          stroke={isHighlighted ? '#3b82f6' : '#ccc'}
          strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
          className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
        />
        {/* Символ перехода */}
        <circle cx={(fromNode.x + toNode.x) / 2} cy={

```

```

        let isHighlighted = activeMode === 'search' || activeMode === 'fail';
        let isJustEstablished = activeMode === 'transition' || activeMode === 'target-link' - cсылку

        // Вычисляем изогнутую кривую для красоты
        const dx = targetNode.x - node.x;
        const dy = targetNode.y - node.y;
        const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -1 : 1) * 30;
        const cy = node.y + dy / 2 - 30;

        return (
          <g key={`fail-${node.id}`}>
            <path
              d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
              fill="none"
              stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
              strokeWidth={isHighlighted ? 4 : 2}
              strokeDasharray="6,6"
              className={isHighlighted ? 'highlighted' : ''}
              opacity={isHighlighted ? 1 : 0.5}
            />
            {isJustEstablished && (
              <text x={cx} y={cy - 10} fill="#10b98f">
                fail[{node.label}] = {targetNode.label}
              </text>
            )}
          </g>
        );
      }
    }
  }
}

/** Рендер вершин (бассейнов водопада) */
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'transition' || activeMode === 'target-link' - cсылку;
  const hasWords = node.words.length > 0;

```

```

        ИЩЕМ FAIL
    </text>
</g>
}}

{ /* Индикатор словарных слов */
{hasWords && {
    <g transform={`translate(${node.x +
    <circle cx="0" cy="0" r="9" fill=
    <text x="0" y="3" fontSize="10" f
        *
    </text>
    </g>
    }}
</g>
});
}}
}}

{ /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
<g>
    { /* Круг-подложка под рыбу */
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    { /* Сама рыба */
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi

```

```
    }}  
  </div>  
);  
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 70/82: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ: 10.5 КБ

```
import React, { useMemo, useState } from "react";  
  
/**  
 * Префиксные суммы: 1D и 2D.  
 *  
 * Главная фишка – наведение на число:  
 * * в 1D наводим на P[i] – подсвечивается кусок массива  
 * * в 1D наводим на a[i] – видно, в какие префиксы он  
 * * в 2D наводим на ячейку – подсвечивается прямоугольник  
 * а при выбранном запросе показывается формула включения  
 */  
  
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];  
  
const A2 = [  
  [1, 2, 3, 4],  
  [5, 6, 7, 8],  
  [9, 1, 2, 3],  
  [4, 5, 6, 7],  
];  
  
const cellBase =  
  "flex items-center justify-center rounded-md font-monospace";
```

```
const [range, setRange] = useState<{ l: number; r: number}>({ l: 0, r: 0 });
```

```
const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;
```

```
const sum = pref[range.r + 1] - pref[range.l];
```

```
return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={` ${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>
      </div>
    </div>
  )
```

```
    {/* массив a */}
    <div className="flex gap-1.5 items-center mb-1">
      <div className="w-[46px] shrink-0 text-right">
        {A1.map((v, i) => {
          const inCover = covered && i >= covered.from;
          const inRange = i >= range.l && i <= range.r;
          return (
            <div
              key={i}
              onMouseEnter={() => setHover({ kind: "pref", i })}
              onMouseLeave={() => setHover(null)}
              onClick={() => setRange((r) => (i < r ? i : r))}
              className={` ${cellBase} cursor-pointer`}>
                {v}
              {inCover ? "a" : "pref"}
            </div>
          );
        })}
      </div>
    </div>
  )
```



```

        });
    }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-800"
  {hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0</b> — начало
        отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        — это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}
        </span>
        </p>
      )
    ) : hover?.kind === "a" ? (
      <p className="text-slate-300">
        <b className="text-indigo-400">
          a[{hover.i}] = {A1[hover.i]}
        </b>{" "}
        входит во все префиксы, начиная с{" "}
        <span className="font-mono text-emerald-300">
          a[0..{hover.i - 1}]
        </span>
        подвинуть границу запроса.
      </p>
    ) : (
      <p className="text-slate-500">
        Наведите курсор на любое число: у <b className="text-emerald-400">у</b>
        у <b className="text-indigo-400">a</b> — ку

```

```

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => setRect((r) => {
                      i < r.r1 || j < r.c1 ? { r1: i, c1: j, r2: i, c2: j } : r
                    })}
                    className={` ${cellBase} cursor-pointer ${
                      inRect ? "bg-indigo-500 text-white" : "bg-white text-slate-400"
                    }`}
                  >{v}</div>
                );
              })}
            </div>
          })}
        </div>
      </div>
    </div>
  </div>
);

```

```

        {corner && !isHover && (
            <span className="absolute -top-
ht">
                {corner}
            </span>
        )}
    </div>
    );
    }}}
  </div>
  )))
</div>
</div>
</div>

```

```

<div className="min-h-[54px] bg-slate-900 border
  {hover ? (
    <p className="text-slate-300">
      <b className="text-amber-400">
         $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$ 
      </b>{" "}
      – сумма всего прямоугольника от левого верх
    </p>
  ) : (
    <p className="text-slate-500">Наведите курсор
  )}
</div>

```

```

<div className="bg-slate-900 border border-indigo
  <p className="font-mono text-sm sm:text-base te
    сумма = <span className="text-emerald-400">A<
    <span className="text-rose-400">C</span> + <s
    <span className="text-indigo-300 font-bold">{
  </p>
  <p className="text-[11px] text-slate-500 mt-1">
    Вычли два «хвоста» и вернули дважды вычитенный
  </p>
</div>

```

```

    [14, 51, 99, 13, 22, 64, 43, 76],
    [89, 32, 54, 71, 15, 88, 29, 60],
    [25, 41, 16, 92, 9, 17, 56, 31],
    [59, 18, 77, 24, 61, 82, 35, 12],
    [73, 8, 38, 85, 47, 95, 19, 58],
    [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
    Array.from({length: 8}, () =>
        Array.from({length: 4}, () =>
            Array(4).fill(0)
        )
    )
);

for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
        ST2D[r][c][0][0] = val2D[r][c];
    }
}

for (let kx = 0; kx <= 3; kx++) {
    for (let ky = 0; ky <= 3; ky++) {
        if (kx === 0 && ky === 0) continue;
        for (let r = 0; r + (1 << kx) <= 8; r++) {
            for (let c = 0; c + (1 << ky) <= 8; c++) {
                if (kx > 0) {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx-1][ky],
                        ST2D[r + (1 << (kx-1))][c][kx-1][ky]
                    );
                } else {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx][ky-1],
                        ST2D[r][c + (1 << (ky-1))][kx][ky-1]
                    );
                }
            }
        }
    }
}

```

```

        <Viz2DBuild key="2d_build" />
      ) : (
        <Viz2DQuery key="2d_query" />
      )}
    </AnimatePresence>
  </div>
);
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))])$ 
          </p>
        </div>
        <div className="flex gap-2">
          <button
            onClick={() => setStep(Math.max(0, step - 1))}
          />
        </div>
      </div>
    </div>
  );
};

```

```

        });
    }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
        <p className="text-slate-300">
            <b className="text-sky-400">
                ST[{hover.i}][{hover.j}] = {stID[hover.i]}
            </b>{" "}
            – это минимум на отрезке{" "}
            <span className="font-mono text-sky-300">
                [{covered.from}, {covered.to}]
            </span>{" "}
            длины  $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$ 
            <span className="font-mono text-slate-400">
                min({arrID.slice(covered.from, covered.to)})
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое число
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-100">
        <div className="grid grid-cols-[auto_repeat(4, 1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-sm">
                 $2^0$  (L=1)
            </div>
            <div className="text-center font-bold text-sm">

```

```

        currentStep.j === j + 1 &&
        currentStep.i === i - (1 << (currentStep.j - j))
      )
      isSrc2 = true; // wait, no. Src2
      // Let's re-eval exact source:
      // We are asking if THIS cell [i][j] is the source
      if (currentStep.j - 1 === j) {
        if (currentStep.i === i) isSrc1 = true;
        if (currentStep.i + (1 << (currentStep.j - j)) === i)
          isSrc2 = true;
      }
    }
  }

  return (
    <div key={j} className="relative flex flex-col items-center justify-center"
      {!isValid ? (
        <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200"
      ) : (
        <motion.div
          onMouseEnter={() => setIsVisible(true)}
          onMouseLeave={() => setIsVisible(false)}
          onClick={() => setIsVisible(!isVisible)}
          initial={{ opacity: 0, scale: 0.8 }}
          animate={{
            opacity: isVisible ? 1 : 0,
            scale: isVisible ? 1 : 0.8,
          }}
          className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200
            hover && hover.i === i && hover.j === j
            ? "bg-sky-500 text-white"
            : isActive
            ? "bg-indigo-500 text-white"
            : isSrc1
            ? "bg-emerald-500 text-white"
            : isSrc2
            ? "bg-amber-500 text-white"
            : isVisible
          }
        />
      )}
    />
  )
)

```

```

    /* Formula helper text */
    <div className="bg-slate-900 border border-slate-
      {step > 0 && step <= maxStep ? (
        (() => {
          const c = computeSteps[step - 1];
          const half = 1 << (c.j - 1);
          return (
            <p className="text-lg text-slate-300">
              <span className="text-white font-bold">
                ST[{c.i}][{c.j}]
              </span>{" "}
              = min(
                <span className="text-emerald-400 font-b
                  ST[{c.i}][{c.j - 1}]
                </span>
                ,
                <span className="text-amber-400 font-bo
                  ST[{c.i + half}][{c.j - 1}]
                </span>
              ) &rarr; min(
                <span className="text-emerald-400">{st1
                <span className="text-amber-400">
                  {st1D[c.i + half][c.j - 1]}
                </span>
              ) ={" "}
                <span className="text-indigo-400 font-b
                  {st1D[c.i][c.j]}
                </span>
              </p>
            );
          })()
        ) : (
          <p className="text-slate-500 italic">
            Нажмите далее, чтобы увидеть, как блоки сво
          </p>
        )
      )
    </div>

```



```

{[...Array(k + 1)].map((_, i) => k - i).map((
  const stepL = 1 << step;
  const stepSize = 8 - stepL + 1;
  const isCurr = step === k;

  return (
    <React.Fragment key={step}>
      {idx > 0 && <div className="text-slate
        <div className={`flex flex-col items-c
          <h4 className={`font-bold mb-3 flex
    }>
      <span>Матрица блоков {stepL}x{step
        <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
      </h4>
      <div className="max-w-full overflow-
        <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
      {Array.from({length: stepSize * st
        const r = Math.floor(idx / step
        const c = idx % stepSize;

        let highlightClass = isCurr ? 'l
/50 cursor-pointer' : 'bg-slate-800/50 text
        let zIndex = 'z-0';

        const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

        if (isActive) {
          if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
          } else {
            const prevL = 1 << (k - 1);
            const isTop = r < activeCel

```



```

            tiveCell.r][activeCell.c + prevL][k-1][k-1]
            </div>
            <div className="flex item
                <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
            </div>
            <div className="flex item
                <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
                <span className="text
[activeCell.r + prevL][activeCell.c + prevL
            </div>
        </>
    )
    }}())}
</div>

    <div className="pt-3 mt-3 border-t
        ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
            Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    })
</div>
</div>
</div>
);

```

```

        </div>
    )
}}}

<div
  className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
  style={{
    top: `${(r1 / 8) * 100}%`,
    left: `${(c1 / 8) * 100}%`,
    marginTop: '8px',
    marginLeft: '8px',
    height: 'calc(' + (H/8*100) +
    width: 'calc(' + (W/8*100) +
  }}
/>

  {/* Показываем 4 угла в самой матрице */}
  <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
    <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`, l
    <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
    <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
  </div>

  <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
    <label className="flex items-cen
      <span className="text-slate-
        <div className="flex gap-2">
          <input type="number" min={

```

```

ext-amber-300">1</span>); <span className="
  <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
  <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

  <span className="text-slate-500">// 2.
  <span className="text-indigo-400">int<
  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-rose-400">
r1</span>][<span className="text-white bg-r

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
  &nbsp;&nbsp;&nbsp;<span className="text-blue-400">
r1</span>][<span className="text-white bg-b

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
рх, чтобы нижний край уперся в r2)</span><b
  &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">
nded">r2 - Lx + 1</span>][<span className="

  &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-amber-300">
">r2 - Lx + 1</span>][<span className="text
  {'}'}));
</div>

<div className="w-full flex justify-cent
  <div className="flex flex-col items-ce
  <h4 className="text-slate-400 font-bo
    Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
  </h4>
  <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>

```

```

        <span className="text-slate-400 font-
          <span className="text-rose-400">
            <span className="text-blue-400 m
n>
          <span className="text-emerald-400
span>
            <span className="text-amber-400 r
            &nbsp;<span className="text-slate-40
            <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
          </div>
        </div>
      </div>
    </div>
  )
}
```

=====

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

=====

ФАЙЛ 72/82: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```

      </div>
    </main>
  </div>

  <script>
    function setMode(mode) {
      document.getElementById('btn-mode-normal').
        ? 'px-3 py-1 rounded text-sm font-bold l
        : 'px-3 py-1 rounded text-sm font-bold l
```

```

        if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
        if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
        if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
    });

    document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
    document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
    document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
    const mobileMenuBtn = document.getElementById('mobile-menu-btn');
    const mobileDrawer = document.getElementById('mobile-drawer');
    const closeDrawerBtn = document.getElementById('close-drawer-btn');
    const drawerOverlay = document.getElementById('drawer-overlay');

    function toggleDrawer() {
        const isClosed = mobileDrawer.classList.contains('hidden');
        if (isClosed) {
            mobileDrawer.classList.remove('hidden');
            if (drawerOverlay) {
                drawerOverlay.classList.remove('hidden');
                // Timeout allows display:block to take effect
                setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
            }
            document.body.style.overflow = "hidden";
        } else {
            mobileDrawer.classList.add('hidden');
            if (drawerOverlay) {
                drawerOverlay.classList.add('hidden');
            }
            document.body.style.overflow = "auto";
        }
    }

    mobileMenuBtn.addEventListener('click', toggleDrawer);
    closeDrawerBtn.addEventListener('click', toggleDrawer);
    drawerOverlay.addEventListener('click', toggleDrawer);
});

```

```
        const id = entry.target.id;
        document.querySelectorAll('aside')
            .forEach(link => {
                if (link.getAttribute('href') === id) {
                    link.classList.add('bg-gray-100');
                } else {
                    link.classList.remove('bg-gray-100');
                }
            });
    });
}, observerOptions);

document.querySelectorAll('section[id^="questions"]')
    .forEach(section => {
        observer.observe(section);
    });
});
</script>
</body>
</html>
```

ФАЙЛ 73/82: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1.5 КБ

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Пособие по алгебре: От пиццы к предикатам</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js"></script>
  <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.1/dist/mathjax.min.js"></script>
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [
          '\frac{1}{2}ab\sin C',
          '\sqrt{a^2 + b^2 - 2ab\cos C}',
          '\frac{a^3 + b^3}{a + b} = a^2 - ab + b^2'
        ],
        displayBlock: [
          'E=mc^2',
          'a^2 + b^2 = c^2'
        ],
        processEscapes: true
      },
      options: {
        skipHtmlTags: ['script', 'noscript', 'style', 'link']
      }
    };
  </script>
</head>
```



```

    mjx-container {
      max-width: 100% !important;
    }
    mjx-container[display="true"] {
      overflow-x: auto;
      overflow-y: hidden;
      max-width: 100%;
      scrollbar-width: none;
      -ms-overflow-style: none;
      -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar
      display: none;
  }

  @media (max-width: 640px) {
    html { font-size: 13px; }

    .uno-card { width: 40px; height: 60px; font-size: 12px; }
    .uno-card::before, .uno-card::after { font-size: 12px; }
    .uno-card::before { top: 1px; left: 2px; }
    .uno-card::after { bottom: 1px; right: 2px; }

    .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
    .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

    .uno-equation .text-2xl { font-size: 1.2rem; }
    .uno-equation.gap-4 { gap: 0.5rem !important; }

    /* Allow horizontal scroll for equations in containers
    .formula-scroll { flex-wrap: nowrap !important; }
  }

  .uno-inner {
    position: absolute; width: 110%; height: 75%;
    border-radius: 50%; transform: rotate(-30deg);
    box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
  }

```

```
        background-image: repeating-linear-gradient(
        background-attachment: local !important;
        font-family: "Comic Sans MS", "Caveat", "Se

    }
    body.theme-notebook .bg-slate-900, body.theme-n
    late-700 {
        background-color: rgba(255, 255, 255, 0.7)
        border-color: #cbd5e1 !important;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme
        ant; }
    body.theme-notebook .text-slate-400 { color: #5
    body.theme-notebook .border-slate-600, body.the
        : #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "
    body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
```

```

</marker>
<marker id="arrow-pink" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#f472b0" stroke="none"/>
</marker>
<marker id="arrow-cyan" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4c1" stroke="none"/>
</marker>
<marker id="arrow-indigo" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="none"/>
</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="none"/>
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="none"/>
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
  <rect x="0" y="0" width="40" height="20" fill="yellow" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-red">
  <rect x="0" y="0" width="40" height="20" fill="red" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-rem">
  <rect x="0" y="0" width="40" height="10" fill="red" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">

```

```
nslate-x-full transition-transform duration-300  
w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto  
<div class="h-full flex flex-col p-6 overflow-hidden  
lg:rounded-2xl lg:border lg:border-slate-700">  
  <div class="flex justify-between items-center">  
    <h3 class="font-black text-transparent bg-clip-text  
bg-gradient-to-r from-indigo-500 to-purple-500  
border border-indigo-500 rounded p-2 text-sm font-medium  
id="close-drawer-btn" class="text-white font-medium text-sm">  
    Содержание
```

```
</h3>  
<button id="close-drawer-btn" class="text-white font-medium text-sm">  
  <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">  
    <path stroke-linecap="round" stroke="white" stroke-width="2">  
    </path>  
  </svg>  
</button>  
</div>  
<nav class="space-y-3 text-sm font-medium">  
  <div class="group">  
    <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-blue-300">  
      1. Алгебраические структуры  
    </summary>  
    <div class="pl-6 space-y-2 text-sm">  
      <a href="#section-1-1" class="text-indigo-500">1.1. Группы</a>  
      <a href="#section-1-2" class="text-indigo-500">1.2. Кольца</a>  
      <a href="#section-1-3" class="text-indigo-500">1.3. Поля</a>  
      <a href="#section-1-4" class="text-indigo-500">1.4. Модули</a>  
    </div>  
  </div>  
</nav>
```

```
<div class="group">  
  <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">  
    2. Комплексные числа  
  </summary>  
</div>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
    5. Группа корней из единицы
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-fuchsia-500">
    <a href="#q5-def" class="block hover-fuchsia-500">
    <a href="#q5-group" class="block hover-fuchsia-500">
    <a href="#q5-prim" class="block hover-fuchsia-500">
    <a href="#q5-crit" class="block hover-fuchsia-500">
    <a href="#q5-cyclic" class="block hover-fuchsia-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-500">
    6. Делимость, НОД и Евклид
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-lime-500">
    <a href="#q6-1" class="block hover-lime-500">
    <a href="#q6-2" class="block hover-lime-500">
    <a href="#q6-3" class="block hover-lime-500">
    <a href="#q6-4" class="block hover-lime-500">
    <a href="#q6-5" class="block hover-lime-500">
    <a href="#q6-6" class="block hover-lime-500">
  </div>
</details>
```

```
<details class="group">
  <summary class="list-none cursor-pointer-pink-500 pl-3 font-bold text-pink-500">
    7.
  </a>
  </summary>
  <div class="pl-6 space-y-2 text-pink-500">
    <a href="#q7-1" class="block hover-pink-500">
    <a href="#q7-2" class="block hover-pink-500">
    <a href="#q7-3" class="block hover-pink-500">
    <a href="#q7-4" class="block hover-pink-500">
    <a href="#q7-5" class="block hover-pink-500">
    <a href="#q7-6" class="block hover-pink-500">
  </div>
</details>
```

```

        <a href="#q9-1" class="block">
        <a href="#q9-2" class="block">
        <a href="#q9-3" class="block">
        <a href="#q9-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
        <a href="#question-10" class="block">
            10. Делимость и Схема Г
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-rose-400">
        <a href="#q10-1" class="block">
        <a href="#q10-2" class="block">
        <a href="#q10-3" class="block">
        <a href="#q10-4" class="block">
        <a href="#q10-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
        <a href="#question-11" class="block">
            11. НОД и НОК многочленов
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-cyan-400">
        <a href="#q11-1" class="block">
        <a href="#q11-2" class="block">
        <a href="#q11-3" class="block">
        <a href="#q11-4" class="block">
    </div>
</details>

```

```

        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q14-1" class="block">
        <a href="#q14-2" class="block">
        <a href="#q14-3" class="block">
deg)</a>
        <a href="#q14-4" class="block">
ition-colors border border-rose-500/30"> K
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer">
        <a href="#question-15" class="block border-cyan-500 pl-3 font-bold text-cyan-400">
            15. Производная. Критер
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q15-1" class="block">
        <a href="#q15-2" class="block">
        <a href="#q15-3" class="block">
        <a href="#q15-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer">
        <a href="#question-16" class="block border-red-500 pl-3 font-bold text-red-400">
            16. Неприводимые многоч
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q16-1" class="block">
        <a href="#q16-2" class="block">
        <a href="#q16-3" class="block">

```

```

        if (detail.open) {
            document.querySelectorA
                if (other !== detail
                    other.removeAtt
                }
            });
        }
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting
                visibleId = entry.t

                // Highlight active
                document.querySelec
                    a.style.fontWeig
                    if (a.href.ends
                        a.style.fon
                }
            }
        });
    });
});

```



```
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузеров, экспорт в PDF осуществляется через меню веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

        modal.appendChild(box);
        document.body.appendChild(modal)
    }

```

```

function downloadMD() {
    let clone = document.querySelector('#clone')

    // Smart markdown parsing based on https://github.com/tekyo/markdown-it
    // Smart markdown parsing based on https://github.com/tekyo/markdown-it
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.classList.contains('img-caption')) {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        el.remove();
    });

    let text = clone.innerHTML;
    text = text.replace(/\n{3,}/g, '\n\n');

    if (window !== window.top) {
        fallbackModal(text, 'Markdown');
        return;
    }

    const blob = new Blob([text], { type: 'text/markdown' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = 'vyisshaya_algebra.md';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
}

```

```

body.bg-white [class*="f
nt; }

body.bg-white [class*="f
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre

```

```

        </button>
        <a href="algebra.pdf" target="_blank">
- sm font-bold flex items-center justify-center
          <span> </span> PDF
        </a>
        <button onclick="downloadMD()">
- bold flex items-center justify-center gap-2
          <span> </span> Markdown
        </button>
        <button onclick="downloadHTML()">
t-sm font-bold flex items-center justify-center
          <span> </span> HTML
        </button>
      </div>

<!-- Смена темы -->
<div>
  <div class="text-xs text-slate-500">
    <div class="flex items-center gap-2">
      <button id="theme-dark" onclick="toggleTheme('dark')">
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
      <button id="theme-light" onclick="toggleTheme('light')">
r justify-center text-amber-50 hover:text-white
">* </button>
      <button id="theme-terminal" onclick="toggleTheme('terminal')">
- center justify-center text-green-400 hover:text-white
терминала"> </button>
    </div>
  </div>
</div>
</div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">
- base min-w-0">

```

```

<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4"
      <div class="grid grid-cols-3 gap-4"
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tran
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tran
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 2: Базовые операции / Евкли
        <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tran
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t

```

```

        </a>

        <!-- Row 5: Корни / Уравнения -->
        <div class="hidden md:block"></div>
        <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <div class="hidden md:block"></div>

        <!-- Row 7: Производная -->
        <div class="hidden md:block"></div>
        <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <div class="hidden md:block"></div>

        <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
        <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл
        <a href="#question-17" onclick="setM

```

```

        <div class="font-bold text-slate-500">
            </a>
        </div>
    </div>

    <!-- ЛИКБЕЗ SECTION INSERTED HERE -->
    <section id="proof-algorithms" class="mb-6">
        <div class="flex items-center mb-6">
            <div class="bg-red-500/20 p-3 rounded" style="width: 100px; height: 100px; display: flex; align-items: center; justify-content: center; margin: 0 auto;">
                <svg class="w-8 h-8 text-red-500" data-bbox="110 400 190 480">
                    <path stroke-linecap="round" stroke-width="2" data-bbox="110 400 190 480">
                        110 400m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4a2 2 0 110-4m0 4
                    </path>
                </svg>
            </div>
            <h2 class="text-3xl pr-4 border-l-2 border-l-slate-500">
                </div>

                <div class="prose prose-invert max-w-4xl">
                    <p class="text-lg">
                        Хватит тупить. Доказательств
                    </p>
                    <p>есть для того, чтобы жать на кнопки. Вот жес
                    </p>

                    <h2 class="text-2xl font-bold border-l-2 border-l-slate-500">
                        (Меметика)</h2>

                    <!-- Прямое -->
                    <div class="bg-slate-800/80 p-6">
                        <h3 class="text-xl text-blue-400">
                            <p class="text-sm text-slate-400">
                                <ul class="list-disc pl-5">
                                    <li><span class="text-white">
                                        k$).</li>
                                    <li><span class="text-white">
                                        <li><span class="text-white">
                                            ово.</li>
                                </ul>
                            </p>
                        </h3>
                    </div>
                </div>
            </div>
        </div>
    </div>
```

```
        <li><span class="text-w
усок  $k$ , замени его на жареного карася из
        </ul>
    </div>
```

```
    <!-- Двойная делимость -->
    <div class="bg-slate-800 p-6
        <h3 class="text-xl text-ora
        <p class="text-sm text-slate
        <ul class="list-disc pl-5 sp
            <li><span class="text-w
вверх по уравнениям).</li>
            <li><span class="text-w
рху вниз, показывая, что  $\Delta \vdots r_i$ 
            <li><span class="text-w
х. Твой  $d$  больше.</li>
        </ul>
    </div>
```

```
    <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
    <div class="grid grid-cols-1 md
        <div class="bg-slate-800 p-
            <h4 class="text-red-400
            <p class="text-sm text-
            <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
        </div>
```

```
        <div class="bg-slate-800 p-
            <h4 class="text-red-400
            <p class="text-sm text-
            <p class="text-xs font-
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
        </div>
```



```

                <p class="text-sm text-gray-500">
                    x двух, потом последних. <br>
                <code class="text-x">
            </div>
        </li>

        <li class="bg-slate-800/50 p-4">
            <span class="text-2xl">
                <div>
                    <h4 class="text-cyan">
                        <p class="text-sm text-gray-500">
                            есть сложение – есть ноль. Если есть умножение
                        <code class="text-x">
                    </div>
                </li>

                <li class="bg-slate-800/50 p-4">
                    <span class="text-2xl">
                        <div>
                            <h4 class="text-cyan">
                                <p class="text-sm text-gray-500">
                                    авь их драться. Умножение всегда врывается
                                <code class="text-x">
                            </div>
                        </li>
                    </ul>
                </div>
            </section>
        </div>

        <div id="questions-container">
            <!-- ===== ВОПРОС 1 =====>
```

```

    try {
      const rules = (sheet as CSSStyleSheet).cssRules;
      parts.push(Array.from(rules).map((r) => r.cssText));
    } catch {
      const href = (sheet as CSSStyleSheet).href;
      if (!href) continue;
      try {
        const res = await fetch(href);
        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен – пропускаем */
      }
    }
  }
  return parts.join("\n");
}

```

```

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }

```

```

    return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
    if (!termIds.length) return "";
    const items = termIds
        .map((id) => {
            const e = GLOSSARY_BY_ID.get(id);
            if (!e) return "";
            return `<div class="export-term" id="term-${escapeHtml(id)}">
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>` : ""}
${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
</div>`;
        })
        .join("\n");

    return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
    ны сюда.</p>
    ${items}
</section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; extraCss: string; }) {
    const stamp = new Date().toLocaleString("ru-RU");
    return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>

```



```

import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("VIZUALIZATORY"), registry.length);
const vizIds = new Set([...registryBody.matchAll(/^\\s{2}[a-zA-Z0-9_]+/g)].map(m => m[0]));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
  };
});

```

ФАЙЛ 77/82: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossa",
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m",
  external: ["react", "react-dom", "motion", "lucide-re",
});

const { chapters } = await import(path.join(ROOT, "tmp/"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;
```

ФАЙЛ 78/82: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
    info: {
```

```
* ШАГ 1: КОД -> TXT
*
* Собирает ВСЕ исходный код репозитория в один текст
* public/export/full_code_all_pages.txt
*
* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel))
```



```

return files
  .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
  .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
  .filter((rel) => CODE_EXT.has(path.extname(rel)))
  .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
  .sort((a, b) => {
    const ia = ORDER.indexOf(categoryOf(a));
    const ib = ORDER.indexOf(categoryOf(b));
    return ia !== ib ? ia - ib : a.localeCompare(b);
  });
}

/** Достаём заголовки глав из данных пособия (без запуска) */
function extractChapters(files) {
  const chapters = [];
  const seen = new Set();
  for (const rel of files.filter((f) => f.startsWith("src/"))) {
    const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
    let m;
    while ((m = re.exec(src)) !== null) {
      const [, id, title] = m;
      if (seen.has(id)) continue;
      seen.add(id);
      chapters.push({ id, title, file: rel });
    }
  }
  return chapters.sort((a, b) => {
    const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
    const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
    return na - nb;
  });
}

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
}

```

```

    push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`)
  });
  push();
  push();

  current = null;
  files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
      current = cat;
      push(HR);
      push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
      push(HR);
      push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel), "utf8");
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`файлов:  ${files.length}`);
  console.log(`глав:    ${chapters.length}`);
  console.log(`строк:   ${txt.split("\n").length}`);

```

```
    monochrome: process.env.EXPORT_GRAYSCALE !== "1",
    size: "A4",
    font: "DejaVu-Sans-Mono",
    fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
    pointsize: 8,
    /** Максимум символов в строке (далее – жёсткий пере
    cols: 138,
    /** Строк содержимого на странице (+2 служебные: шапк
    rows: 76,
  };

  export const HR = "=".repeat(PAGE.cols);
  export const SUBHR = "-".repeat(PAGE.cols);

  export function ensureDir(dir) {
    fs.mkdirSync(dir, { recursive: true });
  }

  export function humanSize(bytes) {
    if (bytes < 1024) return `${bytes} B`;
    if (bytes < 1024 * 1024) return `${(bytes / 1024).toF
    return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
  }

  /** ImageMagick 7 (`magick`) или 6 (`convert`). */
  export function imageMagickCmd() {
    for (const cmd of ["magick", "convert"]) {
      try {
        execFileSync(cmd, ["-version"], { stdio: "ignore"
        return cmd;
      } catch {
        /** пробуем следующий */
      }
    }
    throw new Error(
      "ImageMagick не найден. Установите его: sudo apt-ge
    );
  }
```

```

    first = false;
  }
  return parts;
}

function paginate(txt) {
  const source = txt.replace(/\r\n/g, "\n").split("\n");
  const pages = [];
  for (let i = 0; i < source.length; i += PAGE.rows) {
    pages.push(source.slice(i, i + PAGE.rows));
  }
  return pages;
}

async function main() {
  if (!fs.existsSync(TXT_PATH)) {
    throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
  }

  const im = imageMagickCmd();
  const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

  fs.rmSync(PAGES_DIR, { recursive: true, force: true });
  ensureDir(PAGES_DIR);

  const total = pages.length;
  const jobs = pages.map((lines, idx) => {
    const num = String(idx + 1).padStart(4, "0");
    const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });
});

```

```
-----
  console.log("[2/3] txt -> png");
  console.log(`      страниц: ${total}`);
  console.log(`      выход:   ${path.relative(ROOT, PAGE)}
}

main().catch((err) => {
  console.error("Ошибка рендера страниц:", err.message);
  process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

ФАЙЛ 82/82: [.github/workflows/rebuild-pdf.yml](#)
КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

```
# Пайплайн пересборки PDF при каждом коммите:
#
#   исходный код  →  full_code_all_pages.txt  (script)
#                   |
#                   →  page-0001.png ... page-NNNN.png  (
#
#                                     →  full_code_all_page
#
# Готовые TXT и PDF коммитаются обратно в ветку (их отда
# промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
  push:
    branches:
      - "*/"
    paths-ignore:
```

- ```
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PATTERN\}" rights="none"/rights="none" pattern="\{PATTERN\}" rights="write,write-through"/' "$policy"
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.DENSITY:-300}  
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full code all pages` | \$(du -sh \$GITHUB\_WORKSPACE/public/export/full code all pages) |"  
 }

Универсальное пособие • полный исходный код

---